Federal Office
for Information Security

BSI Technical Guideline TR-03135-3

Machine Authentication of MRTDs for Public Sector Applications

Part 3: High Level Document Check Interface Specification

Version 5.0.0

Federal Office for Information Security
PO Box 20 03 63
53133 Bonn
E-Mail: tr-03135@bsi.bund.de
Internet: https://www.bsi.bund.de

Table of Contents

List of Figures

1 Changelog Part 3

The following tables present the changes introduced between the latest versions of this Technical Guideline. The changelog lists the changes grouped per part of this Technical Guideline, per element (section, table, figure) and type of change (refer to [KeepAChangelog]):

- *Added* for new features
- *Changed* for changes in existing functionality
- *Deprecated* for soon-to-be removed features
- *Removed* for now removed features
- *Fixed* for any bug fixes
- *Security* in case of vulnerabilities

1.1 Version 5.0.0

Element Name	Type of Change	Change Description
Chapter 3	Added	Overview description of the client-server architecture and example High Level Document Check (HLDC) workflow definition, updated diagrams
Chapter 3	Changed	Updated diagrams of the client-side document check process
Chapter 4	Changed	New schemata: • Updated filenames • Updated namespaces
Chapter 4	Added	New schemata: • log_hldc_shared-v5_0_0.xsd • hldc_shared_v5.xsd
Chapter 5	Removed	Document detection: • Removed waitForNewDocument operation for explicit document detection
Chapter 5	Changed	Workflow execution and document detection: • Implicit document detection in beginWorkflow • Mandatory transaction id in beginWorkflow • Skip document detection by triggerWorkflow • User interactions with continueWorkflow
Chapter 5	Added	Workflow feedback: • Added live feedback
Chapter 5	Changed	Cross-document check: • Changed naming: cross-checks are not considered as combined checks
Chapter 5	Changed	Transaction management: • Mandatory transaction id in beginWorkflow • Transaction versioning with getSupportedVersions
Chapter 6	Changed	Filenames and namespaces: • Updated filenames and namespaces
Chapter 6	Added	FeedbackStatus: • Added new enum value err-not-supported

Element Name	Type of Change	Change Description
Chapter 6	Fixed	FeedbackStatus: • Inconsistent enum value name err-reference-document-required • Missing enum value err-reference-document-required in guideline
Chapter 6	Added	WorkflowFeedback: • Added LiveFeedback element • Added error code element
Chapter 6	Added	WorkflowStatus: • Added waiting status
Chapter 6	Added	Data types: • Added ContinueWorkflowParameter • Added LiveFeedback and child elements
Chapter 6	Removed	Fault types: • Removed InvalidClientId fault
Chapter 6	Added	Fault types: • Added InvalidOperation fault • Added InvalidParameter fault
Chapter 6	Changed	beginWorkflow: • The transaction id is mandatory • Added optional parameter for binary data
Chapter 6	Removed	Operations: • Removed waitForNewDocument operation
Chapter 6	Added	Operations: • Added triggerWorkflow operation • Added continueWorkflow operation
Section 6.5	Removed	Data types: • Removed RequestedSchemaVersions and child types
Section 6.5	Added	Data types: • Added type.workflow.sequences for encapsulation • Added type.workflow.scenario.checkconfiguration for encapsulation • Added type.workflow.scenario.livestream and child elements • Added type.workflow.scenario.interactions and child elements
Section 6.5	Changed	Combined/cross-document checks: • Separated cross-document checks from combined checks
Section 6.5	Fixed	Data types: • Fixed wf:type.workflow.conditions.mrz attribute type in schema • Fixed typo in type.workflow.scenario.check.action value
Section 6.5	Changed	Data types: • Added datatype for optical fields which allow predefined and custom fieldnames • Changed type of id in type.workflow.scenario.check • Changed applications in type.workflow.electronic.readseq.datagroup.application

Element Name	Type of Change	Change Description
▸Chapter 7	Added	Data types: • Added datatype for all barcode positions of a document per page • Added datatype for all barcode positions of the frontside of a document • Added datatype for all barcode positions of the backside of a document • Added datatypes for user interactions
▸Chapter 8	Changed	Namespace and filename: • Updated namespace and filename
▸Chapter 8	Added	Data types: • Added TR03135Version type for version selection
▸Chapter 8	Added	Operations: • Added getSupportedVersions operation
▸Chapter 8	Changed	Operations: • Using TR-03135 version instead of namespace and schemaversion in beginTransaction operation
▸Chapter 9	Added	Data types: • Added descriptions of shared data types
Chapter introduction	Added	Added section on publication language

Table 1.1 Version 5.0.0

1.2 Version 2.5

Element Name	Type of Change	Change Description
▸Chapter 6	Added	New workflow elements: • AggregatedCheck • SealCheck
▸Chapter 7	Added	New feedback elements: • type.feedback.seal.visa • type.feedback.seal.visa.durationofstay • type.feedback.seal.emergencytraveldocument • type.feedback.seal.arrivalattestationdocument • type.feedback.seal.socialinsurancecard • type.feedback.seal.socialinsurancecard.socialinsurancenumber • type.feedback.seal.residencepermit • type.feedback.seal.residencepermitsupplementarysheet • type.feedback.seal.addressstickergermanidentitycard • type.feedback.seal.unknown • type.feedback.mrz.unfolded • type.feedback.mrz.unfolded.shortened

Table 1.2 Version 2.5

2 Introduction

This technical guideline specifies two complimentary web services that provide validation of Machine-Readable Travel Documents (MRTDs) according to [BSI TR-03135-1]. They comprise the document check process as well as the required logging of the results.

2.1 Motivation

The checking of Machine Readable Travel Documents (MRTDs) according to [BSI TR-03135-1] requires a large number of individual steps. Some steps require a certain order of execution or may only be relevant for particular documents. In contrast to a fixed definition of scenario-specific check processes, the processing and visualisation of results and/or document data may vary with the purpose of the Inspection Application. The complexity of the document check process causes large efforts for the creation and quality control of Inspection Applications.

The goal of this document is to provide high-level interfaces that reduces the programming effort for Inspection Applications by separating the document check process and standard-compliant logging from the problem-specific processing and visualization of results.

2.2 Publication Language

This Technical Guideline is only published in English. The reasons for this are as follows:

- Relation to other English only publications (esp. [ICAO9303], [BSI TR-03129-2], [BSI TR-03121] and [BSI TR-03110])
- The content is relevant in a European context.

3 Architecture for Inspection Applications

This chapter provides an overview of the architecture used in document checks compliant with [BSI TR-03135].

3.1 Client-Server Architecture

The following actors are involved in the process:

- Document reader component: according to [BSI TR-03135-1], section 3.2
- Inspection application: the frontend or client according to [BSI TR-03135-1], section 3.3
- User(s): the person or people handling the inspection application and the documents which are to be read[1].

The inspection applications are split into two parts - frontends and backends. Frontends, or clients, are application-specific. They provide a user interface and process and visualise results depending on their particular purpose, cf. [BSI TR-03135] part 2. The document check process and the corresponding transaction logging are provided by a backend server (referred to as HLDC5 server) via web services. This has several benefits, such as:

- There exists a clear separation of responsibilities between the frontend and the backend.
- The communication between the frontend and backend is carried out in accordance to a well-defined standard interface, see following sections of this document. This reduces the effort of the implementation, e.g. due to the possibility of generating certain code segments automatically.
- A single backend implementation can be used by multiple frontend implementations with different application scenarios.

3.2 Workflow-Based Document Check

To further reduce the effort for client implementation, the document check process is configured based on a textual description in Extensible Markup Language (XML) rather than program code. A particular configuration is referred to as workflow, its description as workflow definition. This workflow definition allows the customization of the application-scenario-specific document check process within the limits of this technical guideline.

The workflow definition can be provided by either the client or the server. This allows clients to use custom workflows, but also facilitates centralized management on the server. All workflows are managed by the server and are available to all clients. Workflows are executed on the server upon request by the client. Relevant feedback elements are generated on the server and can be fetched by the client.

1 Depending on the context, the user(s) could be government officials (e.g. in stationary application scenarios) or the travellers (e.g. in self service or kiosk application scenarios).

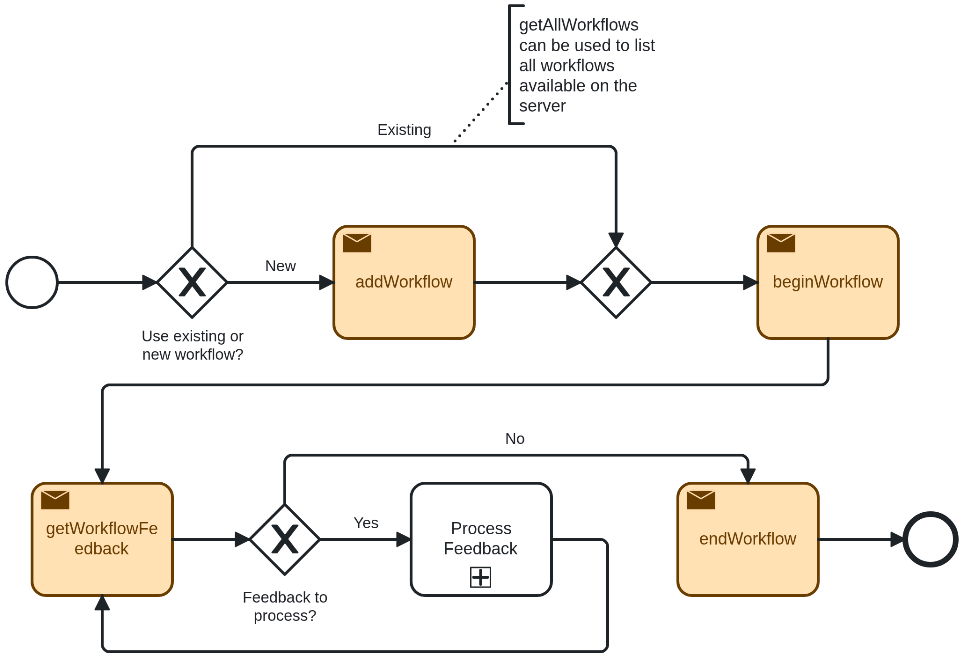

Figure 3.1. Client-side document check process

Figure 3.1 visualises the client side of the document check process. Workflow-related interface operations are highlighted in orange. After a new document is detected, the client requests execution of the relevant workflow. Subsequently, the client individually fetches the feedback elements that are defined in the workflow from the server and processes them appropriately, see Section 3.2.2. After all feedback has been processed successfully, the client informs the server that client-side processing has finished and that the workflow may be completed. Alternative cases of workflow completion (e.g. cancellation and timeouts) are described in Section 3.2.3 and Section 6.4.4.

3.2.1 Workflow initialization

The HLDC5 process is carried out and controlled using the operations outlined in Section 6.4.

The client may register a workflow definition with the HLDC5 server using `addWorkflow` (see Section 6.4.1). Alternatively, an existing workflow definition (i.e. one which has been pre-installed or previously registered on the HLDC5 server) can be used. The list of workflows available on the server can be retrieved by the client using `getAllWorkflows` (see Section 6.4.5); existing workflows can be removed using `removeWorkflow` (see Section 6.4.9)

A workflow definition controls which optical, electronic, digital seal, and combined checks are carried out and which data are required for processing. Please refer to Section 6.5 for an exhaustive specification of the workflow definition. An example workflow definition is provided and discussed in Section 3.5.

The client starts the chosen workflow using `beginWorkflow` (see Section 6.4.2). Depending on the application scenario, the operation inputs may additionally include the Machine Readable Zone (MRZ)/Card Access Number (CAN) if it has been provided by the user. Once the start of the workflow has been confirmed by the HLDC5 server, the client enters a feedback loop, see Section 3.2.2. In some cases, e.g. automatic document detection not working properly, the client may choose to force the processing using `triggerWorkflow` operation (see Section 6.4.10).

3.2.2 Feedback loop

During the feedback loop, the client repeatedly retrieves feedback elements from the HLDC5 server via the `getWorkflowFeedback` operation (see Section 6.4.6 and has to react and/or process the retrieved data accordingly. Please refer to Chapter 7 for the specification of the feedback data format. The feedback elements can be broadly divided into the following categories:

- Information: the data which has been read optically and/or electronically from the document such as the MRZ and the contents of the chip, as well as the results of the performed document checks and the associated defects. The client may process and visualise the data to the user.
- Interaction: a request from the server for a particular action to be performed (typically by the user). Once the action has been performed, the client sends the necessary information back to the HLDC5 server.
- Live feedback: provides additional support for correct positioning of the document on the document reader in the form of live video stream and/or hints for the user. Furthermore, the server can inform the client whether or not certain parts of the process (e.g. reading the optical and electronic data and performing the associated checks) have been completed.

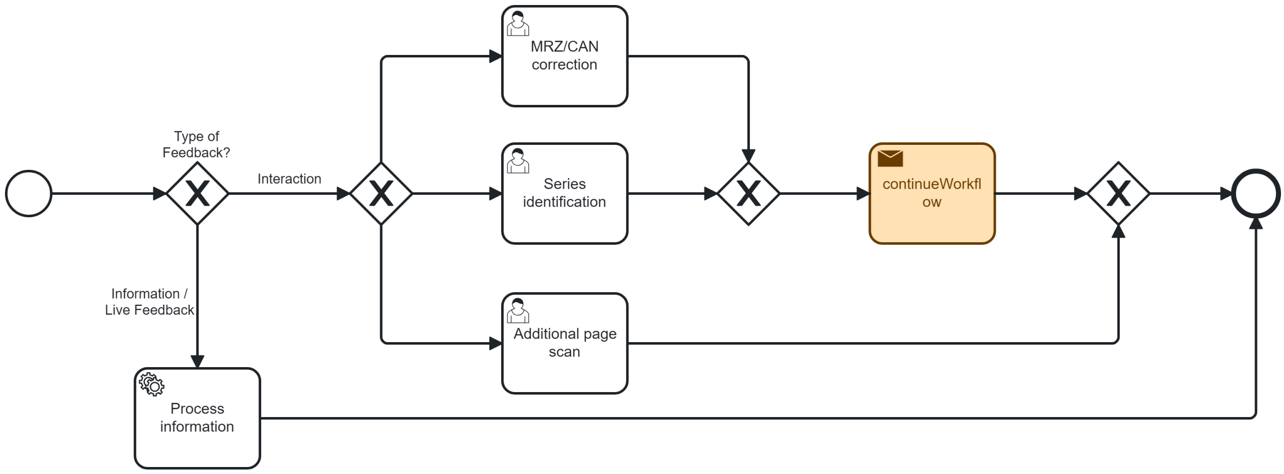

Figure 3.2. Client-side processing of feedback elements ("Process Feedback" step in Figure 3.1, Figure 3.3, and Figure 3.4)

An important distinction to make is that while feedback elements of the category "information" will necessarily occur in basically any error-free process, the feedback elements of the category "interaction" may or may not occur depending on the scanned document data. The elements of the category "live feedback" may or may not occur depending on the workflow. The possible "interaction" scenarios are described in the sections below.

3.2.2.1 MRZ/CAN correction

During the process of reading a document, it may be necessary for the user to correct the MRZ/CAN.

The HLDC5 server attempts to ensure that the MRZ/CAN as provided by the client or as read from the document by the document reader is correct. The purpose of this check is twofold:

- Optical Character Recognition (OCR): Ensuring that the MRZ/CAN contents have been read correctly, i.e. as printed on the document.
- Chip: Ensuring that the electronic chip (if present) can be accessed using the MRZ/CAN.

If the HLDC5 server determines the MRZ/CAN not to be correct, it requests a correction to be provided by the client. This request contains the current MRZ/CAN as an image and the recognised text, the purpose of the correction ("OCR" or "Chip") as well as other optional hints. The client may display this information to the user in an MRZ/CAN correction dialog (or other suitable form) and waits for the user input. Once the corrected MRZ/CAN has been provided by the user, it is sent to the HLDC5 server via the `continueWorkflow` operation (see Section 6.4.11). Whereas the correction with the purpose "OCR" may only take place once, the correction with the purpose "Chip" may happen multiple times, i.e. the HLDC5 server-side check and client-side correction dialog repeat until the MRZ/CAN is considered acceptable by the server.

It should be noted, that ensuring the MRZ/CAN has been read correctly does not necessarily ensure that the chip can be accessed, i.e. it is possible to perform a correction with the purpose "OCR" and thereafter the correction loop with the purpose "Chip".

Alternatively, the user may decide to skip the MRZ/CAN correction and continue the workflow with the incorrect (according to the HLDC5 server) MRZ/CAN data. In certain application scenarios (e.g. self service) it may be decided for the client to automatically skip the MRZ/CAN correction altogether and/or cancel the workflow. Furthermore, both the client and the HLDC5 server may define timeouts in case no user interaction takes place. The HLDC5 server timeout (if defined) may lead to the workflow being cancelled, whereas the client-side timeout leads to the workflow being continued without the MRZ/CAN correction.

In any case, the HLDC5 server logs the results of the MRZ/CAN correction(s) and proceeds with the workflow, unless it has been cancelled.

3.2.2.2 Document series identification

During the process of reading a document, it is necessary to identify the document series in order to be able to perform document specific checks.

If the HLDC5 server was unable to automatically identify the series of the scanned document, e.g. based on the images and/or the MRZ of the document, it requests a series identification to be provided by the client. This request contains additional information such as the scanned images of the document, the document type and country code, and a list of selection options, i.e. a known document series most closely matching the document under inspection. The client may display this information to the user in a document series identification dialog (or other suitable form) and waits for the user input, i.e. the selected series which is subsequently sent to the HLDC5 server via the `continueWorkflow` operation (see ▸Section 6.4.11).

Alternatively, the user may decide to deliberately skip the series identification and continue the workflow with an unidentified, i.e. an unknown, document series. In certain application scenarios (e.g. self service) it may be decided for the client to automatically skip the series identification altogether and/or cancel the workflow. Furthermore, both the client and the HLDC5 server may define timeouts in case no user interaction takes place. The HLDC5 server timeout (if defined) may lead to the workflow being cancelled, whereas the client-side timeout leads to the workflow being continued without the series identification.

In any case, the HLDC5 server logs the results of the document series identification and proceeds with the workflow, unless it has been cancelled.

3.2.2.3 Additional document page

For certain documents (e.g. ID cards), it may be necessary to scan a second page of the document. More generally, a workflow definition may require additional pages of documents to be scanned during the process.

In such cases, the HLDC5 server requests an additional page of a document to be scanned. This request may carry metadata identifying which document page has to be placed by the user. Once the user has placed the requested page of the document on the document reader, the workflow is continued implicitly, i.e. the client can fetch new feedback elements as they are provided by the HLDC5 server. The client may skip the automatic document detection and force the processing using `triggerWorkflow` operation (see ▸Section 6.4.10). Furthermore, both the client and the HLDC5 server may define timeouts in case no user interaction takes place.

Note that the aforementioned subprocesses, i.e. MRZ/CAN correction and document series identification (▸Section 3.2.2.1 and ▸Section 3.2.2.2, respectively), may have to be performed multiple times depending on how many and which pages of the document have been scanned.

3.2.3 Workflow finalization

Once the client has ended the feedback loop, i.e. all feedback elements have been fetched and processed, it may end the workflow execution by using `endWorkflow` (see ▸Section 6.4.4). The client may also start other workflows (e.g. to inspect a secondary document) and optionally link them using `linkWorkflow` (see ▸Section 6.4.8) in order to perform cross-document checks, see ▸Section 3.4. Alternatively, a workflow may be cancelled by the client at any time using `cancelWorkflow` (see ▸Section 6.4.3).

In case a workflow execution is not ended correctly as described above (e.g. timeouts), the HLDC5 server may act as described in ▸Section 6.4.4.

3.3 Transaction Logging

[BSI TR-03135-1] requires logging of all relevant check results which are provided by the interface specified in ▸Chapter 8.

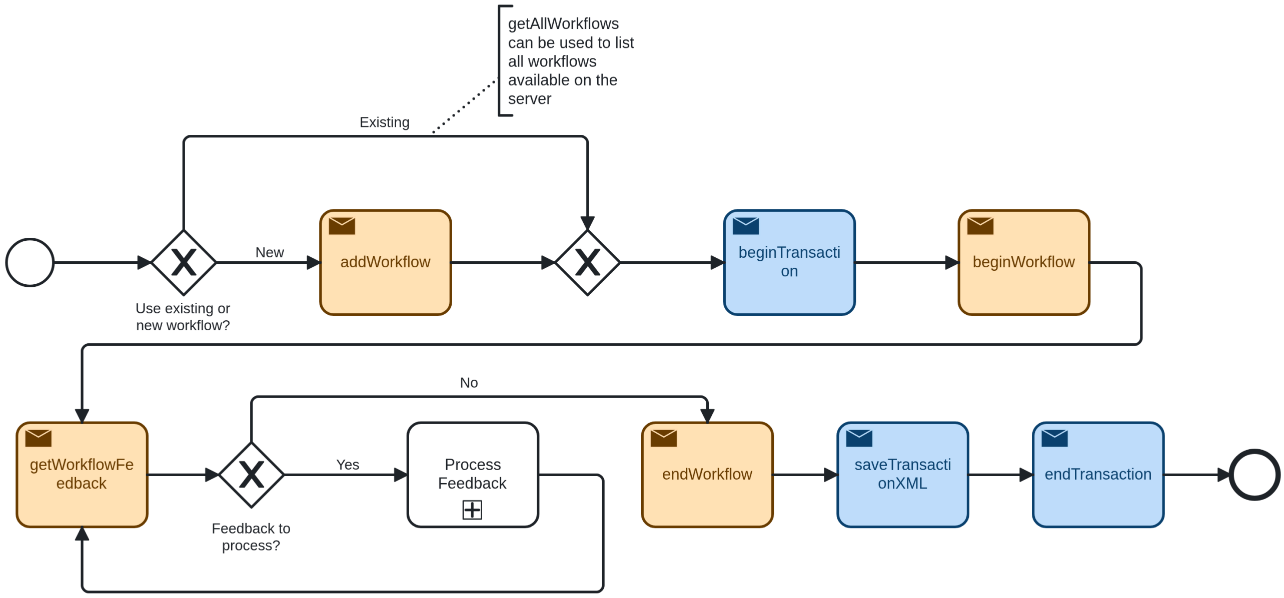

Figure 3.3. Extension of Figure 3.1 accounting for logging compliant with [BSI TR-03135-1]

Figure 3.3 shows the extension of the client side of the document check process from Figure 3.1 with transaction logging operations (highlighted in blue). The client initiates a new transaction after a MRTD was detected and before initiating workflow execution. After the workflow execution and result processing is finished, the client requests saving of the corresponding transaction log and then ends the transaction. Please refer to Section 5.2 for an overview and details of the transaction interface functions.

3.4 Cross-document check

[BSI TR-03135-1] specifies checks across two different documents (e.g. passport and visa). Figure 3.4 shows the extension of the single document checking process in Figure 3.3 which accounts for potential cross-document checks.

Initially, both documents (e.g. a passport and a visa) are read and checked individually in separate transactions and workflows. It is the responsibility of the client to control and decide which documents are considered for cross-document checks. After linking the respective workflows, the cross-document checks are performed. Subsequently, the corresponding transactions are merged to ensure consistent logging of the checking process. Finally, the client requests saving the log of the merged transaction and then ends the transaction.

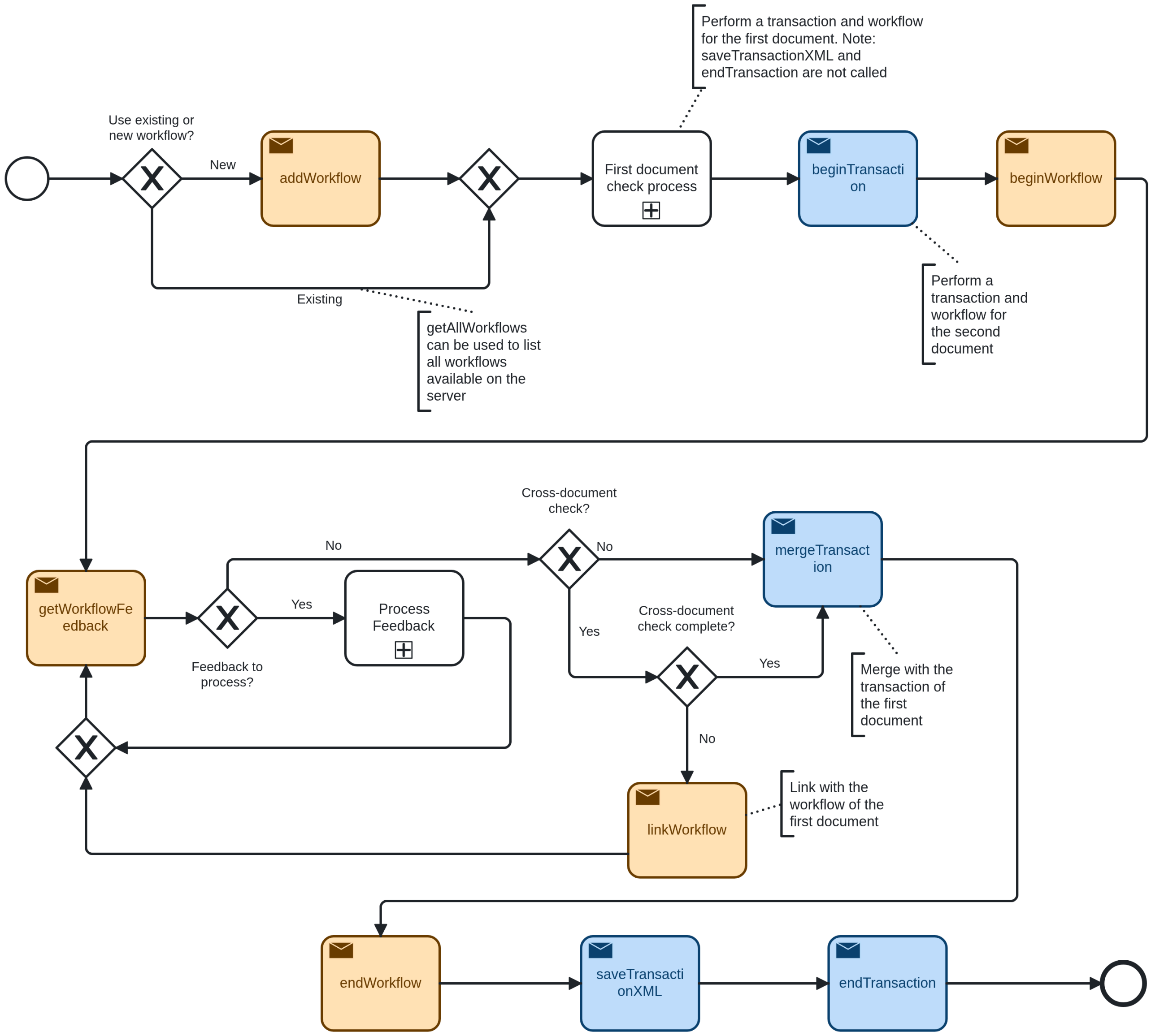

Figure 3.4. Extension of ▸Figure 3.3 accounting for cross-document checks.

3.5 Sample HLDC5 Workflow

Below, an example workflow definition is shown. Brief explanations of the individual elements or element groups are provided within the XML comment tags, i.e. `<!-- Text -->`. The order of the XML elements within a workflow definition is partially enforced by the schema and/or some XML validation tools. The HLDC5 server, however, is not bound by this ordering; hence, the order in which parts of the workflow are executed and in which the feedback elements are returned to the client may differ from the order in the workflow definition. Furthermore, the server may parallelise certain parts of the workflow execution. A complete reference of the HLDC5 and feedback schema is given in ▸Chapter 6 and ▸Chapter 7, respectively.

```
<?xml version="1.0" encoding="UTF-8"?>
<wf:Workflow xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:schemaLocation="http://
trdoccheck.bsi.bund.de/hldc/workflow/5 ../../hldc/wf_v5.xsd" xmlns:wf="http://
trdoccheck.bsi.bund.de/hldc/workflow/5" schemaVersion="1">
    <wf:Information>
        <wf:Vendor>Vendor name</wf:Vendor>
        <wf:Name>Sample workflow</wf:Name>
        <wf:Version>1.0.0.0</wf:Version>
        <wf:Description>Example HLDC5 workflow definition for reading basic document data and
 performing basic document checks</wf:Description>
    </wf:Information>
    <wf:ApplicationScenario preset="stationary"/>
    <wf:Conditions>
```

```
        <!-- define a condition checking if a document is a German ID card -->
        <wf:MRZCondition id="isGermanID" type="ID" issuer="D"/>
        <!-- define a condition checking if a document is a visa -->
        <wf:MRZCondition id="isVisa" type="V."/>
    </wf:Conditions>
    <wf:CheckSequences>
        <wf:ElectronicCheck>
            <wf:AccessConfiguration>
                <!-- always do PACE for German ID cards -->
                <wf:ProtocolOrder condition="isGermanID" order="PACE"/>
            </wf:AccessConfiguration>
            <wf:ChipConfiguration>
                <!-- wait for max 5s to detect a chip -->
                <wf:WaitForChip timeInMs="5000" intervalInMs="500"/>
            </wf:ChipConfiguration>
            <wf:ReadSequence>
                <!-- get chip detection result -->
                <wf:ChipDetection id="chipDetection"/>
                <!-- read EF.ATR/INFO before any data groups -->
                <wf:ElementaryFile id="efatrInfo" name="ef_atr_info"/>
                <!-- read DG1, DG2, DG3 -->
                <wf:Datagroup id="ICAO-DG1" application="ICAO-Chip" number="1"/>
                <wf:Datagroup id="ICAO-DG2" application="ICAO-Chip" number="2"/>
                <wf:Datagroup id="ICAO-DG3" application="ICAO-Chip" number="3" required="false"/>
                <!-- perform electronic checks -->
                <wf:ElectronicCheck id="eCheck"/>
            </wf:ReadSequence>
        </wf:ElectronicCheck>
        <wf:OpticalCheck>
            <wf:ReadSequence>
                <!-- read MRZ, facial image, and document scans -->
                <wf:Text id="OPT-MRZ" field="MRZ"/>
                <wf:Image id="CROP" field="IMG-FACE-PRIMARY-CROPPED"/>
                <wf:Image id="IR" field="IMG-IR-FRONT-CROPPED"/>
                <wf:Image id="UV" field="IMG-UV-FRONT-CROPPED"/>
                <wf:Image id="VIS" field="IMG-VIS-FRONT-CROPPED"/>
                <!-- perform optical checks -->
                <wf:OpticalCheck id="oCheck"/>
            </wf:ReadSequence>
        </wf:OpticalCheck>
        <wf:SealCheck condition="isVisa">
            <wf:ReadSequence>
                <!-- read digital seal data from a visa -->
                <wf:Data id="SEAL-VISA" profile="Visa"/>
                <!-- perform digital seal checks on a visa -->
                <wf:Check id="sCheck"/>
            </wf:ReadSequence>
        </wf:SealCheck>
        <wf:CombinedCheck>
            <wf:ReadSequence>
                <!-- perform combined checks -->
                <wf:CombinedCheck id="cCheck"/>
            </wf:ReadSequence>
        </wf:CombinedCheck>
    </wf:CheckSequences>
    <wf:Feedback>
        <!-- List of feedback elements. Depending on the document scanned during the actual
 process, some feedback elements may not be available (e.g. electronic data in a visa). To
 prevent this, conditions may be used to bind certain feedback elements to certain document
 types. -->
        <wf:Text id="ChipDetection" from="chipDetection"/>
        <wf:Text id="MRZ" from="OPT-MRZ"/>
        <wf:XML id="MRZData" from="OPT-MRZ"/>
        <wf:XML id="DG1Data" from="ICAO-DG1"/>
        <wf:XML id="SealVisa" from="SEAL-VISA" condition="isVisa"/>
        <wf:Image id="OpticalImage" from="CROP" format="bmp"/>
        <wf:Image id="ElectronicImage" from="ICAO-DG2" index="1" format="bmp"/>
        <wf:Image id="Fingerprint1" from="ICAO-
DG3" index="1" format="png" preserveAspectRatio="true"/>
        <wf:Image id="Fingerprint2" from="ICAO-
DG3" index="2" format="png" preserveAspectRatio="true"/>
        <wf:Image id="IrImage" from="IR" format="jpeg"/>
```

```
        <wf:Image id="UvImage" from="UV" format="jpeg"/>
        <wf:Image id="VisImage" from="VIS" format="jpeg"/>
        <!-- specify the log detail level, see section 4.3 in part 2 of this TR -->
        <wf:XML id="ElectronicCheckResult" from="eCheck" profile="basic"/>
        <wf:XML id="OpticalCheckResult" from="oCheck" profile="minimal-extended"/>
        <wf:XML id="CombinedCheckResult" from="cCheck" profile="minimal-extended"/>
        <wf:XML id="SealCheckResult" from="sCheck" profile="basic" condition="isVisa"/>
    </wf:Feedback>
</wf:Workflow>
```

4 Document overview

4.1 Terminology

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [RFC2119].

4.2 Naming conventions

4.2.1 Multiplicity

Generally, XML elements and attributes listed in this document are required, i.e. the respective parent element SHALL contain exactly one such element. Elements and attributes that deviate from this baseline are denoted in this document by a symbol which is appended to the element/attribute name. The symbols are listed in Table 4.1 .

Appended symbol	Meaning
?	Zero or one
*	Zero or more
+	One or more

Table 4.1 Multiplicity symbols

4.2.2 SOAP Interfaces

All operations of this interface follow the request/response model, i.e., communication is initiated by the client by sending a Simple Object Access Protocol (SOAP) message to the server (request). For each request, the server replies with a SOAP message containing the result of the requested operation (response) or, in case of error, a fault.

The body of each SOAP message consists of a single part which is named according to the corresponding operation. For requests, the part name is identical to the name of the operation. For responses, the part name is identical to the name of the operation plus the suffix “Response” (see Table 4.2).

Message type	Part name
Request	`<operation_name>`
Response	`<operation_name>`Response

Table 4.2 Naming convention for SOAP messages

Example: Naming convention

- Operation: `getAllWorkflows`
- Request: `getAllWorkflows`
- Response: `getAllWorkflowsResponse`

Both request and response elements exclusively contain zero or more child elements according to the detailed description in this guideline. They do not carry any attributes.

4.3 Namespaces

The namespaces used in the schemata are given in Table 4.3.

Prefix	Description	Description
`hldc5`	High Level Document Check	http://trdoccheck.bsi.bund.de/hl-dc/wsdl/5

Prefix	Description	Description
wf	HLDC5 Workflow and Feedback	http://trdoccheck.bsi.bund.de/hldc/workflow/5
tl5	Transaction management	http://trdoccheck.bsi.bund.de/tl/wsdl/5
xs	XML Schema	http://www.w3.org/2001/XMLSchema

Table 4.3 Namespaces

4.4 XML Schema and Web Service Definition

The XML Schema Definition (XSD) (`.xsd`) and Web Services Description Language (WSDL) (`.wsdl`) files given in Table 4.4 are provided with this Technical Guideline.

Message type	Part name
hldc_v5.wsdl	HLDC5 web service definition (Chapter 6)
hldc_v5.xsd	XSD for HLDC5 web service
wf_v5.xsd	XSD for workflow definitions (Section 6.5)
fb_v5.xsd	XSD for workflow feedback (Chapter 7)
tl_v5.wsdl	Transaction management web service definition (Chapter 8)
tl_v5.xsd	XSD for transaction management web service
log_hldc_shared-v5_0_0.xsd	XSD for shared data types between web service and logging schema
hldc_shared_v5.xsd	XSD for shared datatypes within web service schemata

Table 4.4 Naming convention for SOAP messages

4.5 Interoperability

To ensure trouble-free interoperability between different SOAP implementations, both client and server implementations SHOULD fulfil the WS-I Basic Profile 1.1.

5 Interface overview

5.1 High-Level Document Check

5.1.1 Objective

HLDC5 interface provides execution of [BSI TR-03135]-compliant document checks controlled by XML-based workflow definitions.

5.1.2 Workflow management

- The client MAY publish custom workflow definitions on the server by calling `addWorkflow`. Identically named workflow definitions are replaced.
- The client MAY remove workflow definitions from the server by calling `removeWorkflow`.
- The function `getAllWorkflows` provides a list of all workflow definitions that are available on the server.

5.1.3 Workflow execution

- The function `beginWorkflow` executes a document check workflow. The server will wait for a document before executing the actual reading and checking process. A transaction Identity (ID) SHALL be provided to ensure [BSI TR-03135]-compliant logging. `beginWorkflow` provides a unique workflow ID that SHALL be used in subsequent calls. In general, the ID SHALL remain valid until the client calls `endWorkflow`. To protect against broken clients, the server MAY limit the ID validity with a timeout.
- The client MAY force document reading and processing by calling `triggerWorkflow` which skips document detection.
- During workflow execution several user interactions may be necessary which will pause the execution. The user has to react accordingly and the client SHALL call `continueWorkflow` to resume the process.
- Workflow execution MAY be cancelled by calling `cancelWorkflow`.
- The client SHALL inform the server that client-side workflow processing has finished by calling `endWorkflow`.

5.1.4 Workflow feedback

Document, check result and live data are provided by the server individually and are referred to as workflow feedback.

- The function `getWorkflowFeedback` provides the next available feedback element.
- The client MAY request particular document and result data by calling `getWorkflowFeedbackById`.

5.1.5 Cross-document checks

If a workflow contains definitions for cross-document checks, the client SHOULD call `linkWorkflow` in order to assign the relevant reference document. The call SHALL appear after feedback processing for the current document has finished.

5.2 Transaction management interface

5.2.1 Objective

The transaction management interface provides functions to link document check operations to a [BSI TR-03135]-compliant transaction and to allow logging of relevant data.

5.2.2 Transaction management

- Transactions are initiated by `beginTransaction`. This function provides an ID that is REQUIRED for all the functions of this interface and SHALL be used when calling `beginWorkflow` in order to link the workflow execution to this particular transaction.
- A transaction version is REQUIRED to start a transaction. A list of supported versions MAY be requested by calling `getSupportedVersions`.
- Multiple transactions MAY be merged into a single one with `mergeTransaction`.
- Transactions are terminated by `endTransaction` which invalidates the transaction ID.

5.2.3 Logging

- The function `saveTransaction` stores the currently available logged transaction data persistently on the server. The client MAY limit the amount of data that is stored.
- The client MAY request the currently available transaction data for client-side use by calling `getTransactionXML`.
- Client-specific data MAY be logged by calling `addLogData`.
- The function `getAllLoggingProviders` is OPTIONAL and MAY return a server-specific list of logging targets that MAY be used when calling `saveTransaction`.
- The function `saveTransactionXML` allows to store client-generated XML via available logging providers.
- The client MAY publish custom logging profiles (Extensible Stylesheet Language Transformation (XSLT)) on the server by calling `addLoggingProfile`. Identically named logging profiles are replaced.
- The client MAY remove logging profiles from the server by calling `removeLoggingProfile`.

5.2.4 Write-protection

In order to prevent inconsistent log data, a transaction is protected against further modification after the first read access, i.e., after calls to `saveTransaction` and `getTransactionXML`. All future write access, i.e., calls that would modify the transaction, SHALL fail then with an appropriate error.

The server MAY fail read access calls with appropriate error if the transaction is currently modified (e.g., by the document check process).

5.3 Error handling

If errors occur during processing of a web service request, a SOAP fault is generated according to the SOAP 1.1 specification. SOAP faults are comparable to exceptions in programming languages such as C++, C# or Java insofar as they allow reporting of errors without the need to account for error codes in function signatures.

SOAP faults are returned in place of the SOAP response. Depending on the type of an error, the fault message may contain additional information about the error. The faults that are specific to the web services in this document are specified in the respective chapters and listed with every function that may generate them. Faults originating from other causes such as network connection problems or validation errors are beyond the scope of this document as they depend on the specific SOAP implementation.

6 High Level Document Check API

The HLDC5 Application Programming Interface (API) contains functions to perform electronic and optical document checks in conformity to part 1 of this Technical Guideline. The check processes are driven by XML workflows and provide a very high level interface to the application. The check results contain multiple subresults which are combined to overall results according to [BSI TR-03135-1]. All checks can be logged in XML format compliant to [BSI TR-03135-1] using the transaction management API in Chapter 8.

The definitions of the HLDC5 API are provided in `hldc_v5.wsdl`. The schemata for the workflow definition and XML-formatted feedback are provided in `wf_v5.xsd` and `fb_v5.xsd` respectively.

6.1 Namespaces

The elements of the server- and client-side APIs are member of the namespace, which is aliased by `hldc5`. The workflow definition schema and the workflow feedback schema use the namespace http://trdoccheck.bsi.bund.de/hldc/workflow/5 aliased by `wf`.

6.2 Data types

In addition to simple XSD types, the SOAP interface uses custom data types, which are described in the following.

6.2.1 FeedbackStatus

Represents the status of a feedback element. Derived from `xsd:string`.

6.2.1.1 Values

The values of the FeedbackStatus values are given in Table 6.1.

Value	Description
`valid`	The feedback element contains valid data which is available in `stringFeedback` or `binaryFeedback`.
`err-invalid-conversion`	An error occurred during feedback preparation. Data could not be converted into the specified target format.
`err-not-available`	Requested data is not available or could not be read.
`err-not-supported`	Requested data is not supported by the inspection system (e.g. no Visual Zone (VIZ) OCR available).
`err-reference-document-required`	Requested data require access to a reference document for cross-document checks (see `linkWorkflow`).

Table 6.1 FeedbackStatus - values

6.2.1.2 WSDL Definition

```
<simpleType name="FeedbackStatus">
 <restriction base="xsd:string"/>
  <enumeration value="valid"/>
  <enumeration value="err-invalid-conversion"/>
  <enumeration value="err-not-available"/>
  <enumeration value="err-not-supported"/>
  <enumeration value="err-reference-document-required"/>
 </restriction>
</simpleType>
```

6.2.2 UUID

Serves to uniquely reference various elements at runtime (e.g. currently executing workflows). Inherits `xsd:string`.

6.2.2.1 Format restrictions

The content SHALL be empty or represent a universally unique identifier of 32 lower-case hexadecimal letters that are separated into 5 groups of length 8, 4, 4, 4 and 12 letters using hyphens (e.g. `01234567-89ab-cdef-0123-456789abcdef`).

6.2.2.2 WSDL Definition

```
<simpleType name="UUID">
 <restriction base="xsd:string">
  <pattern value="([A-Fa-f0-9]{8}-[A-Fa-f0-9]{4}-[A-Fa-f0-9]{4}-[A-Fa-f0-9]{4}-
[A-Fa-f0-9]{12})?"/>
 </restriction>
</simpleType>
```

6.2.3 Workflow

Contains general workflow information.

6.2.3.1 Elements

The Workflow elements are given in ▸Table 6.2.

Element name	Description
`name`	`xsd:string` The name of the workflow.
`vendor`	`xsd:string` The vendor that defined the workflow.
`version`	`xsd:string` The version of the workflow.
`description`	`xsd:string` The description of the workflow.

Table 6.2 Workflow - elements

6.2.3.2 WSDL Definition

```
<complexType name="Workflow">
 <sequence>
  <element name="name" type="xsd:string" />
  <element name="vendor" type="xsd:string"/>
  <element name="version" type="xsd:string"/>
  <element name="description" type="xsd:string"/>
 </sequence>
</complexType>
```

6.2.4 WorkflowFeedback

Represents a feedback element from a HLDC5 workflow.

6.2.4.1 Attributes

None.

6.2.4.2 Elements

The WorkflowFeedback elements are given in ▸Table 6.3.

Element name	Description
feedbackID	xsd:string Reference to a feedback element. SHALL be empty if status is either finished or cancelled. SHALL NOT be empty otherwise.
stringFeedback?	xsd:string The feedback data as string (if possible). SHALL be empty if status is different from valid.
binaryFeedback?	xsd:base64Binary The feedback data as binary (if possible). SHALL be empty if status is different from valid.
liveFeedback?	hldc5:LiveFeedback The live feedback data. Depending on the configuration in the workflow this element may be empty.
status	hldc5:FeedbackStatus The status code of the feedback element.
errorCode	hldc5:type.shared.errorcode Optional error code to signalize errors during processing.

Table 6.3 WorkflowFeedback - elements

6.2.4.3 WSDL Definition

```
<complexType name="WorkflowFeedback">
 <sequence>
   <element name="feedbackID" type="xsd:string" />
   <element name="stringFeedback" type="xsd:string"
       minOccurs="0" maxOccurs="1" />
   <element name="binaryFeedback" type="xsd:base64Binary"
       minOccurs="0" maxOccurs="1" />
   <element name="liveFeedback" type="hldc5:LiveFeedback"
    minOccurs="0" maxOccurs="1"/>
   <element name="status" type="hldc5:FeedbackStatus" />
   <element name="errorCode" type="hldc5:type.shared.errorcode"
    minOccurs="0" maxOccurs="1"/>
 </sequence>
</complexType>
```

6.2.5 WorkflowParameter

Key-value pair to configure conditions of type `type.workflow.conditions.parameter` in a HLDC5 workflow definition.

6.2.5.1 Attributes

None.

6.2.5.2 Elements

The WorkflowParameter elements are given in Table 6.4.

Element name	Description
id	xsd:string The key.
value	xsd:boolean Specifies whether the corresponding condition SHALL be considered fulfilled.

Table 6.4 WorkflowParameter - elements

6.2.5.3 WSDL Definition

```
<complexType name="WorkflowParameter">
 <sequence>
  <element name="id" type="xsd:string" />
  <element name="value" type="xsd:boolean" />
 </sequence>
</complexType>
```

6.2.6 WorkflowParameters

List of key-value pairs.

6.2.6.1 Elements

The WorkflowParameters elements are given in Table 6.5.

Element name	Description
parameter*	hldc5:Parameter A key-value pair.

Table 6.5 WorkflowParameters - elements

6.2.6.2 WSDL Definition

```
<complexType name="WorkflowParameters">
 <sequence>
  <element name="parameter" type="hldc5:WorkflowParameter"
      minOccurs="0" maxOccurs="unbounded"/>
 </sequence>
</complexType>
```

6.2.7 WorkflowStatus

Represents the execution status of a workflow. Derived from xs:string.

6.2.7.1 Values

The WorkflowParameters values are given in Table 6.6.

Value	Description
ok	Workflow execution is still in progress.
waiting	The workflow execution is paused. The server is waiting for a user interaction.
finished	The workflow execution has finished. This is the last feedback element delivered by getWorkflowFeedback.
cancelled	The workflow has been cancelled by the user. This is the last feedback element delivered by getWorkflowFeedback.

Table 6.6 WorkflowStatus - values

6.2.7.2 WSDL Definition

```
<simpleType name="WorkflowStatus">
 <restriction base="xsd:string"/>
  <enumeration value="ok"/>
  <enumeration value="waiting"/>
  <enumeration value="finished"/>
  <enumeration value="cancelled"/>
 </restriction>
</simpleType>
```

6.2.8 ContinueWorkflowParameter

Key-value pair for user interactions.

6.2.8.1 Attributes

None.

6.2.8.2 Elements

The ContinueWorkflowParameter elements are given in Table 6.7.

Element name	Description
id	xsd:string The key. The key SHALL match the feedbackId from the user request.
value	xsd:string Specifies the value of the user decision.

Table 6.7 ContinueWorkflowParameter - elements

6.2.8.3 WSDL Definition

```
<complexType name="ContinueWorkflowParameter">
 <sequence>
  <element name="id" type="xsd:string" />
  <element name="value" type="xsd:string" />
 </sequence>
</complexType>
```

6.2.9 LiveFeedback

The live feedback data contains information about the document reading and checking process while the workflow is in progress (e.g. live stream information).

6.2.9.1 Attributes

None.

6.2.9.2 Elements

The LiveFeedback elements are given in Table 6.8.

Element name	Description
optical?	hldc5:OpticalStatus Contains the optical live data.
electronic?	hldc5:ElectronicStatus Contains the electronic live data.

Table 6.8 LiveFeedback - elements

6.2.9.3 WSDL Definition

```
<complexType name="LiveFeedback">
 <sequence>
  <element name="optical" type="hldc5:OpticalStatus" minOccurs="0" maxOccurs="1"/>
  <element name="electronic" type="hldc5:ElectronicStatus" minOccurs="0" maxOccurs="1"/>
 </sequence>
</complexType>
```

6.2.10 OpticalStatus

The optical status data contains information about the optical processing (e.g. process status and live stream information).

6.2.10.1 Attributes

None.

6.2.10.2 Elements

The OpticalStatus elements are given in ▸Table 6.9.

Element name	Description
notification	hldc5:OpticalNotification Contains information about the optical process status and hints for document positioning.
image?	xsd:base64Binary The server MAY send the current live stream image (i.e. an image of the complete scanning area) if the feature is activated in the workflow and supported by the inspection system.
targetPosition?	hldc5:ImageRect The server SHOULD send information about the target document position during the placement for user assistance. The target position is defined as a rectangle given by coordinates within the image with the origin in the lower left corner of the image.
documentPosition?	hldc5:ImageOutline The server SHOULD send information about the current document position during placement for user assistance. The current position is defined as a polygon given by coordinates within the image with the origin in the lower left corner of the image.
errorCode?	hldc5:type.shared.errorcode Optional error code for optical processing.

Table 6.9 OpticalStatus - elements

6.2.10.3 WSDL Definition

```
<complexType name="OpticalStatus">
 <sequence>
  <element name="notification" type="hldc5:OpticalNotification"/>
  <element name="image" type="xsd:base64Binary" minOccurs="0" maxOccurs="1"/>
  <element name="targetPosition" type="hldc5:ImageRect" minOccurs="0" maxOccurs="1"/>
  <element name="documentPosition" type="hldc5:ImageOutline" minOccurs="0" maxOccurs="1"/>
  <element name="errorCode" type="hldc5:type.shared.errorcode" minOccurs="0" maxOccurs="1"/>
 </sequence>
</complexType>
```

6.2.11 OpticalNotification

The optical notification consists of information about the optical process status and hints for document positioning.

6.2.11.1 Attributes

None.

6.2.11.2 Elements

The OpticalNotification elements are given in ▸Table 6.10.

Element name	Description
status	hldc5:OpticalDocumentStatus Status of the optical process.
hint?	hldc5:OpticalDocumentHint The server MAY send hints for document positioning for user assistance.

Table 6.10 OpticalNotification - elements

6.2.11.3 WSDL Definition

```
<complexType name="OpticalNotification">
 <sequence>
  <element name="status" type="hldc5:OpticalDocumentStatus"/>
  <element name="hint" type="hldc5:OpticalDocumentHint" minOccurs="0" maxOccurs="1"/>
 </sequence>
</complexType>
```

6.2.12 OpticalDocumentStatus

Represents the status of the optical processing. The server SHALL return the best matching status if supported by the document scanning device. Derived from xsd:string.

6.2.12.1 Values

The OpticalDocumentStatus values are given in ▸Table 6.11.

Value	Description
not available	The optical document status cannot be determined (e.g. if the workflow requires optical processing, but there is no scanning device attached to the inspection system. An additional error code in the OpticalStatus SHALL be returned in that case.)
no document	There is no document on the scanning device.
moving	The device detects movement on its scanning area.
bad placement	The document is not moving, but has bad positioning. The server SHOULD return the current and the target positions of the document in the OpticalStatus if supported.
ready	The document is positioned correctly and the device is ready to scan the document.
scanning	The device is currently scanning the document.
processing	The optical process (e.g. optical check execution) is in progress.
optical processing finished	The optical processing is complete.

Table 6.11 OpticalDocumentStatus - values

6.2.12.2 WSDL Definition

```
<simpleType name="OpticalDocumentStatus">
 <restriction base="xsd:string">
  <enumeration value="not available"/>
  <enumeration value="no document"/>
  <enumeration value="moving"/>
  <enumeration value="bad placement"/>
  <enumeration value="ready"/>
  <enumeration value="scanning"/>
  <enumeration value="processing"/>
  <enumeration value="optical processing finished"/>
 </restriction>
</simpleType>
```

6.2.13 OpticalDocumentHint

Represents hints for user assistance. The server SHALL return the best matching hint if supported by the document scanning device. Derived from xsd:string.

6.2.13.1 Values

The OpticalDocumentHint values are given in ▸Table 6.12.

Value	Description
`push`	This hint MAY be sent by the server if the document has to be moved forward or further inside the scanning device.
`pull`	This hint MAY be sent by the server if the document has to be moved backward or further out of the scanning device.
`left`	This hint MAY be sent by the server if the document has to be moved left.
`right`	This hint MAY be sent by the server if the document has to be moved right.
`press`	This hint MAY be sent by the server if the document has to be pressed onto the device (i.e. the document has to lie flat on the scanning area).
`rotate`	This hint MAY be sent by the server if the document has wrong orientation, but the rotation direction is not determined.
`rotate clockwise`	This hint MAY be sent by the server if the document has wrong orientation and shall be rotated clockwise.
`rotate counterclockwise`	This hint MAY be sent by the server if the document has wrong orientation and shall be rotated counterclockwise.
`hold`	This hint MAY be sent by the server if the document is moved for a extended period of time during document positioning (i.e. the document has to lie still on the scanning area). The server MAY also send this hint during the process as long as the document shall not be moved.
`flip`	This hint MAY be sent by the server if the document has to be flipped (in case of two-sided documents). The server MAY also send this hint if the page of the document has to be flipped (in case of booklets).
`open`	This hint MAY be sent by the server if the document has to be opened to scan the correct page.

Table 6.12 OpticalDocumentHint - values

6.2.13.2 WSDL Definition

```
<simpleType name="OpticalDocumentHint">
 <restriction base="xsd:string">
  <enumeration value="push"/>
  <enumeration value="pull"/>
  <enumeration value="left"/>
  <enumeration value="right"/>
  <enumeration value="press"/>
  <enumeration value="rotate"/>
  <enumeration value="rotate clockwise"/>
  <enumeration value="rotate counterclockwise"/>
  <enumeration value="hold"/>
  <enumeration value="flip"/>
  <enumeration value="open"/>
  </restriction>
</simpleType>
```

6.2.14 ImageRect

A basic structure to represent a rectangle via coordinates within an image.

6.2.14.1 Attributes

None.

6.2.14.2 Elements

The ImageRect elements are given in Table 6.13.

Element name	Description
`top_left`	`hldc5:ImagePoint` Coordinates for the top left corner of the rectangle.
`top_right`	`hldc5:ImagePoint` Coordinates for the top left corner of the rectangle.
`bottom_left`	`hldc5:ImagePoint` Coordinates for the top left corner of the rectangle.
`bottom_right`	`hldc5:ImagePoint` Coordinates for the top left corner of the rectangle.

Table 6.13 ImageRect - elements

6.2.14.3 WSDL Definition

```
<complexType name="ImageRect">
 <sequence>
  <element name="top_left" type="hldc5:ImagePoint"/>
  <element name="top_right" type="hldc5:ImagePoint"/>
  <element name="bottom_left" type="hldc5:ImagePoint"/>
  <element name="bottom_right" type="hldc5:ImagePoint"/>
 </sequence>
</complexType>
```

6.2.15 ImageOutline

A basic structure to represent a polygon via coordinates within an image. The polygon is represented by a polyline i.e. a sequence of connected points where the last point has to be connected to the first one to close the polygon.

6.2.15.1 Attributes

None.

6.2.15.2 Elements

The ImageOutline elements are given in Table 6.14.

Element name	Description
`Point`	`hldc5:ImagePoint` A single point of the polyline (i.e. a corner of the polygon)

Table 6.14 ImageOutline - elements

6.2.15.3 WSDL Definition

```
<complexType name="ImageOutline">
 <sequence>
  <element name="Point" type="hldc5:ImagePoint" minOccurs="1" maxOccurs="unbounded"/>
 </sequence>
</complexType>
```

6.2.16 ImagePoint

A basic structure to represent a point via coordinates within an image. The origin of the coordinates SHALL be the lower left corner of the image.

6.2.16.1 Attributes

None.

6.2.16.2 Elements

The ImagePoint elements are given in ▸Table 6.15.

Element name	Description
x	xsd:Integer The horizontal position of the point.
y	xsd:Integer the vertical position of the point.

Table 6.15 ImagePoint - elements

6.2.16.3 WSDL Definition

```
<complexType name="ImagePoint">
 <sequence>
  <element name="x" type="xsd:integer"/>
  <element name="y" type="xsd:integer"/>
 </sequence>
</complexType>
```

6.2.17 ElectronicStatus

The electronic status data contains information about the electronic processing.

6.2.17.1 Attributes

None.

6.2.17.2 Elements

The ElectronicStatus elements are given in ▸Table 6.16.

Element name	Description
notification	hldc5:ElectronicNotification Contains information about the optical process status and hints for document positioning.
numberOfDetectedChips?	xsd:integer The number of chips which were detected. The server SHALL not use this node if chip detection was not done.
errorCode?	hldc5:type.shared.errorcode Optional error code for electronic processing.

Table 6.16 ElectronicStatus - elements

6.2.17.3 WSDL Definition

```
<complexType name="ElectronicStatus">
 <sequence>
  <element name="notification" type="hldc5:ElectronicNotification"/>
  <element name="numberOfDetectedChips" type="xsd:integer" minOccurs="0" maxOccurs="1"/>
  <element name="errorCode" type="hldc5:type.shared.errorcode" minOccurs="0" maxOccurs="1"/>
 </sequence>
```

```
</complexType>
```

6.2.18 ElectronicNotification

The electronic notification consists of information about the electronic process status.

6.2.18.1 Attributes

None.

6.2.18.2 Elements

The ElectronicNotification elements are given in Table 6.17.

Element name	Description
status	hldc5:ElectronicDocumentStatus Status of the electronic process.

Table 6.17 ElectronicNotification - elements

6.2.18.3 WSDL Definition

```
<complexType name="ElectronicNotification">
 <sequence>
  <element name="status" type="hldc5:ElectronicDocumentStatus"/>
 </sequence>
</complexType>
```

6.2.19 ElectronicDocumentStatus

Represents the status of the electronic processing. The server SHALL return the best matching status if supported by the document scanning device. Derived from xs:string.

6.2.19.1 Values

The ElectronicDocumentStatus values are given in Table 6.18.

Value	Description
not available	The electronic document status cannot be determined (e.g. if the workflow requires electronic processing, but there is no chip reading device attached to the inspection system. An additional error code in the ElectronicStatus SHALL be returned in that case.)
no chip	There is no chip found by the device.
searching	The device is searching for a chip.
ready	A chip was found an is ready for reading.
reading	The device is currently accessing or reading the chip.
processing	The electronic process (e.g. electronic check execution) is in progress.
electronic processing finished	The electronic processing is complete.

Table 6.18 ElectronicDocumentStatus - values

6.2.19.2 WSDL Definition

```
<simpleType name="ElectronicDocumentStatus">
 <restriction base="xsd:string">
  <enumeration value="not available"/>
  <enumeration value="no chip"/>
  <enumeration value="searching"/>
  <enumeration value="ready"/>
  <enumeration value="reading"/>
  <enumeration value="processing"/>
  <enumeration value="electronic processing finished"/>
```

```
  </restriction>
</simpleType>
```

6.3 Fault types

This section specifies the SOAP faults that are specific to this SOAP API. No fault has any attributes.

6.3.1 InvalidConditionId

Returned if a condition ID is not defined in the workflow definition.

6.3.1.1 Elements

The InvalidConditionId elements are given in ▸Table 6.19.

Element name	Description
id	`xsd:string` The rejected ID.

Table 6.19 InvalidConditionId - elements

6.3.1.2 WSDL Definition

```
<complexType name="InvalidConditionId">
 <sequence>
  <element name="id" type="xsd:string" />
 </sequence>
</complexType>
```

6.3.2 InvalidFeedbackId

Returned if the feedback ID in the request is not defined in the workflow definition.

6.3.2.1 Elements

None.

6.3.2.2 WSDL Definition

```
<element name="InvalidFeedbackId">
  <complexType />
</element>
```

6.3.3 InvalidWorkflowId

Returned if a workflow ID does not exist on the server. The ID is either invalid or has expired due to a call to `endWorkflow` or limited resources on the server.

6.3.3.1 Elements

The InvalidWorkflowId elements are given in ▸Table 6.20.

Element name	Description
id?	`hldc5:UUID` The rejected ID. Only present in case of ambiguity.

Table 6.20 InvalidWorkflowId - elements

6.3.3.2 WSDL Definition

```
<element name="InvalidWorkflowId">
  <complexType>
    <sequence>
```

```
      <element name="id" type="hldc5:UUID" minOccurs="0"
        maxOccurs="1" />
    </sequence>
  </complexType>
</element>
```

6.3.4 WorkflowLimitExceeded

Returned in case of too many executing workflows. The client either needs to cancel a currently executing workflow or wait until it finishes.

6.3.4.1 Elements

None.

6.3.4.2 WSDL Definition

```
<element name="WorkflowLimitExceeded">
  <complexType />
</element>
```

6.3.5 WorkflowNotFound

Returned if the requested workflow does not exist on the server.

6.3.5.1 Elements

None.

6.3.5.2 WSDL Definition

```
<element name="WorkflowNotFound">
  <complexType />
</element>
```

6.3.6 WorkflowParserError

Returned if the provided workflow definition could not be parsed by the server.

6.3.6.1 Elements

None.

6.3.6.2 WSDL Definition

```
<element name="WorkflowParserError">
  <complexType />
</element>
```

6.3.7 InvalidOperation

Returned if continueWorkflow was called, but the server was not waiting for a user interaction.

6.3.7.1 Elements

None.

6.3.7.2 WSDL Definition

```
<element name="InvalidOperation">
 <complexType />
</element>
```

6.3.8 InvalidParameter

Returned if continueWorkflow was called with a wrong parameter.

6.3.8.1 Elements

None.

6.3.8.2 WSDL Definition

```
<element name="InvalidParameter">
 <complexType />
</element>
```

6.4 Operations

6.4.1 addWorkflow

Transfers a new workflow definition to the server. Input SHALL be validated against the schema `wf_v2.xsd` (Section 6.5) and SHALL be checked for consistency. The workflow SHALL NOT contain invalid ID references and SHALL NOT contain cyclic dependencies. If an identically named workflow exists on the server, it SHALL be replaced with the newly submitted definitions.

6.4.1.1 Request elements

The addWorkflow request elements are given in Table 6.21.

Element name	Description
`workflowDefinition`	`xsd:base64Binary` The base64-encoded XML data of the workflow definition. The XML structure is defined in Section 6.5.

Table 6.21 addWorkflow - request elements

6.4.1.2 Response elements

The addWorkflow response elements are given in Table 6.22.

Element name	Description
`workflowName`	`xsd:string` The name of the workflow which is parsed from the workflow definition.

Table 6.22 addWorkflow - response elements

6.4.1.3 Faults

The addWorkflow faults are given in Table 6.23.

Fault	Cause
`hldc5:WorkflowParserError`	An error occurred while parsing or validating the submitted workflow.

Table 6.23 addWorkflow - faults

6.4.1.4 WSDL Definition

```
<!-- operation request element -->
<element name="addWorkflow">
 <complexType>
  <sequence>
   <element name="workflowDefinition" type="xsd:base64Binary" />
  </sequence>
 </complexType>
</element>
```

```
<!-- operation response element -->
<element name="addWorkflowResponse">
 <complexType>
  <sequence>
   <element name="workflowName" type="xsd:string" />
  </sequence>
 </complexType>
</element>
```

6.4.2 beginWorkflow

Initiates workflow execution on the server to check the current document and/or read out the requested data. Check results and data SHALL be queried individually by calling `getWorkflowFeedback` or `getWorkflowFeedbackById`.

6.4.2.1 Request elements

The beginWorkflow request elements are given in Table 6.24.

Element name	Description
`workflowName`	`xsd:string` The name of the workflow to execute. SHALL be a valid workflow name as returned by `addWorkflow` or `getAllWorkflows`.
`transactionID`	`t15:UUID` Transaction ID returned by `t15:beginTransaction` (see Section 8.4.4) to link the workflow to the specified [BSI TR-03135] transaction.
`docIdentifier?`	`xsd:string` MRZ or CAN to be used to access the document instead of the automatically retrieved (if available) MRZ/CAN.
`binary?`	`xsd:base64Binary` This parameter MAY be used by the client to pass external binary data which is used in the workflow execution (e.g. Digital Travel Credential (DTC)).
`parameters?`	`hldc5:WorkflowParameters` List of key-value pairs which override the default values of the `hldc5:ParameterCondition` entries in the workflow definition. The IDs SHALL match the workflow definition.

Table 6.24 beginWorkflow - request elements

6.4.2.2 Response elements

The beginWorkflow response elements are given in Table 6.25.

Element name	Description
`workflowID`	`hldc5:UUID` Unique ID of the workflow execution which SHALL be used as a reference in corresponding function calls.

Table 6.25 beginWorkflow - response elements

6.4.2.3 Faults

The beginWorkflow faults are given in Table 6.26.

Fault	Cause
`hldc5:InvalidConditionId`	`parameters` contains at least one condition that does not match the workflow definition.
`hldc5:WorkflowLimitExceeded`	Too many workflows are currently executing on the server.

Fault	Cause
`hldc5:WorkflowNotFound`	The workflow `workflowName` does not exist on the server.
`tl5:InvalidTransactionId`	Value of `transactionID` is invalid or has expired.

Table 6.26 beginWorkflow - faults

6.4.2.4 WSDL Definition

```
<!-- operation request element -->
<element name="beginWorkflow">
 <complexType>
  <sequence>
   <element name="workflowName" type="xsd:string" />
   <element name="transactionID" type="tl5:UUID" />
   <element name="docIdentifier" type="xsd:string" minOccurs="0" maxOccurs="1" />
   <element name="binary" type="xsd:base64Binary" minOccurs="0" maxOccurs="1" />
   <element name="parameters" type="hldc5:WorkflowParameters" minOccurs="0" maxOccurs="1" />
  </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="beginWorkflowResponse">
 <complexType>
  <sequence>
   <element name="workflowID" type="hldc5:UUID" />
  </sequence>
 </complexType>
</element>
```

6.4.3 cancelWorkflow

Cancels workflow execution. The workflow is cancelled successfully when the status `cancelled` is returned in the feedback loop.

6.4.3.1 Request elements

The cancelWorkflow request elements are given in ▸Table 6.27.

Element name	Description
`workflowID`	`hldc5:UUID` The ID of the workflow to cancel that was returned by `beginWorkflow`.

Table 6.27 cancelWorkflow - request elements

6.4.3.2 Response elements

None.

6.4.3.3 Faults

The cancelWorkflow faults are given in ▸Table 6.28.

Fault	Cause
`hldc5:InvalidWorkflowId`	Value of `workflowID` is invalid or has expired.

Table 6.28 cancelWorkflow - faults

6.4.3.4 WSDL Definition

```
<!-- operation request element -->
<element name="cancelWorkflow">
 <complexType>
  <sequence>
   <element name="workflowID" type="hldc5:UUID" />
```

```
  </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="cancelWorkflowResponse">
 <complexType />
</element>
```

6.4.4 endWorkflow

Informs the server that client-side workflow processing has finished. Invalidates the workflow ID. SHALL be called by the client for each executed workflow. To protect against broken clients, the server MAY invalidate workflow IDs based on a timeout. SHOULD NOT be called concurrently to other functions that operate on the same workflow ID except `cancelWorkflow`.

6.4.4.1 Request elements

The endWorkflow request elements are given in Table 6.29.

Element name	Description
`workflowID`	`hldc5:UUID` Reference to the workflow to end. If the workflow is still executing on the server, the workflow is cancelled by the server.

Table 6.29 endWorkflow - request elements

6.4.4.2 Response elements

The endWorkflow response elements are given in Table 6.30.

Element name	Description
`workflowStatus`	`hldc5:WorkflowStatus` Final execution status of the workflow. SHALL be either `finished` or `cancelled`.

Table 6.30 endWorkflow - response elements

6.4.4.3 Faults

The endWorkflow faults are given in Table 6.31.

Fault	Cause
`hldc5:InvalidWorkflowId`	Value of `workflowID` is invalid or has expired.

Table 6.31 endWorkflow - faults

6.4.4.4 WSDL Definition

```
<!-- operation request element -->
<element name="endWorkflow">
 <complexType>
  <sequence>
   <element name="workflowID" type="hldc5:UUID" />
  </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="endWorkflowResponse">
 <complexType>
  <sequence>
   <element name="workflowStatus" type="hldc5:WorkflowStatus" />
  </sequence>
 </complexType>
```

```
</element>
```

6.4.5 getAllWorkflows

Returns a list of all available workflows.

6.4.5.1 Request elements

None.

6.4.5.2 Response elements

The getAllWorkflows response elements are given in ▸Table 6.32.

Element name	Description
`workflow*`	`hldc5:Workflow` The list of all available workflows. `getAllWorkflows` returns one `workflow` element per available workflow.

Table 6.32 getAllWorkflows - response elements

6.4.5.3 Faults

None.

6.4.5.4 WSDL Definition

```
<!-- operation request element -->
<element name="getAllWorkflows">
  <complexType />
</element>

<!-- operation response element -->
<element name="getAllWorkflowsResponse">
 <complexType>
  <sequence>
   <element name="workflow" type="hldc5:Workflow"
       minOccurs="0" maxOccurs="unbounded" />
  </sequence>
 </complexType>
</element>
```

6.4.6 getWorkflowFeedback

Returns the next available feedback element during workflow execution. Generally called in a loop until `finished` or `cancelled` is returned as feedback status. The order in which individual feedback elements are returned may be different from the order in the workflow definition. This function MAY skip feedback elements that were previously explicitly requested by `getWorkflowFeedbackById`.

6.4.6.1 Request elements

The getWorkflowFeedback request elements are given in ▸Table 6.33

Element name	Description
`workflowID`	`hldc5:UUID` Reference to the workflow for which the next feedback element is requested.
`timeout-ms`	`xsd:int` Timeout in milliseconds for getting the next feedback element. If no new feedback element becomes available during this time, an error is returned. The server MAY cap the value. Negative values are interpreted as “infinity” which is subject to a server-imposed limit.

Table 6.33 getWorkflowFeedback - request elements

6.4.6.2 Response elements

The getWorkflowFeedback response elements are given in ▸Table 6.34.

Element name	Description
`workflowStatus`	`hldc5:WorkflowStatus` Current execution status of the workflow.
`timeout-expired`	`xsd:boolean` MUST be `true` if no new feedback was available before the timeout expired. SHALL be `false` if new feedback was available in time.
`feedback?`	`hldc5:WorkflowFeedback` The next available feedback element. Omitted in case of timeout.

Table 6.34 getWorkflowFeedback - response elements

6.4.6.3 Faults

The getWorkflowFeedback response elements are given in ▸Table 6.35.

Fault type	Cause
`hldc5:InvalidWorkflowId`	Value of `workflowID` is invalid or has expired.

Table 6.35 getWorkflowFeedback - faults.

6.4.6.4 WSDL Definition

```
<!-- operation request element -->
<element name="getWorkflowFeedback">
 <complexType>
  <sequence>
   <element name="workflowID" type="hldc5:UUID" />
   <element name="timeout-ms" type="xsd:int" />
  </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="getWorkflowFeedbackResponse">
 <complexType>
  <sequence>
   <element name="workflowStatus" type="hldc5:WorkflowStatus" />
   <element name="timeout-expired" type="xsd:boolean" />
   <element name="feedback" type="hldc5:WorkflowFeedback"
       minOccurs="0" maxOccurs="1" />
  </sequence>
 </complexType>
</element>
```

6.4.7 getWorkflowFeedbackById

Returns the requested feedback element.

6.4.7.1 Request elements

The getWorkflowFeedbackById request elements are given in ▸Table 6.36

Element name	Description
`workflowID`	`hldc5:UUID` Reference to the workflow for which the next feedback element is requested.
`feedbackId`	`xsd:string` The requested feedback ID.

Element name	Description
timeout-rms	xsd:int Timeout in milliseconds for getting the feedback element. If the feedback element does not become available during this time, an error is returned. The server MAY cap the value. Negative values are interpreted as “infinity” which is subject to a server-imposed limit.

Table 6.36 getWorkflowFeedbackById - request elements

6.4.7.2 Response elements

The getWorkflowFeedbackById response elements are given in Table 6.37.

Element name	Description
workflowStatus	hldc5:WorkflowStatus Current execution status of the workflow.
timeout-expired	xsd:boolean MUST be `true` if the requested feedback was not available before the timeout expired. SHALL be `false` if the requested feedback was available in time.
feedback?	hldc5:WorkflowFeedback The next available feedback element. Omitted in case of timeout.

Table 6.37 getWorkflowFeedbackById - response elements

6.4.7.3 Faults

The getWorkflowFeedbackById response elements are given in Table 6.38.

Fault type	Cause
hldc5:InvalidFeedbackId	Value of `feedbackId` is not defined in the workflow definition.
hldc5:InvalidWorkflowId	Value of `workflowID` is invalid or has expired.

Table 6.38 getWorkflowFeedbackById - faults

6.4.7.4 WSDL Definition

```
<!-- operation request element -->
<element name="getWorkflowFeedbackById">
 <complexType>
  <sequence>
   <element name="workflowID" type="hldc5:UUID" />
   <element name="feedbackId" type="xsd:string" />
   <element name="timeout-ms" type="xsd:int" />
  </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="getWorkflowFeedbackByIdResponse">
 <complexType>
  <sequence>
   <element name="workflowStatus" type="hldc5:WorkflowStatus" />
   <element name="timeout-expired" type="xsd:boolean" />
   <element name="feedback" type="hldc5:WorkflowFeedback"
      minOccurs="0" maxOccurs="1" />
  </sequence>
 </complexType>
</element>
```

6.4.8 linkWorkflow

Links two executed workflows in order to allow execution of combined checks that refer to multiple documents or document sides. If the workflow status of the workflow `checkWorkflowID` is ok or finished, the workflow status is set to ok and all configured combined checks are executed. Combined checks that require information from a different document check will fetch this information from workflow `referredWorkflowID`. If the workflow status of the workflow `checkWorkflowID` is cancelled, calling `linkWorkflow` has no effect. If `checkWorkflowID` and `referredWorkflowID` are part of different check transactions, it is RECOMMENDED to call tl5:`mergeTransaction` (see ▸Section 8.4.8) for these transactions.

6.4.8.1 Request elements

The linkWorkflow request elements are given in ▸Table 6.39.

Element name	Description
`checkWorkflowID`	`hldc5:UUID` Reference to the workflow that contains combined checks which refer to multiple documents or multiple sides of a document.
`referredWorkflowID`	`hldc5:UUID` Reference to the workflow from which additional data is fetched.

Table 6.39 linkWorkflow - request elements

6.4.8.2 Response elements

None.

6.4.8.3 Faults

The linkWorkflow faults are given in ▸Table 6.40.

Fault type	Cause
`hldc5:InvalidWorkflowId`	Value of `checkWorkflowID` or `referredWorkflowID` is invalid or has expired or values of `checkWorkflowID` are `referredWorkflowID` identical.
`tl5:ReadOnly`	The transaction used by `checkWorkflowID`is write-protected and must not be modified.

Table 6.40 linkWorkflow - faults

6.4.8.4 WSDL Definition

```
<!-- operation request element -->
<element name="linkWorkflow">
 <complexType>
  <sequence>
   <element name="checkWorkflowID" type="hldc5:UUID" />
   <element name="referredWorkflowID" type="hldc5:UUID" />
  </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="linkWorkflowResponse">
  <complexType />
</element>
```

6.4.9 removeWorkflow

Removes a workflow definition which was previously loaded with `addWorkflow`. The server SHALL ensure that workflow removal does not affect the execution of currently running workflows.

6.4.9.1 Request elements

The removeWorkflow faults are given in ▸Table 6.41.

Element name	Description
`workflowName`	`xsd:string` The name of the workflow to remove.

Table 6.41 removeWorkflow - request elements

6.4.9.2 Response elements

None.

6.4.9.3 Faults

The removeWorkflow faults are given in ▸Table 6.42.

Fault type	Cause
`hldc5:WorkflowNotFound`	The workflow `workflowName` does not exist on the server.

Table 6.42 removeWorkflow - faults

6.4.9.4 WSDL Definition

```
<!-- operation request element -->
<element name="removeWorkflow">
 <complexType>
  <sequence>
   <element name="workflowName" type="xsd:string" />
 </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="removeWorkflowResponse">
  <complexType />
</element>
```

6.4.10 triggerWorkflow

Triggers workflow execution on the server to check the current document and/or read out the requested data without document detection. Check results and data SHALL be queried individually by calling `getWorkflowFeedback` or `getWorkflowFeedbackById`.

6.4.10.1 Request elements

The triggerWorkflow request elements are given in ▸Table 6.43.

Element name	Description
`workflowID`	`hldc5:UUID` The ID of the workflow to execute. SHALL be a valid workflow ID as returned by `beginWorkflow`.
`docIdentifier?`	`xsd:string` MRZ or CAN to be used to access the document instead of the automatically retrieved (if available) MRZ/CAN.
`binary?`	`xsd:base64Binary` This parameter MAY be used by the client to pass external binary data which is used in the workflow execution (e.g. DTC).

Element name	Description
parameters?	hldc5:WorkflowParameters List of key-value pairs which override the default values of the hldc5:ParameterCondition entries in the workflow definition. The IDs SHALL match the workflow definition.

Table 6.43 triggerWorkflow - request elements

6.4.10.2 Response elements

The triggerWorkflow response elements are given in Table 6.44.

Element name	Description
errorCode?	hldc5:type.shared.errorcode Optional error code.

Table 6.44 triggerWorkflow - response elements

6.4.10.3 Faults

The triggerWorkflow faults are given in Table 6.45.

Fault	Cause
hldc5:InvalidConditionId	parameters contains at least one condition that does not match the workflow definition.
hldc5:invalidWorkflowId	Value of workflowID is invalid or has expired.
hldc5:InvalidOperation	The workflow is already in progress i.e. the workflow does not wait for a document.

Table 6.45 triggerWorkflow - faults

6.4.10.4 WSDL Definition

```
<!-- operation request element -->
<element name="triggerWorkflow">
 <complexType>
  <sequence>
   <element name="workflowID" type="hldc5:UUID"/>
   <element name="docIdentifier" type="xsd:string" minOccurs="0" maxOccurs="1" />
   <element name="binary" type="xsd:base64Binary" minOccurs="0" maxOccurs="1" />
   <element name="parameters" type="hldc5:WorkflowParameters" minOccurs="0" maxOccurs="1" />
  </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="triggerWorkflowResponse">
 <complexType>
  <sequence>
   <element name="errorCode" type="hldc5:type.shared.errorcode" minOccurs="0" maxOccurs="1"/>
  </sequence>
 </complexType>
</element>
```

6.4.11 continueWorkflow

Continues workflow execution after the server sent a user interaction feedback. The parameter of the operation allows to transmit further information (i.e. user decisions).

6.4.11.1 Request elements

The continueWorkflow request elements are given in Table 6.46.

Element name	Description
`workflowID`	`hldc5:UUID` The ID of the workflow to continue. SHALL be a valid workflow ID as returned by `beginWorkflow`.
`parameter?`	`hldc5:ContinueWorkflowParameter` A key-value for the user decision. If the user interaction requires user data, but the parameter is ommited, then the server will continue with default behaviour (e.g. in case that the user does not select a document series).

Table 6.46 triggerWorkflow - request elements

6.4.11.2 Response elements

The continueWorkflow response elements are given in ▸Table 6.47.

Element name	Description
`workflowStatus`	`hldc5:workflowStatus` The workflow status after the continuation was requested.

Table 6.47 continueWorkflow - response elements

6.4.11.3 Faults

The continueWorkflow faults are given in ▸Table 6.48.

Fault	Cause
`hldc5:InvalidParameter`	`parameter` is invalid.
`hldc5:invalidWorkflowId`	Value of `workflowID` is invalid or has expired.
`hldc5:InvalidOperation`	The workflow is already in progress i.e. the workflow does not wait for user interactions.

Table 6.48 triggerWorkflow - faults

6.4.11.4 WSDL Definition

```
<!-- operation request element -->
<element name="continueWorkflow">
 <complexType>
  <sequence>
   <element name="workflowID" type="hldc5:UUID"/>
   <element name="parameter" type="hldc5:ContinueWorkflowParameter" minOccurs="0" maxOccurs="1" /
>
  </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="continueWorkflowResponse">
 <complexType>
  <sequence>
   <element name="workflowStatus" type="hldc5:WorkflowStatus"/>
  </sequence>
 </complexType>
</element>
```

6.5 Workflow definition schema

This section defines all elements of a workflow definition.

6.5.1 Workflow document

XML document that configures the execution workflow of the document check according to TR#03135 Part 1.

6.5.1.1 Root element

The Workflow root element is given in Table 6.49.

Element name	Description
`Workflow`	`wf:type.workflow` Root element of the workflow definition.

Table 6.49 Workflow - root element

6.5.1.2 XSD Definition

```
<xs:element name="Workflow" type="wf:type.workflow" />
```

6.5.2 type.workflow

Root element of a HLDC5 workflow definition.

6.5.2.1 Attributes

The type.workflow attributes are given in Table 6.50.

Attribute name	Description
`schemaVersion`	`xs:decimal` The schemaVersion currently has the value 1.

Table 6.50 type.workflow - attributes

6.5.2.2 Elements

The type.workflow elements are given in Table 6.51.

Element name	Description
`Information`	`wf:type.workflow.information` Basic information about the workflow.
`ApplicationScenario`	`wf:type.workflow.scenario` Information about the application scenario of the workflow.
`Conditions?`	`wf:type.workflow.conditions` Conditions for certain workflow actions.
`CheckSequences?`	`wf:type.workflow.sequences` Encapsulates all check sequences.
`Feedback?`	`wf:type.workflow.feedback` Defines in which format the read data should be delivered to the application.
`Dependencies?`	`wf:type.workflow.dependencies` Defines dependencies between workflow actions.
`Extension?`	`wf:type.workflow.extension` Root node for implementation-specific workflow extensions.

Table 6.51 type.workflow - elements

6.5.2.3 XSD Definition

```
<xs:complexType name="type.workflow">
 <xs:sequence>
  <xs:element name="Information" type="wf:type.workflow.information"/>
  <xs:element name="ApplicationScenario" type="wf:type.workflow.scenario"/>
  <xs:element name="Conditions" type="wf:type.workflow.conditions" minOccurs="0"/>
  <xs:element name="CheckSequences" type="wf:type.workflow.sequences" minOccurs="0"/>
  <xs:element name="Feedback" type="wf:type.workflow.feedback" minOccurs="0"/>
  <xs:element name="Dependencies" type="wf:type.workflow.dependencies" minOccurs="0"/>
  <xs:element name="Extension" type="wf:type.workflow.extension" minOccurs="0"/>
 </xs:sequence>
 <xs:attribute name="schemaVersion" type="xs:decimal" use="required"/>
</xs:complexType>
```

6.5.3 type.workflow.information

Basic information about the workflow. SHALL provide a workflow name and MAY contain additional information about the vendor, version and a textual description of the workflow.

6.5.3.1 Attributes

None.

6.5.3.2 Elements

The type.workflow elements are given in Table 6.52.

Element name	Description
`Vendor?`	`xs:string` Vendor of the workflow.
`Name`	`xs:string` Name of the workflow. Serves as a reference in some calls. SHALL NOT be empty.
`Version?`	`xs:string` Version of the workflow.
`Description?`	`xs:string` Textual description of the workflow.

Table 6.52 type.workflow.information - elements

6.5.3.3 XSD Definition

```
<xs:complexType name="type.workflow.information">
 <xs:sequence>
  <xs:element name="Vendor" minOccurs="0"  type="xs:string" />
  <xs:element name="Name"  >
   <xs:simpleType>
    <xs:restriction base="xs:string">
     <xs:minLength value="1" />
    </xs:restriction>
   </xs:simpleType>
  </xs:element>
  <xs:element name="Version" minOccurs="0"  type="xs:string" />
  <xs:element name="Description" minOccurs="0"  type="xs:string" />
 </xs:sequence>
</xs:complexType>
```

6.5.4 type.workflow.scenario

Selection of the server-defined application scenario which configures which checks are executed for a given document. The scenario MAY be customized with respect to specific checks. Additionally, user interactions and live data MAY be configured.

6.5.4.1 Attributes

The type.workflow.scenario attributes are given in Table 6.53.

Attribute name	Description
`preset`	`xs:string` Selects a configuration preset. The available presets are server-defined.

Table 6.53 type.workflow.scenario - attributes

6.5.4.2 Elements

The type.workflow.scenario elements are given in Table 6.54.

Element name	Description
`CheckConfiguration?`	`wf:type.workflow.scenario.checkconfiguration` Configuration of optical checks.
`Interactions?`	`wf:type.workflow.interactions` Configuration of electronic checks.
`LiveStream?`	`wf:type.workflow.scenario.livestream` Configuration of single-document combined checks.

Table 6.54 type.workflow.scenario - elements

6.5.4.3 XSD Definition

```
<xs:complexType name="type.workflow.scenario">
 <xs:sequence>
  <xs:element name="CheckConfiguration" type="wf:type.workflow.scenario.checkconfiguration"
                                                                              minOccurs="0"/>
  <xs:element name="Interactions" type="wf:type.workflow.interactions" minOccurs="0"/>
  <xs:element name="LiveStream" type="wf:type.workflow.scenario.livestream" minOccurs="0"/>
 </xs:sequence>
 <xs:attribute name="preset" type="xs:string" use="required"/>
</xs:complexType>
```

6.5.5 type.workflow.scenario.checkconfiguration

Selection of the server-defined application scenario which configures which checks are executed for a given document. The scenario MAY be customized with respect to specific checks. Additionally, user interactions and live data MAY be configured.

6.5.5.1 Attributes

The type.workflow.scenario.checkconfiguration attributes are given in Table 6.55.

Attribute name	Description
`preset`	`xs:string` Selects a configuration preset. The available presets are server-defined.

Table 6.55 type.workflow.scenario - attributes

6.5.5.2 Elements

The type.workflow.scenario.checkconfiguration elements are given in Table 6.56.

Element name	Description
OpticalChecks?	wf:type.workflow.scenario.checks Configuration of optical checks.
ElectronicChecks?	wf:type.workflow.scenario.checks Configuration of electronic checks.
CombinedChecks?	wf:type.workflow.scenario.checks Configuration of single-document combined checks.
CrossDocumentChecks?	wf:type.workflow.scenario.checks Configuration of cross-document combined checks.
SealChecks?	wf:type.workflow.scenario.checks Configuration of seal checks.
AggregatedChecks?	wf:type.workflow.scenario.checks Configuration of the overall result aggregation.

Table 6.56 type.workflow.scenario - elements

6.5.5.3 XSD Definition

```
<xs:complexType name="type.workflow.scenario.checkconfiguration">
 <xs:sequence>
  <xs:element name="OpticalChecks" minOccurs="0"
         type="wf:type.workflow.scenario.checks"/>
  <xs:element name="ElectronicChecks" minOccurs="0"
         type="wf:type.workflow.scenario.checks "/>
  <xs:element name="CombinedChecks" minOccurs="0"
         type="wf:type.workflow.scenario.checks "/>
  <xs:element name="CrossDocumentChecks" minOccurs="0"
         type="wf:type.workflow.scenario.checks "/>
  <xs:element name="SealChecks" minOccurs="0"
         type="wf:type.workflow.scenario.checks "/>
  <xs:element name="AggregatedChecks" minOccurs="0"
         type="wf:type.workflow.scenario.checks "/>
 </xs:sequence>
 <xs:attribute name="preset" type="xs:string" use="required"/>
</xs:complexType>
```

6.5.6 type.workflow.scenario.livestream

Customization of the application scenario preset with respect to a particular check.

6.5.6.1 Attributes

The type.workflow.scenario.livestream attributes are given in ▸Table 6.57.

Attribute name	Description
configuration	wf:type.workflow.scenario.livestream.options Options for the livestream configuration
format	wf:type.workflow.feedback.image.format Determines whether the check should be executed or skipped.
size	xs:string Requested image size in format x (e.g. 1000x800)

Table 6.57 type.workflow.scenario.livestream - attributes

6.5.6.2 Elements

None.

6.5.6.3 XSD Definition

```
<xs:complexType name="type.workflow.scenario.livestream">
 <xs:attribute name="configuration" type="wf:type.workflow.scenario.livestream.options"
                                                                     use="required"/>
 <xs:attribute name="format" type="wf:type.workflow.feedback.image.format"/>
 <xs:attribute name="size" type="xs:string"/>
</xs:complexType>
```

6.5.7 type.workflow.scenario.livestream.options

List of configuration options of the livestream.

6.5.7.1 Values

The type.workflow.scenario.livestream.options values are given in Table 6.58.

Value	Description
inactive	The livestream shall be inactive.
continuous	The server SHALL send new images continuously.
changes only	The server only sends image if the document status or position changes. It is RECOMMENDED to use this option to avoid unnecessary traffic.

Table 6.58 type.workflow.scenario.livestream.options - values

6.5.7.2 XSD Definition

```
<xs:simpleType name="type.workflow.scenario.livestream.options">
 <xs:restriction base="xs:string">
  <xs:enumeration value="inactive"/>
  <xs:enumeration value="continuous"/>
  <xs:enumeration value="changes only"/>
 </xs:restriction>
</xs:simpleType>
```

6.5.8 type.workflow.interactions

Configuration of user interactions.

6.5.8.1 Attributes

None.

6.5.8.2 Elements

The type.workflow.interactions elements are given in Table 6.59.

Element name	Description
TextCorrection?	wf:type.workflow.interactions.textcorrection Configuration of text correction requests. If defined in the workflow the server SHALL send text correction requests. If ommited the server SHALL not send requests.
DocumentSeriesSelection?	wf:type.workflow.interactions.seriesselection Configuration of document series selection requests. If defined in the workflow the server SHALL send document series selection requests. If ommited the server SHALL not send requests.
PageRequests?	wf:type.workflow.interactions.pagerequests Configuration document page requests. If defined in the workflow the server SHALL send page requests. If ommited the server SHALL not send requests.

Table 6.59 type.workflow.interactions - elements

6.5.8.3 XSD Definition

```
<xs:complexType name="type.workflow.interactions">
 <xs:sequence>
  <xs:element name="TextCorrection" type="wf:type.workflow.interactions.textcorrection"
                                                       minOccurs="0" maxOccurs="1"/>
  <xs:element name="DocumentSeriesSelection" type="wf:type.workflow.interactions.seriesselection"
                                                       minOccurs="0" maxOccurs="1"/>
  <xs:element name="PageRequests" type="wf:type.workflow.interactions.pagerequests"
                                                       minOccurs="0" maxOccurs="1"/>
 </xs:sequence>
</xs:complexType>
```

6.5.9 type.workflow.interactions.textcorrection

Configuration of user interactions in context of text corrections.

6.5.9.1 Attributes

The type.workflow.interactions.textcorrection attributes are given in Table 6.60.

Attribute name	Description
condition?	xs:string Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.60 type.workflow.interactions.textcorrection - attributes

6.5.9.2 Elements

The type.workflow.interactions.textcorrection elements are given in Table 6.61.

Element name	Description
MRZOCR?	wf:type.workflow.interactions.textcorrection.ocr Configuration of text correction request for the OCR of the MRZ.
CANOCR?	wf:type.workflow.interactions.textcorrection.ocr Configuration of text correction request for the OCR of the CAN.
MRZChipAccess?	wf:type.workflow.interactions.textcorrection.chip Configuration of text correction request for chip access by MRZ.
CANChipAccess?	wf:type.workflow.interactions.textcorrection.chip Configuration of text correction request for chip access by CAN.

Table 6.61 type.workflow.interactions.textcorrection - elements

6.5.9.3 XSD Definition

```
<xs:complexType name="type.workflow.interactions.textcorrection">
 <xs:sequence>
  <xs:element name="MRZOCR" type="wf:type.workflow.interactions.textcorrection.ocr"
                                                       minOccurs="0" maxOccurs="1"/>
  <xs:element name="CANOCR" type="wf:type.workflow.interactions.textcorrection.ocr"
                                                       minOccurs="0" maxOccurs="1"/>
  <xs:element name="MRZChipAccess" type="wf:type.workflow.interactions.textcorrection.chip"
                                                       minOccurs="0" maxOccurs="1"/>
  <xs:element name="CANChipAccess" type="wf:type.workflow.interactions.textcorrection.chip"
                                                       minOccurs="0" maxOccurs="1"/>
 </xs:sequence>
 <xs:attribute name="condition" type="xs:string"/>
</xs:complexType>
```

6.5.10 type.workflow.interactions.textcorrection.chip

Configuration of user interactions for text corrections for chip access.

.

6.5.10.1 Attributes

The type.workflow.interactions.textcorrection.chip attributes are given in ▸Table 6.62.

Attribute name	Description
maxAttempts?	xs:int Maximum attempts for the correction to access the chip.
condition?	xs:string Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.62 type.workflow.interactions.textcorrection.chip - attributes

6.5.10.2 Elements

None.

6.5.10.3 XSD Definition

```
<xs:complexType name="type.workflow.interactions.textcorrection.chip">
 <xs:attribute name="maxAttempts" type="xs:int"/>
 <xs:attribute name="condition" type="xs:string"/>
</xs:complexType>
```

6.5.11 type.workflow.interactions.textcorrection.ocr

Configuration of user interactions for text corrections for OCR.

.

6.5.11.1 Attributes

The type.workflow.interactions.textcorrection.ocr attributes are given in ▸Table 6.63.

Attribute name	Description
condition?	xs:string Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.63 type.workflow.interactions.textcorrection.ocr - attributes

6.5.11.2 Elements

None.

6.5.11.3 XSD Definition

```
<xs:complexType name="type.workflow.interactions.textcorrection.ocr">
 <xs:attribute name="condition" type="xs:string"/>
</xs:complexType>
```

6.5.12 type.workflow.interactions.seriesselection

Configuration of user interactions in context of document series selection.

6.5.12.1 Attributes

The type.workflow.interactions.seriesselection attributes are given in ▸Table 6.64.

Attribute name	Description
condition?	xs:string Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.64 type.workflow.interactions.seriesselection - attributes

6.5.12.2 Elements

None.

6.5.12.3 XSD Definition

```
<xs:complexType name="type.workflow.interactions.seriesselection">
 <xs:attribute name="condition" type="xs:string"/>
</xs:complexType>
```

6.5.13 type.workflow.interactions.pagerequests

Configuration of user interactions in context of document page requests.

6.5.13.1 Attributes

The type.workflow.interactions.pagerequests attributes are given in Table 6.65.

Attribute name	Description
condition?	xs:string Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.65 type.workflow.interactions.pagerequests - attributes

6.5.13.2 Elements

The type.workflow.interactions.pagerequests elements are given in Table 6.66.

Element name	Description
PageRequest+	wf:type.workflow.interactions.pagerequest Configuration of a single page request.

Table 6.66 type.workflow.interactions.pagerequests - elements

6.5.13.3 XSD Definition

```
<xs:complexType name="type.workflow.interactions.pagerequests">
 <xs:sequence>
  <xs:element name="PageRequest" type="wf:type.workflow.interactions.pagerequest"
                                                  maxOccurs="unbounded"/>
 </xs:sequence>
 <xs:attribute name="condition" type="xs:string"/>
</xs:complexType>
```

6.5.14 type.workflow.interactions.pagerequest

Configuration of user interactions in context of document page requests encapsulating a specific document.

6.5.14.1 Attributes

The type.workflow.interactions.pagerequest attributes are given in Table 6.67.

Attribute name	Description
condition?	xs:string Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.67 type.workflow.interactions.pagerequest - attributes

6.5.14.2 Elements

The type.workflow.interactions.pagerequest elements are given in Table 6.68.

Element name	Description
DocumentType	wf:type.shared.enum.document.semantic.type.extended Document type for the request.
PageList	wf:type.workflow.interactions.pagerequest.list List of pages for the given document which have to be requested.

Table 6.68 type.workflow.interactions.pagerequest - elements

6.5.14.3 XSD Definition

```
<xs:complexType name="type.workflow.interactions.pagerequest">
 <xs:sequence>
  <xs:element name="DocumentType" type="wf:type.shared.enum.document.semantic.type.extended"/>
  <xs:element name="PageList" type="wf:type.workflow.interactions.pagerequest.list"/>
 </xs:sequence>
 <xs:attribute name="condition" type="xs:string"/>
</xs:complexType>
```

6.5.15 type.workflow.interactions.pagerequest.list

List of pages for a given document for page requests.

6.5.15.1 Attributes

None.

6.5.15.2 Elements

The type.workflow.interactions.pagerequest.list elements are given in Table 6.69.

Element name	Description
Page	wf:type.workflow.interactions.pagerequest.list.page A single document page for the request.

Table 6.69 type.workflow.interactions.pagerequest.list - elements

6.5.15.3 XSD Definition

```
<xs:complexType name="type.workflow.interactions.pagerequest.list">
 <xs:sequence>
  <xs:element name="Page" type="wf:type.workflow.interactions.pagerequest.list.page"
                                        minOccurs="1" maxOccurs="unbounded"/>
 </xs:sequence>
</xs:complexType>
```

6.5.16 type.workflow.interactions.pagerequest.list.page

Configuration of user interactions in context of document page requests encapsulating a specific document.

6.5.16.1 Attributes

The type.workflow.interactions.pagerequestlist.page attributes are given in ▸Table 6.70.

Attribute name	Description
attributeGoup	wf:attributeGroup.shared.page.classification.extended Group of attibutes to identify a specific document page.

Table 6.70 type.workflow.interactions.pagerequestlist.page - attributes

6.5.16.2 Elements

None.

6.5.16.3 XSD Definition

```
<xs:complexType name="type.workflow.interactions.pagerequest.list.page">
 <xs:attributeGroup ref="wf:attributeGroup.shared.page.classification.extended"/>
</xs:complexType>
```

6.5.17 type.workflow.scenario.checks

Workflow preset customization for the corresponding type of checks.

6.5.17.1 Attributes

None.

6.5.17.2 Elements

The type.workflow.scenario.checks elements are given in ▸Table 6.71.

Element name	Description
Check*	wf:type.workflow.scenario.check Configuration of a single check.

Table 6.71 type.workflow.scenario.checks - elements

6.5.17.3 XSD Definition

```
<xs:complexType name="type.workflow.scenario.checks">
 <xs:sequence>
  <xs:element name="Check" minOccurs="0" maxOccurs="unbounded"
         type="wf:type.workflow.scenario.check"/>
 </xs:sequence>
</xs:complexType>
```

6.5.18 type.workflow.scenario.check

Customization of the application scenario preset with respect to a particular check.

6.5.18.1 Attributes

The type.workflow.scenario.check attributes are given in ▸Table 6.72.

Attribute name	Description
id	xs:string The server-defined ID of the check to configure.
action	wf:type.workflow.scenario.check.action Determines whether the check should be executed or skipped.

Attribute name	Description
condition?	xs:IDREF Reference to a condition. Controls under which circumstances this check configuration is applicable SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.72 type.workflow.scenario.check - attributes

6.5.18.2 Elements

None.

6.5.18.3 XSD Definition

```
<xs:complexType name="type.workflow.scenario.check">
 <xs:attribute name="id" type="xs:ID" use="required" />
 <xs:attribute name="action"
        type="wf:type.workflow.scenario.check.action" use="required" />
 <xs:attribute name="condition" type="xs:IDREF" />
</xs:complexType>
```

6.5.19 type.workflow.scenario.check.action

Determines whether a check is executed or not (subject to conditional application). Derived from `xs:string`.

6.5.19.1 Values

The type.workflow.scenario.check.action values are given in Table 6.73.

Element name	Description
exec	The check SHALL be performed if it is technically possible.
skip	The check SHALL be skipped.
eval	The check SHALL be performed if it is technically possible but the check result SHALL NOT influence the overall result.

Table 6.73 type.workflow.scenario.check.action - values

6.5.19.2 XSD Definition

```
<xs:simpleType name="type.workflow.scenario.check.action">
 <xs:restriction base="xs:string">
  <xs:enumeration value="exec" />
  <xs:enumeration value="skip" />
  <xs:enumeration value="eval" />
 </xs:restriction>
</xs:simpleType>
```

6.5.20 type.workflow.conditions

List of conditions that MAY be used in other parts of the workflow definition to control workflow execution.

6.5.20.1 Attributes

None.

6.5.20.2 Elements

The elements MAY appear in any order.

The type.workflow.conditions elements are given in Table 6.74.

Element name	Description
MRZCondition*	wf:type.workflow.conditions.mrz Conditions which evaluate data from the optical MRZ or, if CAN is used, from DG1
HasCANCondition*	wf:type.workflow.conditions.hascan Conditions which evaluate if a CAN was detected.
HasChipCondition*	wf:type.workflow.conditions.haschip Conditions which evaluate if an electronic chip was detected.
ParameterCondition*	wf:type.workflow.conditions.parameter Conditions whose fulfillment is controlled by the application.
AndCondition*	wf:type.workflow.conditions.and Conditions that constitute a logical conjunction of two conditions.
OrCondition*	wf:type.workflow.conditions.or Conditions that constitute a logical disjunction of two conditions.
NotCondition*	wf:type.workflow.conditions.not Conditions that negate the result of another condition.
XMLCondition*	wf:type.workflow.conditions.xml Conditions based on XPath expressions.

Table 6.74 type.workflow.conditions - elements

6.5.20.3 XSD Definition

```
<xs:complexType name="type.workflow.conditions">
 <xs:choice minOccurs="0" maxOccurs="unbounded">
  <xs:element name="MRZCondition"
         type="wf:type.workflow.conditions.mrz" />
  <xs:element name="HasCANCondition"
         type="wf:type.workflow.conditions.hascan" />
  <xs:element name="HasChipCondition"
         type="wf:type.workflow.conditions.haschip" />
  <xs:element name="ParameterCondition"
         type="wf:type.workflow.conditions.parameter" />
  <xs:element name="AndCondition"
         type="wf:type.workflow.conditions.and" />
  <xs:element name="OrCondition"
         type="wf:type.workflow.conditions.or" />
  <xs:element name="NotCondition"
         type="wf:type.workflow.conditions.not" />
  <xs:element name="XMLCondition"
         type="wf:type.workflow.conditions.xml" />
 </xs:choice>
</xs:complexType>
```

6.5.21 type.workflow.conditions.mrz

Compares data from the optical MRZ to the specified data. The condition is fulfilled if the data matches or if `type`, `issuer` and `documentNumber` are empty. If Password Authenticated Connection Establishment (PACE) with CAN is performed, the MRZ data is taken from DG1.

6.5.21.1 Attributes

The type.workflow.conditions.mrz attributes are given in Table 6.75.

Attribute name	Description
id	xs:ID User-defined unique ID of the condition. Serves as reference.

Attribute name	Description
`type?`	`xs:string` Perl-compatible regular expression (PCRE) to match the document type. Fill characters SHALL be ignored. SHALL match if empty or attribute was omitted.
`issuer?`	`xs:string` PCRE to match the issuer. Fill characters SHALL be ignored. SHALL match if empty or attribute was omitted.
`documentNumber?`	`xs:string` PCRE to match the document number. Fill characters SHALL be ignored. SHALL match if empty or attribute was omitted.

Table 6.75 type.workflow.conditions.mrz - attributes

6.5.21.2 Elements

None.

6.5.21.3 XSD Definition

```
<xs:complexType name="type.workflow.conditions.mrz">
  <xs:attribute name="id" type="xs:ID" use="required" />
  <xs:attribute name="type" type="xs:string" />
  <xs:attribute name="issuer" type="xs:string" />
  <xs:attribute name="documentNumber" type="xs:string" />
</xs:complexType>
```

6.5.22 type.workflow.conditions.hascan

Condition which is fulfilled if a card access number (CAN) is detected on the document.

6.5.22.1 Attributes

The type.workflow.conditions.hascan attributes are given in Table 6.76.

Attribute name	Description
`id`	`xs:ID` User-defined unique ID of the condition. Serves as reference.

Table 6.76 type.workflow.conditions.hascan - attributes

6.5.22.2 Elements

None.

6.5.22.3 XSD Definition

```
<xs:complexType name="type.workflow.conditions.hascan">
  <xs:attribute name="id" type="xs:ID" use="required" />
</xs:complexType>
```

6.5.23 type.workflow.conditions.parameter

Condition whose fulfillment is controlled by the application. The application can override the value of the condition via `beginWorkflow`.

6.5.23.1 Attributes

The type.workflow.conditions.parameter attributes are given in Table 6.77.

Attribute name	Description
id	xs:ID User-defined unique ID of the condition. Serves as reference.
default	xs:boolean Default value of the condition.

Table 6.77 type.workflow.conditions.parameter - attributes

6.5.23.2 Elements

None.

6.5.23.3 XSD Definition

```
<xs:complexType name="type.workflow.conditions.parameter">
 <xs:attribute name="id" type="xs:ID" use="required" />
 <xs:attribute name="default" type="xs:boolean" use="required" />
</xs:complexType>
```

6.5.24 type.workflow.conditions.and

Constitutes a logical conjunction of multiple conditions. The condition is fulfilled if and only if all referenced conditions are fulfilled.

6.5.24.1 Attributes

The type.workflow.conditions.and attributes are given in Table 6.78.

Attribute name	Description
id	xs:ID Unique ID of the condition. Serves as reference.

Table 6.78 type.workflow.conditions.and - attributes

6.5.24.2 Elements

The type.workflow.conditions.and elements are given in Table 6.79.

Element name	Description
c+	xs:IDREF Reference to a condition. SHALL be considered fulfilled if empty.

Table 6.79 type.workflow.conditions.and - elements

6.5.24.3 XSD Definition

```
<xs:complexType name="type.workflow.conditions.and">
 <xs:sequence>
  <xs:element name="c" type="xs:IDREF" maxOccurs="unbounded" />
 </xs:sequence>
 <xs:attribute name="id" type="xs:ID" use="required" />
</xs:complexType>
```

6.5.25 type.workflow.conditions.or

Constitutes a logical conjunction of multiple conditions. The condition is fulfilled if and only if at least one referenced condition is fulfilled.

6.5.25.1 Attributes

The type.workflow.conditions.or attributes are given in Table 6.80.

Attribute name	Description
id	xs:ID User-defined unique ID of the condition which is used as reference.

Table 6.80 type.workflow.conditions.or - attributes

6.5.25.2 Elements

The type.workflow.conditions.or elements are given in ▸Table 6.81.

Element name	Description
c+	xs:IDREF Reference to a condition. SHALL be considered fulfilled if empty.

Table 6.81 type.workflow.conditions.or - elements

6.5.25.3 XSD Definition

```
<xs:complexType name="type.workflow.conditions.or">
  <xs:sequence>
   <xs:element name="c" type="xs:IDREF" maxOccurs="unbounded" />
  </xs:sequence>
  <xs:attribute name="id" type="xs:ID" use="required" />
</xs:complexType>
```

6.5.26 type.workflow.conditions.not

Constitutes a logical negation of another condition. The condition is fulfilled if and only if the referenced condition is not fulfilled.

6.5.26.1 Attributes

The type.workflow.conditions.not attributes are given in ▸Table 6.82.

Attribute name	Description
id	xs:ID User-defined unique ID of the condition which is used as reference.
c	xs:IDREF Reference to a condition. SHALL be considered fulfilled if empty.

Table 6.82 type.workflow.conditions.not - attributes

6.5.26.2 Elements

None.

6.5.26.3 XSD Definition

```
<xs:complexType name="type.workflow.conditions.not">
 <xs:attribute name="id" type="xs:ID" use="required" />
 <xs:attribute name="c" type="xs:IDREF" use="required" />
</xs:complexType>
```

6.5.27 type.workflow.conditions.xml

Compares the result of an XPath 1.0 expression with a specified value. If XPath result text and value match, the condition is fulfilled.

6.5.27.1 Attributes

The type.workflow.conditions.xml attributes are given in ▸Table 6.83.

Attribute name	Description
`id`	`xs:ID` User-defined unique ID of the condition. Serves as reference.
`from`	`xs:IDREF` Reference to a data element.
`profile`	`xs:string` Logging profile on which the evaluation of this condition is based.
`path`	`xs:string` XPath 1.0 expression which selects a node in the XML. If XPath expression matches multiple nodes, the comparison of this condition SHALL consider the first matching node only. The comparison SHALL only consider the inner text of the respective node.
`value`	`xs:string` The comparative value.

Table 6.83 type.workflow.conditions.xml - attributes

6.5.27.2 Elements

None.

6.5.27.3 XSD Definition

```
<xs:complexType name="type.workflow.conditions.xml" />
 <xs:attribute name="id" type="xs:ID" use="required" />
 <xs:attribute name="from" type="xs:IDREF" use="required" />
 <xs:attribute name="profile" type="xs:string" use="required"/>
 <xs:attribute name="path" type="xs:string" use="required" />
 <xs:attribute name="value" type="xs:string" use="required" />
</xs:complexType>
```

6.5.28 type.workflow.sequences

Encapsulates the reading and checking processes.

6.5.28.1 Attributes

The type.workflow.sequences attributes are given in Table 6.84.

Attributes name	Description
`condition?`	`xs:IDREF` Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.84 type.workflow.sequences - attributes

6.5.28.2 Elements

The type.workflow.sequences elements are given in Table 6.85.

Element name	Description
`AggregatedCheck?`	`wf:type.workflow.aggregation` Defines an overall check result that agrregates the results of optical, electronic and seal checks.
`ElectronicCheck?`	`wf:type.workflow.electronic` Defines which electronic data shall be read and when the electronic checks should be performed.

Element name	Description
OpticalCheck?	wf:type.workflow.optical Defines which optical data shall be read and when the optical checks should be performed.
SealCheck?	wf:type.workflow.seal Defines which barcode data shall be read and when the seal checks should be performed.
CombinedCheck?	wf:type.workflow.combined Defines which combined checks shall be performed.
CrossCheck?	wf:type.workflow.cross Defines which cross-document checks shall be performed.

Table 6.85 type.workflow.sequences - elements

6.5.28.3 XSD Definition

```
<xs:complexType name="type.workflow.sequences">
 <xs:sequence>
  <xs:element name="AggregatedCheck" type="wf:type.workflow.aggregation" minOccurs="0"/>
  <xs:element name="ElectronicCheck" type="wf:type.workflow.electronic" minOccurs="0"/>
  <xs:element name="OpticalCheck" type="wf:type.workflow.optical" minOccurs="0"/>
  <xs:element name="SealCheck" type="wf:type.workflow.seal" minOccurs="0"/>
  <xs:element name="CombinedCheck" type="wf:type.workflow.combined" minOccurs="0"/>
  <xs:element name="CrossCheck" type="wf:type.workflow.cross" minOccurs="0"/>
 </xs:sequence>
 <xs:attribute name="condition" type="xs:IDREF"/>
</xs:complexType>
```

6.5.29 type.workflow.aggregation

Configuration of check process with respect to the aggregation of check results.

6.5.29.1 Attributes

The type.workflow.aggregation attributes are given in ▸Table 6.86.

Attributes name	Description
condition?	xs:IDREF Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.86 type.workflow.aggregation - attributes

6.5.29.2 Elements

The type.workflow.aggregation elements are given in ▸Table 6.87.

Element name	Description
ReadSequence	wf:type.workflow.aggregation.readseq A sequence of the reading process.

Table 6.87 type.workflow.aggregation- elements

6.5.29.3 XSD Definition

```
 <xs:complexType name="type.workflow.aggregation">
  <xs:sequence>
   <xs:element name="ReadSequence" type="wf:type.workflow.aggregation.readseq"/>
  </xs:sequence>
  <xs:attribute name="condition" type="xs:IDREF"/>
```

```
</xs:complexType>
```

6.5.30 type.workflow.aggregation.readseq

The reading sequence.

6.5.30.1 Attributes

None.

6.5.30.2 Elements

The type.workflow.aggregation.readseq elements are given in Table 6.87.

Element name	Description
AggregatedDocumentCheckResult	wf:type.workflow.readseq.check A sequence of checks.

Table 6.88 type.workflow.aggregation.readseq - elements

6.5.30.3 XSD Definition

```
<xs:complexType  name="type.workflow.aggregation.readseq">
 <xs:choice minOccurs="0" maxOccurs="unbounded">
  <xs:element name="AggregatedDocumentCheckResult" type="wf:type.workflow.readseq.check"/>
 </xs:choice>
</xs:complexType>
```

6.5.31 type.workflow.electronic

Configuration of the electronic reading and check process with respect to access protocols and the reading sequence.

6.5.31.1 Attributes

The type.workflow.electronic attributes are given in Table 6.89.

Attribute name	Description
condition?	xs:IDREF Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.89 type.workflow.electronic - attributes

6.5.31.2 Elements

The type.workflow.electronic elements are given in Table 6.90.

Element name	Description
AccessConfiguration?	wf:type.workflow.electronic.access Conditional configuration of access protocols (Basic Access Control (BAC), PACE, PLAIN). MAY override server-side configuration.
EACConfiguration?	wf:type.workflow.electronic.eac Conditional configuration of the extended access protocols (EAC) after PACE. MAY override server-side configuration.
ChipConfiguration?	wf:type.workflow.electronic.chip Conditional timeouts for chip detection.

Element name	Description
ReadSequence?	wf:type.workflow.electronic.readseq Definition of the electronic reading sequence. Determines which electronicdata is read in which order and when to perform the electronic checks.

Table 6.90 type.workflow.electronic - elements

6.5.31.3 XSD Definition

```
<xs:complexType name="type.workflow.electronic">
 <xs:sequence>
  <xs:element name="AccessConfiguration" minOccurs="0"
         type="wf:type.workflow.electronic.access" />
  <xs:element name="EACConfiguration" minOccurs="0"
         type="wf:type.workflow.electronic.eac" />
  <xs:element name="ChipConfiguration" minOccurs="0"
         type="wf:type.workflow.electronic.chip" />
  <xs:element name="ReadSequence" minOccurs="0"
         type="wf:type.workflow.electronic.readseq" />
 </xs:sequence>
  <xs:attribute name="condition" type="xs:IDREF" />
</xs:complexType>
```

6.5.32 type.workflow.electronic.access

List of conditional configurations of the access protocol order (BAC, PACE, PLAIN).

6.5.32.1 Attributes

None.

6.5.32.2 Elements

The type.workflow.electronic.access elements are given in Table 6.91.

Element name	Description
ProtocolOrder*	wf:type.workflow.electronic.access.order Conditional configurations of the access protocol order.

Table 6.91 type.workflow.electronic.access - elements

6.5.32.3 XSD Definition

```
<xs:complexType name="type.workflow.electronic.access">
 <xs:sequence>
  <xs:element name="ProtocolOrder" minOccurs="0" maxOccurs="unbounded"
         type="wf:type.workflow.electronic.access.order" />
 </xs:sequence>
</xs:complexType>
```

6.5.33 type.workflow.electronic.access.order

Conditional configurations of the access protocol order.

6.5.33.1 Attributes

The type.workflow.electronic.access.order attributes are given in Table 6.92.

Attribute name	Description
condition?	xs:IDREF Reference to a condition. SHALL be considered fulfilled if empty or attribute was omitted. If condition uses the special value DEFAULT, this protocol order applies if no other condition is fulfilled. If multiple DEFAULT-orders exist, the first one is used.
order	xs:string Comma separated (,) list of access protocol identifiers from the following list: • PLAIN • BAC • PACE The access protocols SHALL be executed in the specified order until the first one succeeds.

Table 6.92 type.workflow.electronic.access.order - attributes

6.5.33.2 Elements

None.

6.5.33.3 XSD Definition

```
<xs:complexType name="type.workflow.electronic.access.order">
 <xs:attribute name="condition" type="xs:IDREF" />
 <xs:attribute name="order" type="xs:string" use="required" />
</xs:complexType>
```

6.5.34 type.workflow.electronic.eac

List of conditionally executed extended access protocols after PACE.

6.5.34.1 Attributes

None.

6.5.34.2 Elements

The type.workflow.electronic.eac elements are given in Table 6.93.

Element name	Description
AuthenticationAfterPACE*	wf:type.workflow.electronic.eac.authafterpace Conditional configuration of an extended access protocol after PACE.

Table 6.93 type.workflow.electronic.eac - elements

6.5.34.3 XSD Definition

```
<xs:complexType name="type.workflow.electronic.eac">
 <xs:sequence>
  <xs:element name="AuthenticationAfterPACE" minOccurs="0"
         maxOccurs="unbounded"
         type="wf:type.workflow.electronic.eac.authafterpace" />
 </xs:sequence>
</xs:complexType>
```

6.5.35 type.workflow.electronic.eac.authafterpace

Conditional configuration of an extended access protocol (Extended Access Control (EAC)).

6.5.35.1 Attributes

The type.workflow.electronic.eac.authafterpace attributes are given in Table 6.94.

Attribute name	Description
`condition?`	`xs:IDREF` Reference to a condition. SHALL be considered fulfilled if empty or attribute was omitted.
`method`	`wf:type.workflow.electronic.eac.authafterpace.method` The extended access protocol after PACE.

Table 6.94 type.workflow.electronic.eac.authafterpace - attributes

6.5.35.2 Elements

None.

6.5.35.3 XSD Definition

```
<xs:complexType name="type.workflow.electronic.eac.authafterpace">
 <xs:attribute name="condition" type="xs:IDREF" />
 <xs:attribute name="method"
        type="wf:type.workflow.electronic.eac.authafterpace.method"
        use="required" />
</xs:complexType>
```

6.5.36 type.workflow.electronic.eac.authafterpace.method

Allowed extended access protocols after PACE. Derived from `xs:string`.

6.5.36.1 Values

The type.workflow.electronic.eac.authafterpace.method values are given in Table 6.95.

Value	Description
EAC2_AUTO	EAC2-compliant Chip Authentication (CA) and Terminal Authentication (TA) based on the version stored in `EF.CardAccess`. PACE is executed with CHAT. TA is executed with public key hash derived from PACE. Expect SAC document, if `EF.CardAccess` does not contain any TA/CA version information. After PACE, EAC1 will be performed. For documents with CAN, the document number from `EF.DG1` is used for TA static binding.
EAC2_CA1	EAC2-compliant CA v1 followed by TA v1. PACE is executed with CHAT. TA is executed with public key hash derived from PACE.
EAC2_TA2	EAC2-compliant TA v2 followed by CA v2. PACE is executed with CHAT. TA is executed public with key hash derived from PACE.
EAC1_CHAT	EAC1.11-compliant CA and TA. PACE is executed with CHAT. TA is executed with document number from MRZ
EAC1	EAC1.11-compliant CA and TA. PACE is executed without CHAT (CAR is read from EV.CVCA). TA is executed with document number from MRZ.

Table 6.95 type.workflow.electronic.eac.authafterpace.method - values

6.5.36.2 XSD Definition

```
<xs:simpleType name="type.workflow.electronic.eac.authafterpace.method">
 <xs:restriction base="xs:string">
  <xs:enumeration value="EAC2_AUTO" />
  <xs:enumeration value="EAC2_CA1" />
  <xs:enumeration value="EAC2_TA2" />
  <xs:enumeration value="EAC1_CHAT" />
  <xs:enumeration value="EAC1" />
 </xs:restriction>
</xs:simpleType>
```

6.5.37 type.workflow.electronic.chip

List of conditional timeout configurations for chip detection.

6.5.37.1 Attributes

None.

6.5.37.2 Elements

The type.workflow.electronic.chip elements are given in ▸Table 6.96.

Element name	Description
`WaitForChip*`	`wf:type.workflow.electronic.chip.waitforchip` Configuration of a conditional chip detection timeout.

Table 6.96 type.workflow.electronic.chip - elements

6.5.37.3 XSD Definition

```
<xs:complexType name="type.workflow.electronic.chip">
 <xs:sequence>
  <xs:element name="WaitForChip" minOccurs="0" maxOccurs="unbounded"
        type="wf:type.workflow.electronic.chip.waitforchip" />
 </xs:sequence>
</xs:complexType>
```

6.5.38 type.workflow.electronic.chip.waitforchip

Conditional chip detection timeout. Before starting the electronic reading process, the server SHALL wait for the specified time period, unless the chip is detected successfully earlier. If omitted, the server MAY start reading the chip immediately.

6.5.38.1 Attributes

The type.workflow.electronic.chip.waitforchip elements are given in ▸Table 6.97.

Attribute name	Description
`condition?`	`xs:IDREF` Reference to a condition. SHALL be considered fulfilled if empty or attribute was omitted.
`timeInMs`	`xs:int` Chip detection timeout in milliseconds.
`intervalInMs?`	`xs:int` Default: `500` Polling intervall in milliseconds. During `timeInMs`, the server SHOULD wait at least `intervalInMs` milliseconds between attempts to detect the chip.

Table 6.97 type.workflow.electronic.chip.waitforchip - attributes

6.5.38.2 Elements

None.

6.5.38.3 XSD Definition

```
<xs:complexType name="type.workflow.electronic.chip.waitforchip">
 <xs:attribute name="condition" type="xs:IDREF" />
 <xs:attribute name="timeInMs" type="xs:int" use="required" />
 <xs:attribute name="intervalInMs" type="xs:int" default="500" />
</xs:complexType>
```

6.5.39 type.workflow.electronic.readseq

List of read and check operations to perform. The execution order corresponds to the order of the definition.

6.5.39.1 Attributes

None.

6.5.39.2 Elements

The elements MAY appear in any order.

The type.workflow.electronic.readseq elements are given in Table 6.98.

Element name	Description
Datagroup*	wf:type.workflow.electronic.readseq.datagroup Reads an electronic data group.
ElectronicCheck*	wf:type.workflow.readseq.check Performs an electronic document check according to [BSI TR-03135].
DefectInfo*	wf:type.workflow.electronic.readseq.defectinfo Reads known defects for the document.
ChipDetection*	wf:type.workflow.electronic.readseq.chipdetection Checks for the presence of an Radio-Frequency Identification (RFID) chip in the reading field. The result can be acquired by a text feedback element.
ElementaryFile*	wf:type.workflow.electronic.readseq.elementaryfile Configuration of conditional reading of elementary files.

Table 6.98 type.workflow.electronic.readseq - elements.

6.5.39.3 XSD Definition

```
<xs:complexType name="type.workflow.electronic.readseq">
 <xs:choice minOccurs="0" maxOccurs="unbounded">
  <xs:element name="Datagroup"
         type="wf:type.workflow.electronic.readseq.datagroup" />
  <xs:element name="ElectronicCheck"
         type="wf:type.workflow.readseq.check" />
  <xs:element name="DefectInfo"
         type="wf:type.workflow.electronic.readseq.defectinfo" />
  <xs:element name="ChipDetection"
         type="wf:type.workflow.electronic.readseq.chipdetection" />
  <xs:element name="ElementaryFile"
         type="wf:type.workflow.electronic.readseq.elementaryfile"/>
 </xs:choice>
</xs:complexType>
```

6.5.40 type.workflow.electronic.readseq.datagroup

Configuration of conditional reading of data groups.

6.5.40.1 Attributes

The type.workflow.electronic.readseq.datagroup attributes are given in Table 6.99.

Attribute name	Description
`id`	`xs:ID` User-defined unique ID. Serves as reference.
`application`	`wf:type.workflow.electronic.readseq.datagroup.application` The document application from which the data group is to be read.
`number`	`xs:int` The number of the data group to read.
`condition?`	`xs:IDREF` Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or omitted.
`required?`	`xs:boolean` Default: `false` Specifies whether reading this data group is imperative for the respective application. If not, failure to read the data group SHALL NOT negatively affect corresponding data integrity checks.

Table 6.99 type.workflow.electronic.readseq.datagroup - attributes

6.5.40.2 Elements

None.

6.5.40.3 XSD Definition

```
<xs:complexType name="type.workflow.electronic.readseq.datagroup">
 <xs:attribute name="id" type="xs:ID" use="required" />
 <xs:attribute name="application"
        type="wf:type.workflow.electronic.readseq.datagroup.application"
        use="required" />
 <xs:attribute name="number" type="xs:int" use="required" />
 <xs:attribute name="condition" type="xs:IDREF" />
 <xs:attribute name="required" type="xs:boolean" default="false" />
</xs:complexType>
```

6.5.41 type.workflow.electronic.readseq.datagroup.application

Selects the document application for data group reading.

6.5.41.1 Values

The type.workflow.electronic.readseq.datagroup.application values are given in Table 6.100.

Value	Description
ICAO-Chip	International Civil Aviation Organization (ICAO) ePassport data groups directly from the chip.
ICAO-DTC	ICAO ePassport data groups from DTC. The server SHALL not use a fallback to chip data.
ICAO-DTC-Chip-Fallback	ICAO ePassport data groups from DTC if available. The server SHALL use a fallback to chip data.
eID	Electronic Identity Document (eID) data groups

Table 6.100 type.workflow.electronic.readseq.datagroup.application - values

6.5.41.2 XSD Definition

```
<xs:simpleType name="type.workflow.electronic.readseq.datagroup.application">
 <xs:restriction base="xs:string">
 <xs:enumeration value="ICAO-Chip"/>
 <xs:enumeration value="ICAO-DTC"/>
 <xs:enumeration value="ICAO-DTC-Chip-Fallback"/>
 <xs:enumeration value="eID"/>
 </xs:restriction>
</xs:simpleType>
```

6.5.42 type.workflow.electronic.readseq.defectinfo

Conditional reading of the known defects regarding the current document.

6.5.42.1 Attributes

The type.workflow.electronic.readseq.defectinfo attributes are given in Table 6.101.

Attribute name	Description
id	xs:ID User-defined unique ID. Serves as reference.
condition?	xs:IDREF Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.101 type.workflow.electronic.readseq.defectinfo - attributes

6.5.42.2 Elements

None.

6.5.42.3 XSD Definition

```
<xs:complexType name="type.workflow.electronic.readseq.defectinfo">
 <xs:attribute name="id" type="xs:ID" use="required" />
 <xs:attribute name="condition" type="xs:IDREF" />
</xs:complexType>
```

6.5.43 type.workflow.electronic.readseq.chipdetection

Checking for the presence of an RFID chip in the reading field.

6.5.43.1 Attributes

The type.workflow.electronic.readseq.chipdetection attributes are given in Table 6.102.

Attribute name	Description
id	xs:ID User-defined unique ID. Serves as reference.
condition?	xs:IDREF Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.102 type.workflow.electronic.readseq.chipdetection - attributes

6.5.43.2 Elements

None.

6.5.43.3 XSD Definition

```
<xs:complexType name="type.workflow.electronic.readseq.chipdetection">
 <xs:attribute name="id" type="xs:ID" use="required" />
 <xs:attribute name="condition" type="xs:IDREF" />
</xs:complexType>
```

6.5.44 type.workflow.electronic.readseq.elementaryfile

Configuration of conditional reading of elementary files.

6.5.44.1 Attributes

The type.workflow.electronic.readseq.elementaryfile attributes are given in Table 6.103.

Attribute name	Description
id	xs:ID User-defined unique ID. Serves as reference.
name	wf:type.workflow.electronic.readseq.elementaryfile.name The elementary file to be read.
condition?	xs:IDREF Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or omitted.
required?	xs:boolean Default: false Specifies whether reading this elementary file is imperative for the respective application.

Table 6.103 type.workflow.electronic.readseq.elementaryfile - attributes

6.5.44.2 Elements

None.

6.5.44.3 XSD Definition

```
<xs:complexType name="type.workflow.electronic.readseq.elementaryfile">
 <xs:attribute name="id" type="xs:ID" use="required" />
 <xs:attribute name="name"
              type="wf:type.workflow.electronic.readseq.elementaryfile.name"
              use="required" />
 <xs:attribute name="condition" type="xs:IDREF" />
 <xs:attribute name="required" type="xs:boolean" default="false" />
</xs:complexType>
```

6.5.45 type.workflow.electronic.readseq.elementaryfile.name

Selects the elementary file for reading by its name.

6.5.45.1 Values

The type.workflow.electronic.readseq.elementaryfile.name values are given in Table 6.104.

Value	Description
ef_sod	EF.SoD
ef_card_security	EF.CardSecurity
ef_chip_security	EF.ChipSecurity
ef_com	EF.COM

Value	Description
ef_cvca	EF.CVCA
ef_card_access	EF.CardAccess
ef_atr_info	EF.ATR_INFO

Table 6.104 type.workflow.electronic.readseq.elementaryfile.name - values

6.5.45.2 XSD Definition

```
<xs:simpleType name="type.workflow.electronic.readseq.elementaryfile.name">
 <xs:restriction base="xs:string">
  <xs:enumeration value="ef_sod" />
  <xs:enumeration value="ef_card_security" />
  <xs:enumeration value="ef_chip_security" />
  <xs:enumeration value="ef_com" />
  <xs:enumeration value="ef_cvca" />
  <xs:enumeration value="ef_card_access" />
  <xs:enumeration value="ef_atr_info" />
 </xs:restriction>
</xs:simpleType>
```

6.5.46 type.workflow.optical

Configuration of the optical reading and check process with respect to document validity definition and the optical reading sequence.

6.5.46.1 Attributes

The type.workflow.optical attributes are given in Table 6.105.

Attribute name	Description
condition?	xs:IDREF Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.105 type.workflow.optical - attributes

6.5.46.2 Elements

The type.workflow.optical elements are given in Table 6.106.

Element name	Description
DocValidityConfiguration?	wf:type.workflow.optical.validity Document validity configuration. If omitted, a configuration with a default validity period of 10 years is used.
ReadSequence?	wf:type.workflow.optical.readseq Definition of the optical reading sequence. Determines which optical data is read in which order and when to perform the optical checks.

Table 6.106 type.workflow.optical - elements

6.5.46.3 XSD Definition

```
<xs:complexType name="type.workflow.optical">
 <xs:sequence>
  <xs:element name="DocValidityConfiguration" minOccurs="0"
        type="wf:type.workflow.optical.validity" />
  <xs:element name="ReadSequence" minOccurs="0"
        type="wf:type.workflow.optical.readseq" />
 </xs:sequence>
  <xs:attribute name="condition" type="xs:IDREF" />
</xs:complexType>
```

6.5.47 type.workflow.optical.validity

List of document validity configurations.

6.5.47.1 Attributes

None.

6.5.47.2 Elements

The type.workflow.optical.validity elements are given in ▸Table 6.107.

Element name	Description
DocValidity*	wf:type.workflow.optical.validity.element Conditional document validity configuration.

Table 6.107 type.workflow.optical.validity - elements

6.5.47.3 XSD Definition

```
<xs:complexType name="type.workflow.optical.validity">
 <xs:sequence>
  <xs:element name="DocValidity"
         type="wf:type.workflow.optical.validity.element"
         minOccurs="0" maxOccurs="unbounded" />
 </xs:sequence>
</xs:complexType>
```

6.5.48 type.workflow.optical.validity.element

Conditional configuration of the document validity.

6.5.48.1 Attributes

The type.workflow.optical.validity.element attributes are given in ▸Table 6.108.

Attribute name	Description
condition	xs:IDREF Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty. If condition uses the special value DEFAULT, this configuration applies if no other condition is fulfilled. If multiple DEFAULT-configurations exist, the first one is used.
validityInYears	xs:unsignedInt Document validity in years.
toleranceInDays?	xs:int Default: 0 Tolerance in days after document expiration for which the document is still considered valid. This value is used in the optical document check. This value can also be negative to indicate that a document is already considered as invalid before its regular expiration.

Table 6.108 type.workflow.optical.validity.element - attributes

6.5.48.2 Elements

None.

6.5.48.3 XSD Definition

```
<xs:complexType name="type.workflow.optical.validity.element">
 <xs:attribute name="condition" type="xs:IDREF" use="required" />
 <xs:attribute name="validityInYears" type="xs:unsignedInt" use="required" />
```

```
  <xs:attribute name="toleranceInDays" type="xs:int" default="0" />
</xs:complexType>
```

6.5.49 type.workflow.optical.readseq

List of optical read and check operations to perform. The execution order corresponds to the order of the definition.

6.5.49.1 Attributes

None.

6.5.49.2 Elements

The elements MAY appear in any order.

The type.workflow.optical.readseq elements are given in Table 6.109.

Element name	Description
Text*	wf:type.workflow.optical.readseq.element Reads a text field.
Binary*	wf:type.workflow.optical.readseq.element Reads a binary field.
Image*	wf:type.workflow.optical.readseq.element Reads an image field.
OpticalCheck*	wf:type.workflow.readseq.check Performs optical document checks according to [BSI TR-03135].

Table 6.109 type.workflow.optical.readseq - elements

6.5.49.3 XSD Definition

```
<xs:complexType name="type.workflow.optical.readseq">
 <xs:choice minOccurs="0" maxOccurs="unbounded">
  <xs:element name="Text" type="wf:type.workflow.optical.readseq.element">
  <xs:element name="Binary" type="wf:type.workflow.optical.readseq.element">
  <xs:element name="Image" type="wf:type.workflow.optical.readseq.element">
  <xs:element name="OpticalCheck"
              type="wf:type.workflow.readseq.check">
 </xs:choice>
</xs:complexType>
```

6.5.50 type.workflow.optical.readseq.element

Conditionally specifies which optical data is to be read.

6.5.50.1 Attributes

The type.workflow.optical.readseq.element attributes are given in Table 6.110.

Attributes name	Description
id	xs:ID User-defined unique ID. Serves as reference.
field	wf:type.workflow.optical.readseq.element.field Reference to the field to read.
condition?	xs:IDREF Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.110 type.workflow.optical.readseq.element - attributes

6.5.50.2 Elements

None.

6.5.50.3 XSD Definition

```
<xs:complexType name="type.workflow.optical.readseq.element">
 <xs:attribute name="id" type="xs:ID" use="required">
 <xs:attribute name="field" type="wf:type.workflow.optical.readseq.element.field" use="required">
 <xs:attribute name="condition" type="xs:IDREF" >
</xs:complexType>
```

6.5.51 type.workflow.optical.readseq.element.field

Union for predefined field names and custom filed names.

6.5.51.1 Attributes

None.

6.5.51.2 Elements

None.

6.5.51.3 XSD Definition

```
<xs:simpleType name="type.workflow.optical.readseq.element.field">
 <xs:union memberTypes="wf:type.workflow.optical.readseq.element.field.enum
 wf:type.workflow.optical.readseq.element.field.custom"/>
</xs:simpleType>
```

6.5.52 type.workflow.optical.readseq.element.field.enum

List of predefined field names.

Note that OCR may be involved to get the field data. The server MAY return the text feedback based on such fields as string with line breaking or ending characters if not specified differently. The string characters SHALL be as identified by the OCR except for potentially unidentified characters. The server SHOULD NOT omit unidentified characters and mark them if possible. It is RECOMMENDED to use question marks for unidentified characters.

Also note that OCR within the VIZ may not be available for every character set. The server MAY mark feedback based on fields which require OCR within the VIZ as not supported.

Further note that for the predefined fieldnames the frontside of the document is considered to be the document page with the primary facial image.

6.5.52.1 Values

The type.workflow.optical.readseq.element.field.enum values are given in Table 6.111.

Value	Description
`MRZ`	Reference to the optical MRZ as found on the document. The server SHALL return text feedback based on this field as string without line breaking or ending characters.
`MRZ-LINE-1`	Reference to the first line of the optical MRZ as found on the document. The server SHALL return text feedback based on this field as string without line breaking or ending characters.
`MRZ-LINE-2`	Reference to the second line of the optical MRZ as found on the document. The server SHALL return text feedback based on this field as string without line breaking or ending characters.
`MRZ-LINE-3`	Reference to the third line of the optical MRZ as found on the document. The server SHALL return text feedback based on this field as string without line breaking or ending characters.

Value	Description
MRZ-SURNAME	Reference to the surname from the optical MRZ as found on the document.
MRZ-GIVEN-NAMES	Reference to the given names from the optical MRZ as found on the document.
MRZ-DATE-OF-BIRTH	Reference to the date of birth from the optical MRZ as found on the document.
MRZ-DATE-OF-EXPIRY	Reference to the date of expiry from the optical MRZ as found on the document.
MRZ-DOCUMENTNUMBER	Reference to the document number from the optical MRZ as found on the document.
MRZ-DOCUMENTTYPE	Reference to the document type from the optical MRZ as found on the document.
MRZ-SEX	Reference to the sex from the optical MRZ as found on the document.
MRZ-ISSUER	Reference to the issuer from the optical MRZ as found on the document.
MRZ-NATIONALITY	Reference to the nationality from the optical MRZ as found on the document.
MRZ-OPTIONAL-DATA	Reference to the optional data from the optical MRZ as found on the document.
MRZ-CHECKDIGIT-COMPOSITE	Reference to the composite check digit from the optical MRZ as found on the document.
MRZ-CHECKDIGIT-DATE-OF-BIRTH	Reference to the date of birth check digit from the optical MRZ as found on the document.
MRZ-CHECKDIGIT-DATE-OF-EXPIRY	Reference to the date of expiry check not ommit digit from the optical MRZ as found on the document.
MRZ-CHECKDIGIT-DOCUMENTNUMBER	Reference to the document number check digit from the optical MRZ as found on the document.
MRZ-CHECKDIGIT-OPTIONAL-DATA	Reference to the optional data check digit from the optical MRZ as found on the document.
VIZ-SURNAME	Reference to the surname from the VIZ as found on the document.
VIZ-GIVEN-NAMES	Reference to the given names from the VIZ as found on the document.
VIZ-DATE-OF-BIRTH	Reference to the date of birth from the VIZ as found on the document.
VIZ-DATE-OF-EXPIRY	Reference to the date of expiry from the VIZ as found on the document.
VIZ-DATE-OF-ISSUE	Reference to the issuing date from the VIZ as found on the document.
VIZ-DOCUMENTNUMBER	Reference to the document number from the VIZ as found on the document.
VIZ-DOCUMENTTYPE	Reference to the document type from the VIZ as found on the document.
VIZ-SEX	Reference to the sex from the VIZ as found on the document.
VIZ-ISSUER	Reference to the issuer from the VIZ as found on the document.
VIZ-NATIONALITY	Reference to the nationality from the VIZ as found on the document.
VIZ-PERSONALNUMER	Reference to the personal number from the VIZ as found on the document.
VIZ-PLACE-OF-BIRTH	Reference to the place of birth from the VIZ as found on the document.
VIZ-PLACE-OF-ISSUE	Reference to the place of issue from the VIZ as found on the document.
VIZ-AUTHORITY	Reference to the authority from the VIZ as found on the document.
CAN	Reference to the CAN as found on the document.
IMG-FACE-PRIMARY-CROPPED	Reference to a cropped version of the primary face image from the VIZ.
IMG-IR-FRONT-FULL	Reference to an image of the full scanning area in infrared light with the frontside of the document.

Value	Description
IMG-VIS-FRONT-FULL	Reference to an image of the full scanning area in visible light with the frontside of the document.
IMG-UV-FRONT-FULL	Reference to an image of the full scanning area in ultraviolet light with the frontside of the document.
IMG-IR-FRONT-CROPPED	Reference to an image of the cropped document image in infrared light with the frontside of the document.
IMG-VIS-FRONT-CROPPED	Reference to an image of the cropped document image in visible light with the frontside of the document.
IMG-UV-FRONT-CROPPED	Reference to an image of the cropped document image in ultraviolet light with the frontside of the document.
IMG-IR-FRONT-REFERENCE	Reference to an example image of the frontside of the document in infrared light.
IMG-VIS-FRONT-REFERENCE	Reference to an example image of the frontside of the document in visible light.
IMG-UV-FRONT-REFERENCE	Reference to an example image of the frontside of the document in ultraviolet light.
IMG-IR-BACK-FULL	Reference to an image of the full scanning area in infrared light with the backside of the document.
IMG-VIS-BACK-FULL	Reference to an image of the full scanning area in visible light with the backside of the document.
IMG-UV-BACK-FULL	Reference to an image of the full scanning area in ultraviolet light with the backside of the document.
IMG-IR-BACK-CROPPED	Reference to an image of the cropped document image in infrared light with the backside of the document.
IMG-VIS-BACK-CROPPED	Reference to an image of the cropped document image in visible light with the backside of the document.
IMG-UV-BACK-CROPPED	Reference to an image of the cropped document image in ultraviolet light with the backside of the document.
IMG-IR-BACK-REFERENCE	Reference to an example image of the backside of the document in infrared light.
IMG-VIS-BACK-REFERENCE	Reference to an example image of the backside of the document in visible light.
IMG-UV-BACK-REFERENCE	Reference to an example image of the backside of the document in ultraviolet light.
XML-BARCODE-POSITIONS-FRONT	Reference to an XML data structure which contains all barcode position on the frontside of the document.
XML-BARCODE-POSITIONS-BACK	Reference to an XML data structure which contains all barcode position on the backside of the document.
XML-BARCODE-POSITIONS-ALL	Reference to an XML data structure which contains all barcode position on all scanned sides of the document.

Table 6.111 type.workflow.optical.readseq.element.field.enum - values

6.5.52.2 XSD Definition

```
<xs:simpleType name="type.workflow.optical.readseq.element.field.enum">
 <xs:restriction base="xs:string">
  <xs:enumeration value="MRZ"/>
  <xs:enumeration value="MRZ-LINE-1"/>
  <xs:enumeration value="MRZ-LINE-2"/>
  <xs:enumeration value="MRZ-LINE-3"/>
  <xs:enumeration value="MRZ-SURNAME"/>
  <xs:enumeration value="MRZ-GIVEN-NAMES"/>
  <xs:enumeration value="MRZ-DATE-OF-BIRTH"/>
  <xs:enumeration value="MRZ-DATE-OF-EXPIRY"/>
```

```
   <xs:enumeration value="MRZ-DOCUMENTNUMBER"/>
   <xs:enumeration value="MRZ-DOCUMENTTYPE"/>
   <xs:enumeration value="MRZ-SEX"/>
   <xs:enumeration value="MRZ-ISSUER"/>
   <xs:enumeration value="MRZ-NATIONALITY"/>
   <xs:enumeration value="MRZ-OPTIONAL-DATA"/>
   <xs:enumeration value="MRZ-CHECKDIGIT-COMPOSITE"/>
   <xs:enumeration value="MRZ-CHECKDIGIT-DATE-OF-BIRTH"/>
   <xs:enumeration value="MRZ-CHECKDIGIT-DATE-OF-EXPIRY"/>
   <xs:enumeration value="MRZ-CHECKDIGIT-DOCUMENTNUMBER"/>
   <xs:enumeration value="MRZ-CHECKDIGIT-OPTIONAL-DATA"/>
   <xs:enumeration value="VIZ-SURNAME"/>
   <xs:enumeration value="VIZ-GIVEN-NAMES"/>
   <xs:enumeration value="VIZ-DATE-OF-BIRTH"/>
   <xs:enumeration value="VIZ-DATE-OF-EXPIRY"/>
   <xs:enumeration value="VIZ-DATE-OF-ISSUE"/>
   <xs:enumeration value="VIZ-DOCUMENTNUMBER"/>
   <xs:enumeration value="VIZ-DOCUMENTTYPE"/>
   <xs:enumeration value="VIZ-SEX"/>
   <xs:enumeration value="VIZ-ISSUER"/>
   <xs:enumeration value="VIZ-NATIONALITY"/>
   <xs:enumeration value="VIZ-PERSONALNUMBER"/>
   <xs:enumeration value="VIZ-PLACE-OF-BIRTH"/>
   <xs:enumeration value="VIZ-PLACE-OF-ISSUE"/>
   <xs:enumeration value="VIZ-AUTHORITY"/>
   <xs:enumeration value="CAN"/>
   <xs:enumeration value="IMG-FACE-PRIMARY-CROPPED"/>
   <xs:enumeration value="IMG-IR-FRONT-FULL"/>
   <xs:enumeration value="IMG-VIS-FRONT-FULL"/>
   <xs:enumeration value="IMG-UV-FRONT-FULL"/>
   <xs:enumeration value="IMG-IR-FRONT-CROPPED"/>
   <xs:enumeration value="IMG-VIS-FRONT-CROPPED"/>
   <xs:enumeration value="IMG-UV-FRONT-CROPPED"/>
   <xs:enumeration value="IMG-IR-FRONT-REFERENCE"/>
   <xs:enumeration value="IMG-VIS-FRONT-REFERENCE"/>
   <xs:enumeration value="IMG-UV-FRONT-REFERENCE"/>
   <xs:enumeration value="IMG-IR-BACK-FULL"/>
   <xs:enumeration value="IMG-VIS-BACK-FULL"/>
   <xs:enumeration value="IMG-UV-BACK-FULL"/>
   <xs:enumeration value="IMG-IR-BACK-CROPPED"/>
   <xs:enumeration value="IMG-VIS-BACK-CROPPED"/>
   <xs:enumeration value="IMG-UV-BACK-CROPPED"/>
   <xs:enumeration value="IMG-IR-BACK-REFERENCE"/>
   <xs:enumeration value="IMG-VIS-BACK-REFERENCE"/>
   <xs:enumeration value="IMG-UV-BACK-REFERENCE"/>
   <xs:enumeration value="XML-BARCODE-POSITIONS-FRONT"/>
   <xs:enumeration value="XML-BARCODE-POSITIONS-BACK"/>
   <xs:enumeration value="XML-BARCODE-POSITIONS-ALL"/>
  </xs:restriction>
</xs:simpleType>
```

6.5.53 type.workflow.optical.readseq.element.field.custom

Allows the usage of custom filed names.

6.5.53.1 Attributes

None.

6.5.53.2 Elements

None.

6.5.53.3 XSD Definition

```
<xs:simpleType name="type.workflow.optical.readseq.element.field.custom">
 <xs:restriction base="xs:string"/>
</xs:simpleType>
```

6.5.54 type.workflow.combined

Configuration of the combined reading and check process.

6.5.54.1 Attributes

The type.workflow.combined attributes are given in ▸Table 6.112.

Attributes name	Description
condition?	xs:IDREF Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.112 type.workflow.combined - attributes

6.5.54.2 Elements

The type.workflow.combined elements are given in ▸Table 6.113.

Element name	Description
ReadSequence?	wf:type.workflow.combined.readseq Definition of the combined reading sequence. Determines which combined data is read in which order and when to perform the combined checks.

Table 6.113 type.workflow.combined - elements

6.5.54.3 XSD Definition

```
<xs:complexType name="type.workflow.combined">
 <xs:sequence>
  <xs:element name="ReadSequence" type="wf:type.workflow.combined.readseq" minOccurs="0" />
 </xs:sequence>
 <xs:attribute name="condition" type="xs:IDREF" />
</xs:complexType>
```

6.5.55 type.workflow.combined.readseq

List of combined check operations to perform. The execution order corresponds to the order of the definition.

6.5.55.1 Attributes

None.

6.5.55.2 Elements

The elements MAY appear in any order.

The type.workflow.combined.readseq elements are given in ▸Table 6.114.

Element name	Description
CombinedCheck*	wf:type.workflow.readseq.check A single-document combined check.

Table 6.114 type.workflow.combined.readseq - elements

6.5.55.3 XSD Definition

```
<xs:complexType name="type.workflow.combined.readseq">
 <xs:choice minOccurs="0" maxOccurs="unbounded">
  <xs:element name="CombinedCheck" type="wf:type.workflow.readseq.check" />
 </xs:choice>
</xs:complexType>
```

6.5.56 type.workflow.cross

Configuration of the cross-document reading and check process.

6.5.56.1 Attributes

The type.workflow.cross attributes are given in ▸Table 6.115.

Attributes name	Description
`condition?`	`xs:IDREF` Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.115 type.workflow.cross - attributes

6.5.56.2 Elements

The type.workflow.cross elements are given in ▸Table 6.116.

Element name	Description
`ReadSequence?`	`wf:type.workflow.cross.readseq` Definition of the cross reading sequence. Determines which combined data is read in which order and when to perform the combined checks.

Table 6.116 type.workflow.cross- elements

6.5.56.3 XSD Definition

```
<xs:complexType name="type.workflow.cross">
 <xs:sequence>
  <xs:element name="ReadSequence" type="wf:type.workflow.cross.readseq" minOccurs="0" />
 </xs:sequence>
 <xs:attribute name="condition" type="xs:IDREF" />
</xs:complexType>
```

6.5.57 type.workflow.cross.readseq

List of document-cross check operations to perform. The execution order corresponds to the order of the definition.

6.5.57.1 Attributes

None.

6.5.57.2 Elements

The elements MAY appear in any order.

The type.workflow.cross.readseq elements are given in ▸Table 6.117.

Element name	Description
`CrossCheck*`	`wf:type.workflow.readseq.check` A cross-document check.

Table 6.117 type.workflow.cross.readseq - elements

6.5.57.3 XSD Definition

```
<xs:complexType name="type.workflow.cross.readseq">
 <xs:choice minOccurs="0" maxOccurs="unbounded">
  <xs:element name="CrossCheck" type="wf:type.workflow.readseq.check" />
 </xs:choice>
</xs:complexType>
```

6.5.58 type.workflow.readseq.check

Conditional execution of a particular document check according to [BSI TR-03135].

6.5.58.1 Attributes

The type.workflow.readseq.check attributes are given in Table 6.118.

Attribute name	Description
`id`	`xs:ID` User-defined unique ID. Serves as reference.
`condition?`	`xs:IDREF` Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.118 type.workflow.readseq.check - attributes

6.5.58.2 Elements

None.

6.5.58.3 XSD Definition

```
<xs:complexType name="type.workflow.readseq.check">
 <xs:attribute name="id" type="xs:ID" use="required" />
 <xs:attribute name="condition" type="xs:IDREF" />
</xs:complexType>
```

6.5.59 type.workflow.readseq.defect

Conditional execution of a particular document defect according to [BSI TR-03135].

6.5.59.1 Attributes

The type.workflow.readseq.defect attributes are given in Table 6.119.

Attribute name	Description
`id`	`xs:ID` User-defined unique ID. Serves as reference.
`condition?`	`xs:IDREF` Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.119 type.workflow.readseq.defect - attributes

6.5.59.2 Elements

None.

6.5.59.3 XSD Definition

```
  <xs:complexType name="type.workflow.readseq.defect"/>
  <xs:attribute name="id" type="xs:ID" use="required"/>
  <xs:attribute name="condition" type="xs:IDREF"/>
 </xs:complexType>
```

6.5.60 type.workflow.seal

Configuration of the seal reading and check process with respect to document validity definition and the seal reading sequence.

6.5.60.1 Attributes

The type.workflow.seal are given in ▸Table 6.120.

Attribute name	Description
`condition?`	`xs:IDREF` Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.120 type.workflow.seal - attributes

6.5.60.2 Elements

The type.workflow.seal.readseq elements are given in ▸Table 6.121.

Element name	Description
`ReadSequence?`	`wf:type.workflow.seal.readseq` Definition of the seal reading sequence. Determines which seal data is read in which order and when to perform the seal checks.

Table 6.121 type.workflow.seal - elements

6.5.60.3 XSD Definition

```
<xs:complexType name="type.workflow.seal">
  <xs:sequence>
   <xs:element name="ReadSequence" minOccurs="0" type="wf:type.workflow.seal.readseq"/>
  </xs:sequence>
  <xs:attribute name="condition" type="xs:IDREF"/>
 </xs:complexType>
```

6.5.61 type.workflow.seal.readseq

List of read and check operations to perform. The execution order corresponds to the order of the definition.

6.5.61.1 Attributes

None.

6.5.61.2 Elements

The elements MAY appear in any order.

The type.workflow.seal.readseq elements are given in ▸Table 6.122.

Element name	Description
`Data*`	`wf:type.workflow.seal.readseq.data` Reads the seal data.
`Check*`	`wf:type.workflow.readseq.check` Performs a digital seal document check according to [BSI TR-03135].
`Defect`	`wf:type.workflow.readseq.defect` Reads known defects for the document.

Table 6.122 type.workflow.electronic.readseq - elements.

6.5.61.3 XSD Definition

```
<xs:complexType name="type.workflow.seal.readseq">
  <xs:choice minOccurs="0" maxOccurs="unbounded">
   <xs:element name="Data" type="wf:type.workflow.seal.readseq.data"/>
   <xs:element name="Defect" type="wf:type.workflow.readseq.defect"/>
   <xs:element name="Check" type="wf:type.workflow.readseq.check"/>
```

```
  </xs:choice>
 </xs:complexType>
```

6.5.62 type.workflow.seal.readseq.data

Configuration of conditional reading of data.

6.5.62.1 Attributes

The type.workflow.seal.readseq.data attributes are given in ▸Table 6.123.

Attribute name	Description
id	xs:ID User-defined unique ID. Serves as reference.
profile	wf:type.workflow.seal.readseq.data.profile The document profile from which the data is to be read.
condition?	xs:IDREF Reference to a condition. Controls whether this configuration is applicable. SHALL be considered fulfilled if empty or omitted.
required?	xs:boolean Default: false Specifies whether reading this data group is imperative for the respective application. If not, failure to read the data group SHALL NOT negatively affect corresponding data integrity checks.

Table 6.123 type.workflow.seal.readseq.data - attributes

6.5.62.2 Elements

None.

6.5.62.3 XSD Definition

```
<xs:complexType name="type.workflow.seal.readseq.data">
  <xs:attribute name="id" type="xs:ID" use="required"/>
  <xs:attribute name="profile" type="wf:type.workflow.seal.readseq.data.profile" use="required"/>
  <xs:attribute name="condition" type="xs:IDREF"/>
  <xs:attribute name="required" type="xs:boolean"/>
 </xs:complexType>
```

6.5.63 type.workflow.feedback

List of feedback elements that provide data from the elements of the electronic, optical, seal and combined reading sequences. The feedback elements are the only data that are delivered to the application. Feedback is provided in the feedback loop.

6.5.63.1 Attributes

None.

6.5.63.2 Elements

The elements MAY appear in any order.

The type.workflow.feedback elements are given in ▸Table 6.124.

Element name	Description
Binary*	wf:type.workflow.feedback.binary Returns binary data to the application.

Element name	Description
Text*	wf:type.workflow.feedback.text Returns text data to the application.
Image*	wf:type.workflow.feedback.image Returns image data to the application.
XML*	wf:type.workflow.feedback.xml Returns XML data to the application.

Table 6.124 type.workflow.feedback - elements

6.5.63.3 XSD Definition

```
<xs:complexType name="type.workflow.feedback">
 <xs:choice minOccurs="0" maxOccurs="unbounded">
  <xs:element name="Binary" type="wf:type.workflow.feedback.binary" />
  <xs:element name="Text" type="wf:type.workflow.feedback.text" />
  <xs:element name="Image" type="wf:type.workflow.feedback.image" />
  <xs:element name="XML" type="wf:type.workflow.feedback.xml" />
 </xs:choice>
</xs:complexType>
```

6.5.64 type.workflow.feedback.binary

Requests binary data feedback. Binary data is always returned in the binaryFeedback element. It is only available for electronic data groups.

6.5.64.1 Attributes

The type.workflow.feedback.binary attributes are given in Table 6.125.

Attribute name	Description
id	xs:ID User-defined unique ID. Serves as reference.
from	xs:IDREF Reference to a data element. Referenced element SHALL be of type type.workflow.electronic.readseq.datagroup.
condition?	xs:IDREF References a condition. Controls whether the feedback element is delivered. SHALL be considered fulfilled if empty or attribute was omitted.
onRequestOnly?	xs:boolean Default: false Controls whether the feedback element is available as part of the general feedback loop via getWorkflowFeedback or only upon explicit request via getWorkflowFeedbackById.

Table 6.125 type.workflow.feedback.binary - attributes

6.5.64.2 Elements

None.

6.5.64.3 XSD Definition

```
<xs:complexType name="type.workflow.feedback.binary">
 <xs:attribute name="id" type="xs:ID" use="required" />
 <xs:attribute name="from" type="xs:IDREF" use="required" />
 <xs:attribute name="condition" type="xs:IDREF" />
<xs:attribute name="onRequestOnly" type="xs:boolean" default="false" />
```

```
</xs:complexType>
```

6.5.65 type.workflow.feedback.text

Requests text data feedback. Text data is always returned in the `stringFeedback` element.

6.5.65.1 Attributes

The type.workflow.feedback.text attributes are given in Table 6.126.

Attribute name	Description
`id`	`xs:ID` User-defined unique ID. Serves as reference.
`from`	`xs:IDREF` Reference to a data element. SHALL reference any of the following: • ICAO data group 1 • eID data group 19 • eID data group 20 • any optical text field
`condition?`	`xs:IDREF` References a condition. Controls whether the feedback element is delivered. SHALL be considered fulfilled if empty or attribute was omitted.
`onRequestOnly?`	`xs:boolean` Default: `false` Controls whether the feedback element is available as part of the general feedback loop via `getWorkflowFeedback` or only upon explicit request via `getWorkflowFeedbackById`.

Table 6.126 type.workflow.feedback.text - attributes

6.5.65.2 Elements

None.

6.5.65.3 XSD Definition

```
<xs:complexType name="type.workflow.feedback.text">
 <xs:attribute name="id" type="xs:ID" use="required" />
 <xs:attribute name="from" type="xs:IDREF" use="required" />
 <xs:attribute name="condition" type="xs:IDREF" />
 <xs:attribute name="onRequestOnly" type="wf:type.boolean" default="false" />
</xs:complexType>
```

6.5.66 type.workflow.feedback.image

Requests image data feedback. Image data is always returned in the `binaryFeedback` field.

6.5.66.1 Attributes

The type.workflow.feedback.image attributes are given in Table 6.127.

Attribute name	Description
`id`	`xs:ID` Unique ID. Serves as reference.

Attribute name	Description
`from`	`xs:IDREF` Reference to a data element. SHALL reference any of the following: • ICAO data group 2 • ICAO data group 3 • ICAO data group 4 • ICAO data group 7 • any optical image field
`format?`	`wf:type.workflow.feedback.image.format` Requested image format. If omitted, the image is delivered in the original format.
`size?`	`xs:string` Requested image size in format `<Width>x<Height>` (e.g. `1000x800`). If omitted, the image is delivered in the original image size.
`preserveAspectRatio?`	`xs:boolean` Default: `true` Controls whether the aspect ratio of the image is preserved on scaling.
`index?`	`xs:unsignedInt` Requested image index for `from` elements that may provide more than one image.
`condition?`	`xs:IDREF` References a condition. Controls whether the feedback element is delivered. SHALL be considered fulfilled if empty or attribute was omitted.
`onRequestOnly?`	`xs:boolean` Default: `false` Controls whether the feedback element is available as part of the general feedback loop via `getWorkflowFeedback` or only upon explicit request via `getWorkflowFeedbackById`.

Table 6.127 type.workflow.feedback.image - attributes

6.5.66.2 Elements

None.

6.5.66.3 XSD Definition

```
<xs:complexType name="type.workflow.feedback.image">
 <xs:attribute name="id" type="xs:ID" use="required"/>
 <xs:attribute name="from" type="xs:IDREF" use="required"/>
 <xs:attribute name="format" type="wf:type.workflow.feedback.image.format" />
 <xs:attribute name="size" type="xs:string" />
 <xs:attribute name="preserveAspectRatio" type="xs:boolean" default="true"/>
 <xs:attribute name="index" type="xs:unsignedInt" />
 <xs:attribute name="condition" type="xs:IDREF" />
 <xs:attribute name="onRequestOnly" type="xs:boolean" default="false" />
</xs:complexType>
```

6.5.67 type.workflow.feedback.xml

Requests XML-formatted feedback. XML data is always returned in the `stringFeedback` field.

6.5.67.1 Attributes

The type.workflow.feedback.xml attributes are given in ▸Table 6.128.

Attribute name	Description
id	xs:ID Unique ID. Serves as reference.

from	xs:IDREF
	
	Reference to a data element. SHALL reference any of the following:
	• ICAO data group 1
	• ICAO data group 2
	• ICAO data group 3
	• ICAO data group 4
	• ICAO data group 11
	• ICAO data group 12
	• any field of type
	    • type.workflow.electronic.readseq.defectinfo
	    • type.workflow.readseq.check

format?	wf:type.workflow.feedback.image.format Requested image format. If omitted, the image is delivered in the original format.
size?	xs:string Requested image size in format <Width>x<Height> (e.g. 1000x800). If omitted, the image is delivered in the original image size.
preserveAspectRatio?	xs:boolean Default: true Controls whether the aspect ratio of the image is preserved on scaling.
profile	xs:string Logging profile on which the XPath evaluation is based.
path?	xs:string XPath 1.0 expression to limit the returned XML data to matching nodes. If empty or omitted, the complete XML data is delivered.
condition?	xs:IDREF References a condition. Controls whether the feedback element is delivered. SHALL be considered fulfilled if empty or attribute was omitted.
onRequestOnly?	xs:boolean Default: false Controls whether the feedback element is available as part of the general feedback loop via getWorkflowFeedback or only upon explicit request via getWorkflowFeedbackById.
innerTextOnly?	xs:boolean Default: false Controls whether only the inner text of a selected node is delivered as stringFeedback. If yes, path SHALL NOT be empty.

Table 6.128 type.workflow.feedback.xml - attributes

6.5.67.2 Elements

None.

6.5.67.3 XSD Definition

```
<xs:complexType name="type.workflow.feedback.xml">
 <xs:attribute name="id" type="xs:ID" use="required">
 <xs:attribute name="from" type="xs:IDREF" use="required"/>
 <xs:attribute name="format" type="wf:type.workflow.feedback.image.format" />
 <xs:attribute name="size" type="xs:string" />
 <xs:attribute name="preserveAspectRatio" type="xs:string" default="true" />
 <xs:attribute name="profile" type="xs:string" />
 <xs:attribute name="path" type="xs:string" />
 <xs:attribute name="condition" type="xs:IDREF" />
 <xs:attribute name="onRequestOnly" type="xs:boolean" default="false" />
 <xs:attribute name="innerTextOnly" type="xs:boolean" default="false" />
</xs:complexType>
```

6.5.68 type.workflow.feedback.image.format

Denotes the binary data format of image feedback. Derived from `xs:string`.

6.5.68.1 Values

The type.workflow.feedback.image.format values are given in ▸Table 6.129.

Value	Description
`bmp`	Bitmap
`jpeg`	JPEG
`jpeg2000`	JPEG 2000
`png`	Portable Network Graphics (Portable Network Graphics (PNG))
`wsq`	Wavelet Scalar Quantization (WSQ)
`iso19794_4`	Finger image data according to ISO 19794-4
`iso19794_5`	Face image data according to ISO 19794-5

Table 6.129 type.workflow.feedback.image.format - values

6.5.68.2 Elements

None.

6.5.68.3 XSD Definition

```
<xs:simpleType name="type.workflow.feedback.image.format">
 <xs:restriction base="xs:string">
  <xs:enumeration value="bmp" />
  <xs:enumeration value="jpeg" />
  <xs:enumeration value="jpeg2000" />
  <xs:enumeration value="png" />
  <xs:enumeration value="wsq" />
  <xs:enumeration value="iso19794_4" />
  <xs:enumeration value="iso19794_5" />
 </xs:restriction>
</xs:simpleType>
```

6.5.69 type.workflow.dependencies

List of dependencies between elements of the workflow. The dependencies always refer to elements from the reading sequence or the feedback definition. Cyclic dependencies SHALL be detected in `addWorkflow` and SHALL prevent the workflow from loading.

6.5.69.1 Attributes

None.

6.5.69.2 Elements

The type.workflow.dependencies elements are given in ▸Table 6.130.

Element name	Description
`FinishToStart*`	`wf:type.workflow.dependencies.finishtostart` A dependency where an element has to wait for another element.

Table 6.130 type.workflow.dependencies - elements

6.5.69.3 XSD Definition

```
<xs:complexType name="type.workflow.dependencies">
 <xs:choice minOccurs="0" maxOccurs="unbounded">
  <xs:element name="FinishToStart"
         type="wf:type.workflow.dependencies.finishtostart">
 </xs:choice>
</xs:complexType>
```

6.5.70 type.workflow.dependencies.finishtostart

Declares a sequential dependency. The processing of the first element SHALL be finished before the processing of the dependent element is started.

6.5.70.1 Attributes

The type.workflow.dependencies.finishtostart attributes are given in ▸Table 6.131.

Attribute name	Description
`id1`	`xs:IDREF` Reference to a data or feedback element. Processing of this element SHALL be finished before processing of `id2` starts.
`id2`	`xs:IDREF` Reference to a data or feedback element. Processing of this element SHALL NOT be started before processing of `id1` finishes.
`condition?`	`xs:IDREF` Reference to a condition. Controls whether this dependency is applicable. SHALL be considered fulfilled if empty or attribute was omitted.

Table 6.131 type.workflow.dependencies.finishtostart - attributes

6.5.70.2 Elements

None.

6.5.70.3 XSD Definition

```
<xs:complexType name="type.workflow.dependencies.finishtostart">
 <xs:attribute name="id1" type="xs:IDREF" use="required">
 <xs:attribute name="id2" type="xs:IDREF" use="required">
 <xs:attribute name="condition" type="xs:IDREF" >
</xs:complexType>
```

6.5.71 type.workflow.extension

Allows access to implementation-specific workflow extensions. The server SHALL consider known extensions during validation of the workflow definition. The server SHALL reject unknown extensions with an appropriate `WorkflowParserError` message.

6.5.71.1 Attributes

None.

6.5.71.2 Elements

MAY contain a single element of any type from a different namespace than http://trdoccheck.bsi.bund.de/hldc/workflow/5 [http://trdoccheck.bsi.bund.de/hldc/workflow/5].

6.5.71.3 XSD Definition

```
<xs:complexType name="type.workflow.extension" >
 <xs:sequence>
  <xs:any namespace="##other" minOccurs="0"
          processContents="lax" />
 </xs:sequence>
</xs:complexType>
```

7 Workflow feedback schema

Feedback requested by elements of type `type.workflow.feedback.xml` in the workflow definition is provided as an XML document in the `stringFeedback` element of `hldc5:WorkflowFeedback`.

The XML schema for the XML feedback is located in the file `fb_v5.xsd` and is based on XSD 1.0.

7.1 Feedback document

XML document that contains feedback requested by elements of `type.workflow.feedback.xml` in the workflow definition.

7.1.1 Root element

Element name	Description
`Feedback`	`wf:type.feedback` Root element of the feedback document.

Table 7.1 .

7.1.2 XSD Definition

```
<xs:element name="Feedback" type="wf:type.feedback" />
```

7.2 type.feedback

Root element of the XML feedback document. If an XPath expression was specified in the workflow definition of the relevant feedback request, the result is always returned in `XPathResult`. In all other cases,

the feedback is returned in the corresponding element.

7.2.1 Attributes

Attribute name	Description
`schemaVersion`	`xs:decimal` Feedback schema version. SHALL be “1”.

Table 7.2 .

7.2.2 Elements

`type.feedback` SHALL contain only one of the following elements.

Element name	Description
`DG1Data`	`wf:type.feedback.dg1` Data from data group 1.
`DG2Data`	`wf:type.feedback.dg2` Data from data group 2.
`DG3Data`	`wf:type.feedback.dg3` Data from data group 3.
`DG4Data`	`wf:type.feedback.dg4` Data from data group 4.
`DG11Data`	`wf:type.feedback.dg11` Data from data group 11.

Element name	Description
`DG12Data`	`wf:type.feedback.dg12` Data from data group 12.
`EIDDG17Data`	`wf:type.feedback.eid.placeofresidence` Data from eID data group 17.
`DefectInfo`	`wf:type.feedback.defects` Defect information.
`OpticalMRZData`	`wf:type.feedback.optmrz` Data from the optical MRZ.
`SealVisa`	`wf:type.feedback.seal.visa` Data from the digital seal on a visa.
`SealEmergencyTravelDocument`	`wf:type.feedback.seal.emergencytraveldocument` Data from the digital seal on an emergency travel document.
`SealArrivalAttestationDocument`	`wf:type.feedback.seal.arrivalattestationdocument` Data from the digital seal of an arrival attestation document.
`SealResidencePermit`	`wf:type.feedback.seal.residencepermit` Data from the digital seal on a residence permit.
`SealResidencePermitSupplementarySheet`	`wf:type.feedback.seal.residencepermitsupplementarysheet` Data from the digital seal on a residence permit supplementay sheet.
`SealAddressStickerGermanIdentityCard`	`wf:type.feedback.seal.addressstickergermanidentitycard` Data from the digital seal on a German identity card address sticker.
`SealUnknown`	`wf:type.feedback.seal.unknown` Data from the digital seal of an unknown type.
`AllBarcodePositions`	`wf:type.feedback.barcode.positions.perpage` Structure for all barcode positions of a document per page.
`FrontBarcodePositions`	`wf:type.feedback.barcode.positions` Structure for all barcode positions of the frontside of a document.
`BackBarcodePositions`	`wf:type.feedback.barcode.positions` Structure for all barcode positions of the backside of a document.
`ElectronicCheckResult`	`wf:type.feedback.checkresult` Result of an electronic check. Formatted according to the relevant document check namespace defined in [BSI TR-03135-1] , e.g. http://trdoccheck.bsi.bund.de/dce/4.
`OpticalCheckResult`	`wf:type.feedback.checkresult` Result of an optical check. Formatted according to the relevant document check namespace defined in [BSI TR-03135-1], e.g. http://trdoccheck.bsi.bund.de/dco/4.
`SealCheckResult`	`wf:type.feedback.checkresult` Result of a seal check. Formatted according to the relevant document check namespace defined in [BSI TR-03135-1], e.g. http://trdoccheck.bsi.bund.de/dcs/1.

Element name	Description
CombinedCheckResult	wf:type.feedback.checkresult Result of a combined check. Formatted according to the relevant document check namespace defined in [BSI TR-03135-1], e.g. http://trdoccheck.bsi.bund.de/dcc/4.
AggregatedDocumentCheckResult	wf:type.feedback.checkresult Result of an aggregation of document check results.
XPathResult	wf:type.feedback.xpathresult Subset of XML nodes of the original feedback (data group or check result). Used if an XPath expression is provided in the path element of type.workflow.feedback.xml.
ExtensionFeedback	wf:type.feedback.extended Implementation-specific feedback.
OCRCorrectionRequest	wf:type.feedback.interaction.correction.ocr Implementation-specific feedback.
ChipCorrectionRequest	wf:type.feedback.interaction.correction.chip Implementation-specific feedback.
SeriesSelectionRequest	wf:type.feedback.interaction.seriesselection Implementation-specific feedback.
PageRequest	wf:type.feedback.interaction.nextpage Implementation-specific feedback.

Table 7.3 .

7.2.3 XSD Definition

```
<xs:complexType name="type.feedback">
 <xs:choice>
  <xs:element name="DG1Data" type="wf:type.feedback.dg1" />
  <xs:element name="DG2Data" type="wf:type.feedback.dg2" />
  <xs:element name="DG3Data" type="wf:type.feedback.dg3" />
  <xs:element name="DG4Data" type="wf:type.feedback.dg4" />
  <xs:element name="DG11Data" type="wf:type.feedback.dg11" />
  <xs:element name="DG12Data" type="wf:type.feedback.dg12" />
  <xs:element name="EIDDG17Data" type="wf:type.feedback.eid.placeofresidence" />
  <xs:element name="DefectInfo" type="wf:type.feedback.defects" />
  <xs:element name="OpticalMRZData" type="wf:type.feedback.optmrz" />
  <xs:element name="SealVisa" type="wf:type.feedback.seal.visa" />
  <xs:element name="SealEmergencyTravelDocument"
   type="wf:type.feedback.seal.emergencytraveldocument" />
  <xs:element name="SealArrivalAttestationDocument"
   type="wf:type.feedback.seal.arrivalattestationdocument" />
  <xs:element name="SealSocialInsuranceCard" type="wf:type.feedback.seal.socialinsurancecard" />
  <xs:element name="SealResidencePermit" type="wf:type.feedback.seal.residencepermit" />
  <xs:element name="SealResidencePermitSupplementarySheet"
   type="wf:type.feedback.seal.residencepermitsupplementarysheet" />
  <xs:element name="SealAddressStickerGermanIdentityCard"
   type="wf:type.feedback.seal.addressstickergermanidentitycard"/>
  <xs:element name="SealUnknown" type="wf:type.feedback.seal.unknown"/>
  <xs:element name="AllBarcodePositions" type="wf:type.feedback.barcode.positions.perpage"/>
  <xs:element name="FrontBarcodePositions" type="wf:type.feedback.barcode.positions"/>
  <xs:element name="BackBarcodePositions" type="wf:type.feedback.barcode.positions"/>
  <xs:element name="ElectronicCheckResult" type="wf:type.feedback.checkresult" />
  <xs:element name="OpticalCheckResult" type="wf:type.feedback.checkresult" />
  <xs:element name="SealCheckResult" type="wf:type.feedback.checkresult" />
  <xs:element name="CombinedCheckResult" type="wf:type.feedback.checkresult" />
  <xs:element name="CrossDocumentCheckResult" type="wf:type.feedback.checkresult" />
  <xs:element name="AggregatedDocumentCheckResult" type="wf:type.feedback.checkresult" />
  <xs:element name="XPathResult" type="wf:type.feedback.xpathresult" />
```

```
   <xs:element name="ExtensionFeedback" type="wf:type.feedback.extended" />
   <xs:element name="OCRCorrectionRequest" type="wf:type.feedback.interaction.correction.ocr"/>
   <xs:element name="ChipCorrectionRequest" type="wf:type.feedback.interaction.correction.chip"/>
   <xs:element name="SeriesSelectionRequest" type="wf:type.feedback.interaction.seriesselection"/>
   <xs:element name="PageRequest" type="wf:type.feedback.interaction.nextpage"/>
  </xs:choice>
  <xs:attribute name="schemaVersion" type="xs:decimal" use="required" />
 </xs:complexType>
```

7.3 type.feedback.dg1

Contains the data from data group 1.

7.3.1 Attributes

None

7.3.2 Elements

Element name	Description
DocumentType	xs:string Type of the document.
Issuer	xs:string Issuer of the document.
GivenName	xs:string Given name of the document holder.
Surname	xs:string Surname of the document holder.
DocumentNumber	xs:string Document number (without check digit).
Nationality	xs:string Nationality of the document holder.
DateOfBirth	xs:string Date of birth of the document holder, encoded in the same way as in the MRZ (YYMMDD).
Sex	xs:string Sex of the document holder.
ExpiryDate	xs:string Expiry date of the document.
OptionalData	xs:string Optional data stored in the MRZ.
ChkDigitDocumentNumber	xs:string Check digit of the document number.
ChkDigitDateOfBirth	xs:string Check digit of the date of birth.
ChkDigitExpiryDate	xs:string Check digit of the expiry date.
ChkDigitOptionalData	xs:string Check digit for optional data.

Element name	Description
ChkDigitComposite	xs:string Check digit of the MRZ.
IsoDateOfBirth?	type.feedback.string.date The full date of birth.
IsoExpiryDate?	type.feedback.string.date The full date of expiry.

Table 7.4 .

7.3.3 XSD Definition

```
<xs:complexType name="type.feedback.dg1">
 <xs:sequence>
  <xs:element name="DocumentType" type="xs:string"/>
  <xs:element name="Issuer" type="xs:string"/>
  <xs:element name="GivenName" type="xs:string"/>
  <xs:element name="Surname" type="xs:string"/>
  <xs:element name="DocumentNumber" type="xs:string"/>
  <xs:element name="Nationality" type="xs:string"/>
  <xs:element name="DateOfBirth" type="xs:string"/>
  <xs:element name="Sex" type="xs:string"/>
  <xs:element name="ExpiryDate" type="xs:string"/>
  <xs:element name="OptionalData" type="xs:string"/>
  <xs:element name="ChkDgtDocumentNumber" type="xs:string"/>
  <xs:element name="ChkDgtDateOfBirth" type="xs:string"/>
  <xs:element name="ChkDgtExpiryDate" type="xs:string"/>
  <xs:element name="ChkDgtOptionalData" type="xs:string"/>
  <xs:element name="ChkDgtComposite" type="xs:string"/>
  <xs:element name="IsoDateOfBirth" type="wf:type.feedback.string.date" minOccurs="0"/>
  <xs:element name="IsoDateOfExpiry" type="wf:type.feedback.string.date" minOccurs="0"/>
 </xs:sequence>
</xs:complexType>
```

7.4 type.feedback.dg2

Contains a list of facial image templates.

7.4.1 Attributes

None

7.4.2 Elements

Element name	Description
Template*	wf:type.feedback.dg2.template A facial image template.

Table 7.5 .

7.4.3 XSD Definition

```
<xs:complexType name="type.feedback.dg2">
 <xs:sequence>
  <xs:element name="Template" minOccurs="0" maxOccurs="unbounded"
         type="wf:type.feedback.dg2.template" />
 </xs:sequence>
</xs:complexType>
```

7.5 type.feedback.dg2.template

Contains Base64-encoded image data of a single face. Derived from `xs:base64binary`.

7.5.1 Attributes

None

7.5.2 Elements

Element name	Description
`Image*`	`wf:type.feedback.dg2.image` A facial image template.

Table 7.6 .

7.5.3 XSD Definition

```
<xs:complexType name="type.feedback.dg2.template">
  <xs:sequence>
    <xs:element name="Image" minOccurs="0" maxOccurs="unbounded"
      type="wf:type.feedback.dg2.image" />
  </xs:sequence>
</xs:complexType>
```

7.6 type.feedback.dg2.image

Contains Base64-encoded image data of a single face. Derived from `xs:base64binary`.

7.6.1 Attributes

Attribute name	Description
`width`	`xs:int` The image width.
`height`	`xs:int` The image height.
`imageType`	`xs:unsignedByte` The type of the face image according to ISO 19794-5. Can contain the following values: • 0x00 – Basic • 0x01 – Full Frontal • 0x02 – Token Frontal • 0x03–0xFF – Reserved / Not used in the scope of this interface

Table 7.7 .

7.6.2 Elements

None

7.6.3 XSD Definition

```
<xs:complexType name="type.feedback.dg2.image">
 <xs:simpleContent>
  <xs:extension base="xs:base64Binary">
   <xs:attribute name="width" type="xs:int" use="required" />
   <xs:attribute name="height" type="xs:int" use="required" />
   <xs:attribute name="imageType" type="xs:unsignedByte"use="required" />
```

```
  </xs:extension>
 </xs:simpleContent>
</xs:complexType>
```

7.7 type.feedback.dg3

Contains a list of fingerprint templates.

7.7.1 Attributes

None.

7.7.2 Elements

Element name	Description
`Template*`	`wf:type.feedback.dg3.template` A fingerprint template.

Table 7.8 .

7.7.3 XSD Definition

```
<xs:complexType name="type.feedback.dg3">
 <xs:sequence>
  <xs:element name="Template" minOccurs="0" maxOccurs="unbounded"
       type="wf:type.feedback.dg3.template" />
 </xs:sequence>
</xs:complexType>
```

7.8 type.feedback.dg3.template

Contains a list of fingerprint images.

7.8.1 Attributes

None.

7.8.2 Elements

Element name	Description
`Image*`	`wf:type.feedback.dg3.image` A fingerprint image.

Table 7.9 .

7.8.3 XSD Definition

```
<xs:complexType name="type.feedback.dg3.template">
 <xs:sequence>
  <xs:element name="Image" minOccurs="0" maxOccurs="unbounded"
      type="wf:type.feedback.dg3.image" />
 </xs:sequence>
</xs:complexType>
```

7.9 type.feedback.dg3.image

Contains Base64-encoded image data of a single fingerprint according to ISO 19794-4. Derived from `xs:base64binary`.

7.9.1 Attributes

Attribute name	Description
`width`	`xs:int` The image width.
`height`	`xs:int` The image height.
`fingerPos`	`type.feedback.dg3.image.fingerpos` Finger position code according to ISO 19794-4.

Table 7.10 .

7.9.2 Elements

None.

7.9.3 XSD Definition

```
<xs:complexType name="type.feedback.dg3.image">
 <xs:simpleContent>
  <xs:extension base="xs:base64Binary">
   <xs:attribute name="width" type="xs:int" use="required" />
   <xs:attribute name="height" type="xs:int" use="required" />
   <xs:attribute name="fingerPos" type="wf:type.feedback.dg3.image.fingerpos" use="required" />
  </xs:extension>
 </xs:simpleContent>
</xs:complexType>
```

7.10 type.feedback.dg3.image.fingerpos

Represents finger position codes according to ISO 19794-4 (Table 6).

7.10.1 Values

Value	Description
0	Unknown
1	Right thumb
2	Right index finger
3	Right middle finger
4	Right ring finger
5	Right little finger
6	Left thumb
7	Left index finger
8	Left middle finger
9	Left ring finger
10	Left little finger
13	Plain right four fingers
14	Plain left four fingers
15	Plain thumbs

Table 7.11 .

7.10.2 XSD Definition

```
<xs:simpleType name="type.feedback.dg3.image.fingerpos">
 <xs:union>
  <xs:simpleType>
   <xs:restriction base="xs:unsignedByte">
    <xs:minInclusive value="0" />
    <xs:maxInclusive value="10" />
   </xs:restriction>
  </xs:simpleType>
  <xs:simpleType>
   <xs:restriction base="xs:unsignedByte">
    <xs:minInclusive value="13" />
    <xs:maxInclusive value="15" />
   </xs:restriction>
  </xs:simpleType>
 </xs:union>
</xs:simpleType>
```

7.11 type.feedback.dg4

Contains a list of iris templates.

7.11.1 Attributes

None.

7.11.2 Elements

Element name	Description
Template*	wf:type.feedback.dg4.template An iris template.

Table 7.12 .

7.11.3 XSD Definition

```
<xs:complexType name="type.feedback.dg4">
 <xs:sequence>
  <xs:element name="Template" minOccurs="0" maxOccurs="unbounded"
        type="wf:type.feedback.dg4.template" />
 </xs:sequence>
</xs:complexType>
```

7.12 type.feedback.dg4.template

Contains a list of iris images.

7.12.1 Attributes

None.

7.12.2 Elements

Element name	Description
Image*	wf:type.feedback.dg4.image An iris image.

Table 7.13 .

7.12.3 XSD Definition

```
<xs:complexType name="type.feedback.dg4.template">
 <xs:sequence>
  <xs:element name="Image" minOccurs="0" maxOccurs="unbounded"
         type="wf:type.feedback.dg4.image" />
 </xs:sequence>
</xs:complexType>
```

7.13 type.feedback.dg4.image

Contains Base64-encoded image data of a single iris. Derived from `xs:base64binary`.

7.13.1 Attributes

Attribute name	Description
`width`	`xs:int` The image width.
`height`	`xs:int` The image height.

Table 7.14 .

7.13.2 Elements

None.

7.13.3 XSD Definition

```
<xs:complexType name="type.feedback.dg4.image">
 <xs:simpleContent>
  <xs:extension base="xs:base64Binary">
   <xs:attribute name="width" type="xs:int" use="required">
   <xs:attribute name="height" type="xs:int" use="required">
 </xs:extension>
 </xs:simpleContent>
</xs:complexType>
```

7.14 type.feedback.dg11

Contains the XML feedback for data group 11.

7.14.1 Attributes

None.

7.14.2 Elements

Element name	Description
`Name`	`wf:type.feedback.name` The full name of the person.
`PersonalNumber`	`xs:string` Personal number.
`FullDateOfBirth`	`xs:string` The full date of birth including the century in format DD.MM.YYYY.
`PlaceOfBirth`	`xs:string` The place of birth.

Element name	Description
PermanentAddress	xs:string The permanent address.
TelephoneNumber	xs:string The telephone number.
Profession	xs:string The profession.
Title	xs:string The title of the person.
PersonalSummary	xs:string Personal summary.
TDNumbers	xs:string Additional travel document numbers.
CustodyInformation	xs:string Custody information.
OtherNames	wf:type.feedback.namelist List of additional names.
ProofOfCitizenship?	xs:base64Binary Image of the proof of citizenship.

Table 7.15 .

7.14.3 XSD Definition

```
<xs:complexType name="type.feedback.dg11">
 <xs:sequence>
  <xs:element name="Name" type="wf:type.feedback.name"/>
  <xs:element name="PersonalNumber" type="xs:string"/>
  <xs:element name="FullDateOfBirth" type="xs:string"/>
  <xs:element name="PlaceOfBirth" type="xs:string"/>
  <xs:element name="PermanentAddress" type="xs:string"/>
  <xs:element name="TelephoneNumber" type="xs:string"/>
  <xs:element name="Profession" type="xs:string"/>
  <xs:element name="Title" type="xs:string"/>
  <xs:element name="PersonalSummary" type="xs:string"/>
  <xs:element name="TDNumbers" type="xs:string"/>
  <xs:element name="CustodyInformation" type="xs:string"/>
  <xs:element name="OtherNames" type="wf:type.feedback.namelist"/>
  <xs:element name="ProofOfCitizenship" minOccurs="0" type="xs:base64Binary"/>
 </xs:sequence>
</xs:complexType>
```

7.15 type.feedback.dg12

Contains the XML feedback for data group 12.

7.15.1 Attributes

None.

7.15.2 Elements

Element name	Description
IssuingAuthority	xs:string The issuing authority.
DateOfIssue	xs:string The date of issue in format DD.MM.YYYY.
Endorsements	xs:string Endorsements.
TaxExitRequirements	xs:string The tax and exit requirements
PersTime	xs:string Time of personalization in format DD.MM.YYYY HH:MM:SS.
SnrPersSystem	xs:string The serial number of the personalization system.
OtherPeople	wf:type.feedback.namelist List of other people.
FrontImage?	xs:base64Binary Front image of the document.
RearImage?	xs:base64Binary Rear image of the document.

Table 7.16 .

7.15.3 XSD Definition

```
<<xs:complexType name="type.feedback.dg12">
 <xs:sequence>
  <xs:element name="IssuingAuthority" type="xs:string"/>
  <xs:element name="DateOfIssue" type="xs:string"/>
  <xs:element name="Endorsements" type="xs:string"/>
  <xs:element name="TaxExitRequirements" type="xs:string"/>
  <xs:element name="PersTime" type="xs:string"/>
  <xs:element name="SnrPersSystem" type="xs:string"/>
  <xs:element name="OtherPeople" type="wf:type.feedback.namelist"/>
  <xs:element name="FrontImage" minOccurs="0" type="xs:base64Binary"/>
  <xs:element name="RearImage" minOccurs="0" type="xs:base64Binary"/>
 </xs:sequence>
</xs:complexType>
```

7.16 type.feedback.optmrz

Contains optical MRZ data.

7.16.1 Attributes

None.

7.16.2 Elements

Element name	Description
DocumentType	xs:string Type of the document.

Element name	Description
Issuer	xs:string Issuer of the travel document.
GivenName	xs:string Given name of the document holder.
Surname	xs:string Surname of the document holder.
DocumentNumber	xs:string Document number (without check digit).
Nationality	xs:string Nationality of the document holder.
DateOfBirth	xs:string Date of birth of the document holder, encoded in the same way as in the MRZ (YYMMDD).
Sex	xs:string Sex of the document holder.
ExpiryDate	xs:string Expiry date of the document.
OptionalData	xs:string Optional data stored in the MRZ.
ChkDigitDocumentNumber	xs:string Check digit of the document number.
ChkDigitDateOfBirth	xs:string Check digit of the date of birth.
ChkDigitExpiryDate	xs:string Check digit of the expiry date.
ChkDigitOptionalData	xs:string Check digit for optional data.
ChkDigitComposite	xs:string Check digit of the MRZ.
IsoDateOfBirth?	xs:string The full date of birth.
IsoExpiryDate?	xs:string The full date of expiry.

Table 7.17 .

7.16.3 XSD Definition

```
<xs:complexType name="type.feedback.optmrz">
 <xs:sequence>
  <xs:element name="DocumentType" type="xs:string"/>
  <xs:element name="Issuer" type="xs:string"/>
  <xs:element name="GivenName" type="xs:string"/>
  <xs:element name="Surname" type="xs:string"/>
  <xs:element name="DocumentNumber" type="xs:string"/>
  <xs:element name="Nationality" type="xs:string"/>
  <xs:element name="DateOfBirth" type="xs:string"/>
  <xs:element name="Sex" type="xs:string"/>
```

```
    <xs:element name="ExpiryDate" type="xs:string"/>
    <xs:element name="OptionalData" type="xs:string"/>
    <xs:element name="ChkDgtDocumentNumber" type="xs:string"/>
    <xs:element name="ChkDgtDateOfBirth" type="xs:string"/>
    <xs:element name="ChkDgtExpiryDate" type="xs:string"/>
    <xs:element name="ChkDgtOptionalData" type="xs:string"/>
    <xs:element name="ChkDgtComposite" type="xs:string"/>
    <xs:element name="IsoDateOfBirth" type="wf:type.feedback.string.date" minOccurs="0"/>
    <xs:element name="IsoDateOfExpiry" type="wf:type.feedback.string.date" minOccurs="0"/>
  </xs:sequence>
</xs:complexType>
```

7.17 type.feedback.eid.placeofresidence

Type for eID data group 17 place of residence.

7.17.1 Attributes

None.

7.17.2 Elements

Element name	Description
StructuredPlace*	wf:type.feedback.eid.placeofresidence.structuredplace Contains a structured representation of the place of residence.
FreeTextPlace*	xs:string Contains a free text representation of the place.
NoPlaceInfo*	xs:string Indicates that no place of residence is present.

Table 7.18 .

7.17.3 XSD Definition

```
<xs:complexType name="type.feedback.eid.placeofresidence">
  <xs:choice minOccurs="0" maxOccurs="unbounded">
    <xs:element name="StructuredPlace"
        type="wf:type.feedback.eid.placeofresidence.structuredplace" />
    <xs:element name="FreeTextPlace" type="xs:string" />
    <xs:element name="NoPlaceInfo" type="xs:string" />
  </xs:choice>
</xs:complexType>
```

7.18 type.feedback.eid.placeofresidence.structuredplace

Type for structured place of residence information.

7.18.1 Attributes

None.

7.18.2 Elements

Element name	Description
City	xs:string Contains the city of the structured place.
Country	xs:string Contains the country of the structured place.

Element name	Description
Street?	xs:string Contains the street of the structured place.
State?	xs:string Contains the state of the structured place.
ZipCode?	xs:string Contains the zip code of the structured place.

Table 7.19 .

7.18.3 XSD Definition

```
<xs:complexType name="type.feedback.eid.placeofresidence.structuredplace">
 <xs:sequence>
  <xs:element name="City" type="xs:string" />
  <xs:element name="Country" type="xs:string" />
  <xs:element name="Street" type="xs:string" minOccurs="0" />
  <xs:element name="State" type="xs:string" minOccurs="0" />
  <xs:element name="ZipCode" type="xs:string" minOccurs="0" />
 </xs:sequence>
</xs:complexType>
```

7.19 type.feedback.defects

Contains a list of known defects for the current document.

7.19.1 Attributes

None.

7.19.2 Elements

Element name	Description
Defect*	wf:type.feedback.defects.defect A known defect for the current document.

Table 7.20 .

7.19.3 XSD Definition

```
<xs:complexType name="type.feedback.defects">
 <xs:sequence>
  <xs:element name="Defect" minOccurs="0" maxOccurs="unbounded"
         type="wf:type.feedback.defects.defect" />
 </xs:sequence>
</xs:complexType>
```

7.20 type.feedback.oid

Description of an ASN.1 Object Identifier (OID).

7.20.1 Format restrictions

The content SHALL match the specified regular expression pattern, for example “1.2.3.4”.

7.20.2 XSD Definition

```
<xs:complexType name="type.feedback.oid">
 <xs:restriction base="xs:string">
```

```
  <xs:pattern value="(\d+\.)+\d+"/>
  </xs:restriction>
</xs:complexType>
```

7.21 type.feedback.defects.defect

Description of a defect.

7.21.1 Attributes

None.

7.21.2 Elements

Element name	Description
`Oid`	`wf:type.feedback.oid` The defect OID according to TR-03129-2.
`ParameterInfo?`	`xs:string` Additional information about the defect which must be interpreted in the context of the defect type.

Table 7.21 .

7.21.3 XSD Definition

```
<xs:complexType name="type.feedback.defects.defect">
 <xs:sequence>
  <xs:element name="Oid"
          type="wf:type.feedback.oid" />
  <xs:element name="ParameterInfo" minOccurs="0"
          type="xs:string" />
 </xs:sequence>
</xs:complexType>
```

7.22 type.feedbacks.name

Detailed name information.

7.22.1 Attributes

None.

7.22.2 Elements

Element name	Description
`FullName`	`xs:string` The full name.
`GivenName`	`xs:string` The given name.
`Surname`	`xs:string` The surname.

Table 7.22 .

7.22.3 XSD Definition

```
<xs:complexType name="type.feedback.name">
 <xs:sequence>
  <xs:element name="FullName" type="xs:string"/>
```

```
  <xs:element name="GivenName" type="xs:string"/>
  <xs:element name="Surname" type="xs:string"/>
 </xs:sequence>
</xs:complexType>
```

7.23 type.feedback.namelist

List of names.

7.23.1 Attributes

None.

7.23.2 Elements

Element name	Description
`Name*`	`wf:type.feedback.name` The full name.

Table 7.23 .

7.23.3 XSD Definition

```
<xs:complexType name="type.feedback.namelist">
 <xs:sequence>
  <xs:element name="Name" minOccurs="0" maxOccurs="unbounded"
         type="wf:type.feedback.name" />
 </xs:sequence>
</xs:complexType>
```

7.24 type.feedback.checkresult

Contains the results of an electronic, optical or combined check. Results are formatted according to the relevant document check schemas defined in [BSI TR-03135-1], e.g. `document_check_electronic_v2.xsd`.

7.24.1 Attributes

None.

7.24.2 Elements

MAY contain one element of any type from a different namespace than http://trdoccheck.bsi.bund.de/hldc/workflow/1 [http://trdoccheck.bsi.bund.de/hldc/workflow/1], which is the root element of the relevant document check schema.

7.24.3 XSD Definition

```
<xs:complexType name="type.feedback.extended">
 <xs:sequence>
 <xs:any namespace="##other" minOccurs="0"
 processContents="lax" />
 </xs:sequence>
</xs:complexType>
```

7.25 type.feedback.xpathresult

Contains a subset of XML nodes of the original feedback (data group or check result) if an XPath expression is provided in the `path` element of `type.workflow.feedback.xml`.

7.25.1 Attributes

XPath-based feedback MAY contain any number of any attributes.

7.25.2 Elements

XPath-based feedback MAY contain any number of any elements.

7.25.3 XSD Definition

```
<xs:complexType name="type.feedback.xpathresult">
 <xs:sequence>
  <xs:any minOccurs="0" maxOccurs="unbounded" processContents="lax" />
 </xs:sequence>
 <xs:anyAttribute processContents="lax" />
</complexType>
```

7.26 type.feedback.seal.visa

Contains the data of a digital seal on a visa.

7.26.1 Attributes

None.

7.26.2 Elements

Element name	Description
UnfoldedMRZ	wf:type.feedback.mrz.unfolded.shortened Truncated MRZ contained in the digital seal.
DurationOfStay	wf:type.feedback.seal.visa.durationofstay Duration of stay.
PassportNumber	wf:type.feedback.seal.passportnumber Number of the passport corresponding to the visa.
VisaType?	xs:base64Binary Visa type encoded in the seal
NumberOfEntries?	xs:unsignedByte Number of entries permitted using the visa document.

Table 7.24 .

7.26.3 XSD Definition

```
 <xs:complexType name="type.feedback.seal.visa">
   <xs:sequence>
    <xs:element name="UnfoldedMRZ" type="wf:type.feedback.mrz.unfolded.shortened" />
    <xs:element name="DurationOfStay" type="wf:type.feedback.seal.visa.durationofstay" />
    <xs:element name="PassportNumber" type="wf:type.feedback.seal.passportnumber" />
    <xs:element name="VisaType" type="xs:base64Binary" minOccurs="0"/>
    <xs:element name="NumberOfEntries" type="xs:unsignedByte" minOccurs="0"/>
   </xs:sequence>
  </xs:complexType>
```

7.27 type.feedback.seal.visa.durationofstay

Contains the duration permitted to stay according to the visa document.

7.27.1 Attributes

None.

7.27.2 Elements

Element name	Description
NumberOfDays	xs:unsignedByte Amount of days permitted to stay.
NumberOfMonths	xs:unsignedByte Amount of months permitted to stay.
NumberOfYears	xs:unsignedByte Amount of years permitted to stay.

Table 7.25 .

7.27.3 XSD Definition

```
<xs:complexType name="type.feedback.seal.visa.durationofstay">
 <xs:sequence>
  <xs:element name="NumberOfDays" type="xs:unsignedByte" />
  <xs:element name="NumberOfMonths" type="xs:unsignedByte" />
  <xs:element name="NumberOfYears" type="xs:unsignedByte" />
 </xs:sequence>
</xs:complexType>
```

7.28 type.feedback.seal.emergencytraveldocument

Contains the data of a digital seal on an emergency travel document.

7.28.1 Attributes

None.

7.28.2 Elements

Element name	Description
AdditionalFeatures	xs:string Features defined by the issuer.
UnfoldedMRZ	wf:type.feedback.mrz.unfolded Content of the MRZ contained in the emergency travel document.

Table 7.26 .

7.28.3 XSD Definition

```
<xs:complexType name="type.feedback.seal.emergencytraveldocument">
 <xs:sequence>
  <xs:element name="AdditionalFeatures" type="xs:string"/>
  <xs:element name="UnfoldedMRZ" type="wf:type.feedback.mrz.unfolded"/>
 </xs:sequence>
</xs:complexType>
```

7.29 type.feedback.seal.arrivalattestationdocument

Contains the data of a digital seal on an arrival attestation document.

7.29.1 Attributes

None.

7.29.2 Elements

Element name	Description
`AzrNumber`	`wf:type.feedback.seal.arrivalattestationdocumentazrnumber` Number of the AZR.
`UnfoldedMRZ`	`wf:type.feedback.mrz.unfolded` Content of the MRZ contained in the arrival attestation document.

Table 7.27 .

7.29.3 XSD Definition

```
<xs:complexType name="type.feedback.seal.arrivalattestationdocument">
 <xs:sequence>
  <xs:element name="AzrNumber"
                  type="wf:type.feedback.seal.arrivalattestationdocumentazrnumber"/>
  <xs:element name="UnfoldedMRZ" type="wf:type.feedback.mrz.unfolded"/>
 </xs:sequence>
</xs:complexType>
```

7.30 type.feedback.seal.socialinsurancecard

Contains the data of a digital seal on a social insurance card.

7.30.1 Attribute

None.

7.30.2 Elements

Element name	Description
`SocialInsuranceNumber`	`wf:type.feedback.seal.socialinsurancecard.socialinsuran-cenumber` Number of the social insurance.
`Surname`	`wf:type.feedback.seal.socialinsurancecard.surname` Surname contained in the document.
`Firstname`	`wf:type.feedback.seal.socialinsurancecard.firstname` Firstname contained in the document.
`NameAtBirth?`	`wf:type.feedback.seal.socialinsurancecard.nameatbirth` Name at birth of the document holder.

Table 7.28 .

7.30.3 XSD Definition

```
<xs:complexType name="type.feedback.seal.socialinsurancecard">
 <xs:sequence>
  <xs:element name="SocialInsuranceNumber"
       type="wf:type.feedback.seal.socialinsurancecard.socialinsurancenumber"/>
  <xs:element name="Surname" type="wf:type.feedback.seal.socialinsurancecard.surname"/>
  <xs:element name="Firstname" type="wf:type.feedback.seal.socialinsurancecard.firstname"/>
  <xs:element name="NameAtBirth"
       type="wf:type.feedback.seal.socialinsurancecard.nameatbirth" minOccurs="0"/>
 </xs:sequence>
```

```
</xs:complexType>
```

7.31 type.feedback.seal.residencepermit

Contains the data of a digital seal on a residence permit.

7.31.1 Attribute

None.

7.31.2 Elements

Element name	Description
PassportNumber	wf:type.feedback.seal.passportnumber Number of the corresponding passport.
UnfoldedMRZ	wf:type.feedback.mrz.unfolded MRZ contained in the document.

Table 7.29 .

7.31.3 XSD Definition

```
<xs:complexType name="type.feedback.seal.residencepermit">
 <xs:sequence>
  <xs:element name="PassportNumber" type="wf:type.feedback.seal.passportnumber"/>
  <xs:element name="UnfoldedMRZ" type="wf:type.feedback.mrz.unfolded"/>
 </xs:sequence>
</xs:complexType>
```

7.32 type.feedback.seal.residencepermitsupplementarysheet

Contains the data of a residence permit supplementary sheet.

7.32.1 Attribute

None.

7.32.2 Elements

Element name	Description
NumberOfSupplementarySheet	wf:type.feedback.seal.residencepermitsupplementarysheet.n-umberofsupplementarysheet Number of the supplementary sheet.
UnfoldedMRZ	wf:type.feedback.mrz.unfolded MRZ contained in the document.

Table 7.30 .

7.32.3 XSD Definition

```
<xs:complexType name="type.feedback.seal.residencepermitsupplementarysheet">
 <xs:sequence>
   <xs:element name="NumberOfSupplementarySheet"
     type="wf:type.feedback.seal.residencepermitsupplementarysheet.numberofsupplementarysheet"/>
   <xs:element name="UnfoldedMRZ" type="wf:type.feedback.mrz.unfolded"/>
 </xs:sequence>
</xs:complexType>
```

7.33 type.feedback.seal.addressstickergermanidentitycard

Contains the content of an address sticker of a German identity card.

7.33.1 Attribute

None.

7.33.2 Elements

Element name	Description
DocumentNumber	xs:string Number of the document.
OfficialMunicipalityCodeNumber	xs:string Official municipality code number.
ResidentialAddress	xs:string Residential address.

Table 7.31 .

7.33.3 XSD Definition

```
<xs:complexType name="type.feedback.seal.addressstickergermanidentitycard">
 <xs:sequence>
  <xs:element name="DocumentNumber" type="xs:string"/>
  <xs:element name="OfficialMunicipalityCodeNumber" type="xs:string"/>
  <xs:element name="ResidentialAddress" type="xs:string"/>
 </xs:sequence>
</xs:complexType>
```

7.34 type.feedback.seal.unknown

Contains the content of an unknown seal type.

7.34.1 Attribute

None.

7.34.2 Elements

Element name	Description
BinaryContent	xs:base64Binary Digita seal content in an unsupported format.

Table 7.32 .

7.34.3 XSD Definition

```
 <xs:complexType name="type.feedback.seal.unknown">
 <xs:sequence>
  <xs:element name="BinaryContent" type="xs:base64Binary"/>
 </xs:sequence>
</xs:complexType>
```

7.35 type.feedback.barcode.positions.perpage

Contains a list for all barcode positions per page.

7.35.1 Attributes

None.

7.35.2 Elements

Element name	Description
BarcodePositionsPage+	wf:type.feedback.barcode.positions All barcode positions for a single page.

Table 7.33 .

7.35.3 XSD Definition

```
<xs:complexType name="type.feedback.barcode.positions.perpage">
 <xs:sequence>
  <xs:element name="BarcodePositionsPage" type="wf:type.feedback.barcode.positions"
    minOccurs="1" maxOccurs="unbounded"/>
 </xs:sequence>
</xs:complexType>
```

7.36 type.feedback.barcode.positions

Contains a list for all barcode positions of a single page.

7.36.1 Attributes

None.

7.36.2 Elements

Element name	Description
ImageRef	xs:string Reference to the image for which the positions are given. The reference SHALL match a feedbackId of the image data.
ImagePositions?	xs:string List of barcode positions on the referenced image.

Table 7.34 .

7.36.3 XSD Definition

```
<xs:complexType name="type.feedback.barcode.positions">
 <xs:sequence>
  <xs:element name="ImageRef" type="xs:string"/>
  <xs:element name="ImagePositions" type="wf:type.feedback.barcode.image.positions"
    minOccurs="0"/>
 </xs:sequence>
</xs:complexType>
```

7.37 type.feedback.image.positions

Contains a list for rectangles which mark areas on an image.

7.37.1 Attributes

None.

7.37.2 Elements

Element name	Description
`ImagePosition*`	`wf:type.feedback.image.position.rect` Single rectangle to mark an area on an image

Table 7.35 .

7.37.3 XSD Definition

```
<xs:complexType name="type.feedback.barcode.image.positions">
 <xs:sequence>
  <xs:element name="ImagePosition" type="wf:type.feedback.image.position.rect"
    minOccurs="0" maxOccurs="unbounded"/>
 </xs:sequence>
</xs:complexType>
```

7.38 type.feedback.image.position.rect

Represents a rectangle on an image. The origin of the coordinate system SHALL be the lower left corner of the image.

7.38.1 Attributes

None.

7.38.2 Elements

Element name	Description
`x1`	`xs:int` Coordinate for the left edge of the rectangle.
`y1`	`xs:int` Coordinate for the bottom edge of the rectangle.
`x2`	`xs:int` Coordinate for the right edge of the rectangle.
`y2`	`xs:int` Coordinate for the top edge of the rectangle.

Table 7.36 .

7.38.3 XSD Definition

```
<xs:complexType name="type.feedback.image.position.rect">
 <xs:sequence>
  <xs:element name="x1" type="xs:int"/>
  <xs:element name="y1" type="xs:int"/>
  <xs:element name="x2" type="xs:int"/>
  <xs:element name="y2" type="xs:int"/>
 </xs:sequence>
</xs:complexType>
```

7.39 type.feedback.mrz.unfolded

Contains an unfolded MRZ.

7.39.1 Attributes

None.

7.39.2 Elements

Element name	Description
DocumentType	xs:string The type of the document.
Issuer	xs:string The issuer of the document.
GivenName	xs:string The given name of the document holder.
Surname	xs:string The surname of the document holder.
DocumentNumber	xs:string The number of the document.
Nationality	xs:string The nationality of the document holder.
DateOfBirth	xs:string The date of birth of the document holder.
Sex	xs:string The sex of the document holder.
ExpiryDate	xs:string The expiry date of the document.
OptionalData	xs:string Optional data in the document.
ChkDgtDocumentNumber	xs:string The check digit of the document number.
ChkDgtDateOfBirth	xs:string The check digit of the date of birth number.
ChkDgtExpiryDate	xs:string The check digit of the expiry date.
ChkDgtOptionalData	xs:string The check digit of the optional data.
ChkDgtComposite	xs:string The composite check digit.
IsoDateOfBirth?	wf:type.feedback.string.date The date of birth in ISO format.
IsoDateOfExpiry?	wf:type.feedback.string.date The date of expiry in ISO format.

Table 7.37 .

7.39.3 XSD Definition

```
<xs:complexType name="type.feedback.mrz.unfolded">
<xs:sequence>
 <xs:element name="DocumentType" type="xs:string" />
 <xs:element name="Issuer" type="xs:string" />
 <xs:element name="GivenName" type="xs:string" />
 <xs:element name="Surname" type="xs:string" />
```

```
    <xs:element name="DocumentNumber" type="xs:string" />
    <xs:element name="Nationality" type="xs:string" />
    <xs:element name="DateOfBirth" type="xs:string" />
    <xs:element name="Sex" type="xs:string" />
    <xs:element name="ExpiryDate" type="xs:string" />
    <xs:element name="OptionalData" type="xs:string" />
    <xs:element name="ChkDgtDocumentNumber" type="xs:string" />
    <xs:element name="ChkDgtDateOfBirth" type="xs:string" />
    <xs:element name="ChkDgtExpiryDate" type="xs:string" />
    <xs:element name="ChkDgtOptionalData" type="xs:string" />
    <xs:element name="ChkDgtComposite" type="xs:string" />
    <xs:element name="IsoDateOfBirth" type="wf:type.feedback.string.date" minOccurs="0" />
    <xs:element name="IsoDateOfExpiry" type="wf:type.feedback.string.date" minOccurs="0" />
   </xs:sequence>
  </xs:complexType>
```

7.40 type.feedback.mrz.unfolded.shortened

Contains a shortened unfolded MRZ, i.e. without optional data and optional data check digit.

7.40.1 Attributes

None.

7.40.2 Elements

Element name	Description
DocumentType	xs:string The type of the document.
Issuer	xs:string The issuer of the document.
GivenName	xs:string The given name of the document holder.
Surname	xs:string The surname of the document holder.
DocumentNumber	xs:string The number of the document.
Nationality	xs:string The nationality of the document holder.
DateOfBirth	xs:string The date of birth of the document holder.
Sex	xs:string The sex of the document holder.
ExpiryDate	xs:string The expiry date of the document.
ChkDgtDocumentNumber	xs:string The check digit of the document number.
ChkDgtDateOfBirth	xs:string The check digit of the date of birth number.
ChkDgtExpiryDate	xs:string The check digit of the expiry date.

Element name	Description
ChkDgtComposite	xs:string The composite check digit.
IsoDateOfBirth?	wf:type.feedback.string.date The date of birth in ISO format.
IsoDateOfExpiry?	wf:type.feedback.string.date The date of expiry in ISO format.

Table 7.38 .

7.40.3 XSD Definition

```
  <xs:complexType name="type.feedback.mrz.unfolded.shortened">
  <xs:sequence>
   <xs:element name="DocumentType" type="xs:string" />
   <xs:element name="Issuer" type="xs:string" />
   <xs:element name="GivenName" type="xs:string" />
   <xs:element name="Surname" type="xs:string" />
   <xs:element name="DocumentNumber" type="xs:string" />
   <xs:element name="Nationality" type="xs:string" />
   <xs:element name="DateOfBirth" type="xs:string" />
   <xs:element name="Sex" type="xs:string" />
   <xs:element name="ExpiryDate" type="xs:string" />
   <xs:element name="ChkDgtDocumentNumber" type="xs:string" />
   <xs:element name="ChkDgtDateOfBirth" type="xs:string" />
   <xs:element name="ChkDgtExpiryDate" type="xs:string" />
   <xs:element name="ChkDgtComposite" type="xs:string" />
   <xs:element name="IsoDateOfBirth" type="wf:type.feedback.string.date" minOccurs="0" />
   <xs:element name="IsoDateOfExpiry" type="wf:type.feedback.string.date" minOccurs="0" />
  </xs:sequence>
 </xs:complexType>
```

7.41 type.feedback.string.date

Represents a date string with full year information. Missing parts shall be encoded with “00”, e.g. “1990-10-00” or “2011-00-00”.

7.41.1 Values

The string SHALL conform to the pattern `\d{4}-\d{2}-\d{2}`.

7.41.2 XSD Definition

```
<xs:simpleType name="type.feedback.string.date">
 <xs:restriction base="xs:string">
  <xs:pattern value="\d{4}-\d{2}-\d{2}" />
 </xs:restriction>
</xs:simpleType>
```

7.42 type.feedback.extended

Allows the server to return implementation-specific feedback. SHALL only be used if the workflow definition contains corresponding requests for custom feedback as part of an implementation-specific workflow extension configuration.

7.42.1 Attributes

None.

7.42.2 Elements

Implementation-specific feedback MAY contain one element of any type from a different namespace than `http://trdoccheck.bsi.bund.de/hldc/workflow/1` [http://trdoccheck.bsi.bund.de/hldc/workflow/1].

7.42.3 XSD Definition

```
<xs:complexType name="type.feedback.extended">
 <xs:sequence>
  <xs:any namespace="##other" minOccurs="0"
          processContents="lax" />
 </xs:sequence>
</xs:complexType>
```

7.43 type.feedback.interaction.correction.ocr

Contains information about a text which has to be corrected in case of OCR errors.

7.43.1 Attributes

None.

7.43.2 Elements

Element name	Description
`Image`	`xs:base64Binary` The image which contains the text which has to be corrected.
`Text`	`xs:string` The text which has to be corrected.
`Hints?`	`wf:type.feedback.interaction.correction.hints` The server MAY send hints to assist in the text correction.

Table 7.39 .

7.43.3 XSD Definition

```
<xs:complexType name="type.feedback.interaction.correction.ocr">
 <xs:sequence>
  <xs:element name="Image" type="xs:base64Binary" />
  <xs:element name="Text" type="xs:string"/>
  <xs:element name="Hints" type="wf:type.feedback.interaction.correction.hints" minOccurs="0"/>
 </xs:sequence>
</xs:complexType>
```

7.44 type.feedback.interaction.correction.chip

Contains information about a text which has to be corrected in case of chip access errors.

7.44.1 Attributes

None.

7.44.2 Elements

Element name	Description
`Image?`	`xs:base64Binary` The image which contains the text which has to be corrected.

Element name	Description
Text	xs:string The text which has to be corrected.
Hints?	wf:type.feedback.interaction.correction.hints The server MAY send hints to assist in the text correction.

Table 7.40 .

7.44.3 XSD Definition

```
<xs:complexType name="type.feedback.interaction.correction.chip">
 <xs:sequence>
  <xs:element name="Image" type="xs:base64Binary" minOccurs="0" maxOccurs="1"/>
  <xs:element name="Text" type="xs:string"/>
  <xs:element name="Hints" type="wf:type.feedback.interaction.correction.hints" minOccurs="0"/>
 </xs:sequence>
</xs:complexType>
```

7.45 type.feedback.interaction.correction.hints

Contains a list of hints to assist in text correction.

7.45.1 Attributes

None.

7.45.2 Elements

Element name	Description
Hint+	wf:type.feedback.interaction.correction.hint Single hint to assist in text correction.

Table 7.41 .

7.45.3 XSD Definition

```
<xs:complexType name="type.feedback.interaction.correction.hints">
 <xs:sequence>
  <xs:element name="Hint" type="wf:type.feedback.interaction.correction.hint"
    minOccurs="1" maxOccurs="unbounded"/>
 </xs:sequence>
</xs:complexType>
```

7.46 type.feedback.interaction.correction.hint

Single hint to assist in text correction.

7.46.1 Attributes

None.

7.46.2 Elements

Element name	Description
Component	wf:type.feedback.interaction.correction.component The component which is represented by the text e.g. the MRZ.
ExpectedLength?	xs:positiveInteger The server MAY send the expected length for the text.

Element name	Description
PositionHint?	wf:type.feedback.interaction.correction.hints.position The server MAY send position information if it can determine more precise information about the error within the text e.g. a specific part of the text or a single character.

Table 7.42 .

7.46.3 XSD Definition

```
<xs:complexType name="type.feedback.interaction.correction.hint">
 <xs:sequence>
  <xs:element name="Component" type="wf:type.feedback.interaction.correction.component"/>
  <xs:element name="ExpectedLength" type="xs:positiveInteger"
    minOccurs="0" maxOccurs="1"/>
  <xs:element name="PositionHint" type="wf:type.feedback.interaction.correction.hints.position"
    minOccurs="0" maxOccurs="1"/>
 </xs:sequence>
</xs:complexType>
```

7.47 type.feedback.interaction.correction.component

List of text correction components.

7.47.1 Values

The type.type.feedback.interaction.correction.component values are given in Table 7.43.

Value	Description
CAN	Reference to the CAN.
MRZ	Reference to the optical MRZ.
MRZ line 1	Reference to the first line of the MRZ.
MRZ line 2	Reference to the second line of the MRZ.
MRZ line 3	Reference to the third line of the MRZ.
document type	Reference to the document type from the MRZ.
issuer	Reference to the issuer from the MRZ.
given name	Reference to the given name from the MRZ.
surname	Reference to the surname from the MRZ.
nationality	Reference to the nationality from the MRZ.
date of birth	Reference to the date of birth from the MRZ.
sex	Reference to the sex from the MRZ.
date of expiry	Reference to the date of expiry from the MRZ.
document number	Reference to the document number from the MRZ.
optional data	Reference to the optional data from the MRZ.
check digit composite	Reference to the composite check digit from the MRZ.
check digit date of birth	Reference to the date of birth check digit from the MRZ.
check digit date of expiry	Reference to the date of expiry check digit from the MRZ.
check digit document number	Reference to the document number check digit from the MRZ.
check digit optional data	Reference to the optional data check digit from the MRZ.

Table 7.43 type.feedback.interaction.correction.component - values

7.47.2 XSD Definition

```
<xs:simpleType name="type.feedback.interaction.correction.component">
 <xs:restriction base="xs:string">
  <xs:enumeration value="CAN"/>
  <xs:enumeration value="MRZ"/>
  <xs:enumeration value="MRZ line 1"/>
  <xs:enumeration value="MRZ line 2"/>
  <xs:enumeration value="MRZ line 3"/>
  <xs:enumeration value="document type"/>
  <xs:enumeration value="issuer"/>
  <xs:enumeration value="given name"/>
  <xs:enumeration value="surname"/>
  <xs:enumeration value="document number"/>
  <xs:enumeration value="nationality"/>
  <xs:enumeration value="date of birth"/>
  <xs:enumeration value="sex"/>
  <xs:enumeration value="date of expiry"/>
  <xs:enumeration value="optional data"/>
  <xs:enumeration value="check digit document number"/>
  <xs:enumeration value="check digit date of birth"/>
  <xs:enumeration value="check digit date of expiry"/>
  <xs:enumeration value="check digit optional data"/>
  <xs:enumeration value="check digit composite"/>
 </xs:restriction>
</xs:simpleType
```

7.48 type.feedback.interaction.correction.hints.position

Contains position information to assist in text correction.

7.48.1 Attributes

None.

7.48.2 Elements

Element name	Description
TextPosition	wf:type.feedback.interaction.correction.hints.positi-on.text Position information within the given text.
ImagePosition?	wf:type.feedback.image.position.rect Position information within the image if an image is the source of the text.

Table 7.44 .

7.48.3 XSD Definition

```
<xs:complexType name="type.feedback.interaction.correction.hints.position">
 <xs:sequence>
  <xs:element name="TextPosition"
    type="wf:type.feedback.interaction.correction.hints.position.text"/>
  <xs:element name="ImagePosition" type="wf:type.feedback.image.position.rect"
    minOccurs="0" maxOccurs="1"/>
 </xs:sequence>
</xs:complexType>
```

7.49 type.feedback.interaction.correction.hints.position.text

Contains position information within the given text to assist in text correction.

7.49.1 Attributes

None.

7.49.2 Elements

Element name	Description
StartIndex	xs:positiveInteger Index of the first character in the text segment.
EndIndex	xs:positiveInteger Index of the last character in the text segment.
CheckDigitIndex?	xs:positiveInteger Index of the checkdigit in the text segment if there is an corresponding checkdigit for the text.

Table 7.45 .

7.49.3 XSD Definition

```
<xs:complexType name="type.feedback.interaction.correction.hints.position.text">
 <xs:sequence>
  <xs:element name="StartIndex" type="xs:positiveInteger"/>
  <xs:element name="EndIndex" type="xs:positiveInteger"/>
  <xs:element name="CheckDigitIndex" type="xs:positiveInteger" minOccurs="0" maxOccurs="1"/>
 </xs:sequence>
</xs:complexType>
```

7.50 type.feedback.interaction.seriesselection

Contains request information for the document series selection.

7.50.1 Attributes

None.

7.50.2 Elements

Element name	Description
ScannedDocument	wf:type.feedback.interaction.seriesselection.document Information about the scanned document.
Candidates?	wf:type.feedback.interaction.seriesselection.candidates List of candidates for the document series selection.

Table 7.46 .

7.50.3 XSD Definition

```
<xs:complexType name="type.feedback.interaction.seriesselection">
 <xs:sequence>
  <xs:element name="ScannedDocument"
              type="wf:type.feedback.interaction.seriesselection.document"/>
  <xs:element name="Candidates" type="wf:type.feedback.interaction.seriesselection.candidates"
    minOccurs="0"/>
 </xs:sequence>
</xs:complexType>
```

7.51 type.feedback.interaction.seriesselection.document

Contains information about the currently scanned document in case of a requested document series selection.

7.51.1 Attributes

None.

7.51.2 Elements

Element name	Description
Country	xs:string The issuing country of the document.
DocumentSemanticType	wf:type.shared.enum.document.semantic.type The document semantic type of the document.
Images	wf:type.feedback.interaction.seriesselection.images Images of the scanned document.

Table 7.47 .

7.51.3 XSD Definition

```
<xs:complexType name="type.feedback.interaction.seriesselection.document">
 <xs:sequence>
  <xs:element name="Country" type="xs:string"/>
  <xs:element name="DocumentSemanticType" type="wf:type.shared.enum.document.semantic.type"/>
  <xs:element name="Images" type="wf:type.feedback.interaction.seriesselection.images"/>
 </xs:sequence>
</xs:complexType>
```

7.52 type.feedback.interaction.seriesselection.candidates

Contains information about the candidates for document series selection.

7.52.1 Attributes

None.

7.52.2 Elements

Element name	Description
Candidate	wf:type.feedback.interaction.seriesselection.candidate A single candidate for document series selection.

Table 7.48 .

7.52.3 XSD Definition

```
<xs:complexType name="type.feedback.interaction.seriesselection.candidates">
 <xs:sequence>
  <xs:element name="Candidate" type="wf:type.feedback.interaction.seriesselection.candidate"
    minOccurs="1" maxOccurs="unbounded"/>
 </xs:sequence>
</xs:complexType>
```

7.53 type.feedback.interaction.seriesselection.candidate

Contains information about a single candidate for document series selection.

7.53.1 Attributes

The type.feedback.interaction.seriesselection.candidate attributes are given in Table 7.49.

Attributes name	Description
id	xs:string A unique identifier for the candidate.

Table 7.49 type.feedback.interaction.seriesselection.candidate - attributes

7.53.2 Elements

Element name	Description
Hint	xs:string A hint to identify the series.
Images?	wf:type.feedback.interaction.seriesselection.images A list of images of the candidate.

Table 7.50 .

7.53.3 XSD Definition

```
<xs:complexType name="type.feedback.interaction.seriesselection.candidate">
 <xs:sequence>
  <xs:element name="Hint" type="xs:string"/>
  <xs:element name="Images" type="wf:type.feedback.interaction.seriesselection.images"
    minOccurs="0"/>
 </xs:sequence>
 <xs:attribute name="id" type="xs:string" use="required"/>
</xs:complexType>
```

7.54 type.feedback.interaction.seriesselection.images

A list of images for the document series selection.

7.54.1 Attributes

None.

7.54.2 Elements

Element name	Description
Image	wf:type.feedback.interaction.seriesselection.image A single image.

Table 7.51 .

7.54.3 XSD Definition

```
<xs:complexType name="type.feedback.interaction.seriesselection.images">
 <xs:sequence>
  <xs:element name="Image" type="wf:type.feedback.interaction.seriesselection.image"
    minOccurs="1" maxOccurs="unbounded"/>
 </xs:sequence>
</xs:complexType>
```

7.55 type.feedback.interaction.seriesselection.image

A single image for the document series selection.

7.55.1 Attributes

The type.feedback.interaction.seriesselection.image attributes are given in Table 7.52.

Attributes name	Description
attributeGroup	wf:attributeGroup.shared.document.image.classification An attribute group for image classification.

Table 7.52 type.feedback.interaction.seriesselection.image - attributes

7.55.2 Elements

None.

7.55.3 XSD Definition

```
<xs:complexType name="type.feedback.interaction.seriesselection.image">
 <xs:complexContent>
  <xs:extension base="wf:type.feedback.interaction.seriesselection.image.base">
   <xs:attributeGroup ref="wf:attributeGroup.shared.document.image.classification"/>
  </xs:extension>
 </xs:complexContent>
</xs:complexType>
```

7.56 type.feedback.interaction.seriesselection.image.base

A basic image for the document series selection. Derived from xs:base64Binary.

7.56.1 Attributes

The type.feedback.interaction.seriesselection.image.base attributes are given in Table 7.53.

Attributes name	Description
width	xs:int The width of the image.
height	xs:int The height of the image.

Table 7.53 type.feedback.interaction.seriesselection.image.base - attributes

7.56.2 Elements

None.

7.56.3 XSD Definition

```
<xs:complexType name="type.feedback.interaction.seriesselection.image.base">
 <xs:simpleContent>
  <xs:extension base="xs:base64Binary">
   <xs:attribute name="width" type="xs:int" use="required"/>
   <xs:attribute name="height" type="xs:int" use="required"/>
  </xs:extension>
 </xs:simpleContent>
</xs:complexType>
```

7.57 type.feedback.interaction.nextpage

Contains information about a single page of a document which is requested.

7.57.1 Attributes

None.

7.57.2 Elements

Element name	Description
DocumentType	wf:type.shared.enum.document.semantic.type.extended The document type which is requested.
Page	wf:type.feedback.interaction.nextpage.page The page of the document which is requested.

Table 7.54 .

7.57.3 XSD Definition

```
<xs:complexType name="type.feedback.interaction.nextpage">
 <xs:sequence>
  <xs:element name="DocumentType" type="wf:type.shared.enum.document.semantic.type.extended"/>
  <xs:element name="Page" type="wf:type.feedback.interaction.nextpage.page"/>
 </xs:sequence>
</xs:complexType>
```

7.58 type.feedback.interaction.nextpage.page

Contains information about a single page.

7.58.1 Attributes

The type.feedback.interaction.nextpage.page are given in ▸Table 7.55.

Attributes name	Description
attributeGoup	attributeGroup.shared.page.classification.extended Group of attibutes to identify a specific document page.

Table 7.55 type.feedback.interaction.nextpage.page - attributes

7.58.2 Elements

None.

7.58.3 XSD Definition

```
<xs:complexType name="type.feedback.interaction.nextpage.page">
  <xs:attributeGroup ref="wf:attributeGroup.shared.page.classification.extended"/>
</xs:complexType>
```

7.59 Workflow extensions

The workflow and feedback specification of the high-level document check allow for implementation-specific extensions. This section specifies the constraints for such extensions.

7.59.1 Workflow definition

1. All server implementations SHALL correctly validate all workflow definitions that conform to this document.
2. All implementation-specific extensions SHALL be placed below the `hldc#wf:Extension` node. To prevent conflicts between multiple extension implementations, only one extension is allowed per workflow.
3. A server SHALL validate known extensions. Any errors MAY be reported with a regular validation error message as part of the `WorkflowParserError`.

4. A server SHALL reject unknown extensions to prevent incomplete or incorrect interpretation of the workflow definition. The `WorkflowParserError` SHALL explicitly state that the extension was rejected.
5. Extension elements MAY refer to IDs in the regular workflow definition and vice versa. This allows extensions that are transparent to the client (e.g. by adding a new type of condition).

7.59.2 Feedback

1. If the workflow extension does not define custom feedback elements, the workflow and its feedback SHALL be processable by all clients that conform to this document without modification of the client.
2. All XML-based feedback SHALL be validatable by all conformant clients.
3. All feedback requests that conform to this Technical Guideline SHALL be answered by corresponding conformant feedback. Custom feedback MAY only be used for implementation-specific feedback requests.
4. Extensions MAY provide feedback as
   - modifications of regular feedback
   - text-based feedback in the `stringFeedback` field of `WorkflowFeedback`
   - binary feedback in the `binaryFeedback` field of `WorkflowFeedback`
   - Custom XML-based feedback using the `wf:ExtensionFeedback` node

8 Transaction management

The Transaction Management API provides functions for transaction-based logging of document checks in compliance to [BSI TR-03135-1].

The definitions of the Transaction Management API are provided in `tl_v5.wsdl`.

8.1 Namespace

All elements that are defined in this chapter are member of the namespace `http://trdoccheck.bsi.bund.de/tl/wsdl/5` [http://trdoccheck.bsi.bund.de/tl/wsdl/5], which is aliased by `tl5`.

8.2 Data types

In addition to simple XSD types, the SOAP interface uses custom data types, which are described in the following.

8.2.1 ExternalKey

Represents a name-value combination.

8.2.1.1 Attributes

None.

8.2.1.2 Elements

Element name	Description
`name`	`xsd:string` Name of the external key.
`value`	`xsd:string` Value of the external key.

Table 8.1 .

8.2.1.3 WSDL Definition

```
<complexType name="ExternalKey">
 <sequence>
  <element name="name" type="xsd:string" />
  <element name="value" type="xsd:string" />
 </sequence>
</complexType>
```

8.2.2 LoggingProvider

Represents a [BSI TR-03135]-compliant logging provider which saves [BSI TR-03135] XML data persistently.

8.2.2.1 Attributes

None.

8.2.2.2 Elements

Element name	Description
`name`	`xsd:string` Name of the logging provider.

Element name	Description
type	xsd:string The type of the logging provider (can be empty).

Table 8.2 .

8.2.2.3 WSDL Definition

```
<complexType name="LoggingProvider">
 <sequence>
 <element name="name" type="xsd:string"/>
 <element name="type" type="xsd:string" />
 </sequence>
</complexType>
```

8.2.3 LogType

Characterizes a subcategory of a [BSI TR-03135] log entry. Derived from `xsd:string`.

8.2.3.1 Values

Value	Description
doc-check	Document check with [BSI TR-03135]-compliant XML data.
bio-check	Biometric check with [BSI TR-03121]-compliant XML data.
background-check	Background system check with arbitrary XML data.
app-specific-check	Application specific check with arbitrary XML data.

Table 8.3 .

8.2.3.2 WSDL Definition

```
<simpleType name="LogType">
 <restriction base="xsd:string">
  <enumeration value="doc-check" />
  <enumeration value="bio-check" />
  <enumeration value="background-check" />
  <enumeration value="app-specific-check"/>
 </restriction>
</simpleType>
```

8.2.4 TR03135Version

Version of the TR03135 and its logging schema. The client SHALL use this type to choose the logging schema version.

8.2.4.1 WSDL Definition

```
<simpleType name="TR03135Version">
 <restriction base="xsd:string">
  <pattern value="\d+\.\d+\.\d+"/>
 </restriction>
</simpleType>
```

8.2.5 UUID

The definition of `t15:UUID` is identical to `hldc5:UUID` except for the namespace. Please refer to ▸Section 6.2.2 for the definition.

8.3 Fault types

This section specifies the SOAP faults that are specific to this SOAP API. No fault has any attributes.

8.3.1 InProgress

Returned by methods that prevent further modification of a transaction (read-only) if the transaction is currently written to.

8.3.1.1 WSDL Definition

```
<element name="InProgress">
  <complexType />
</element>
```

8.3.2 InvalidLoggingProvider

Returned if an invalid logging provider was requested or if no default logging provider is available.

8.3.2.1 WSDL Definition

```
<element name="InvalidLoggingProvider">
  <complexType />
</element>
```

8.3.3 InvalidLoggingParameter

Returned if the logging provider cannot interpret the provider-specific parameter.

8.3.3.1 WSDL Definition

```
<element name="InvalidLoggingParameter">
  <complexType />
</element>
```

8.3.4 InvalidTransactionId

Returned if a transaction ID does not exist on the server. The ID is either invalid or has expired due to a call to `endTransaction` or limited resources on the server.

8.3.4.1 Elements

Element name	Description
`id?`	`tl5:UUID` The rejected ID. Only present in case of ambiguity.

Table 8.4 .

8.3.4.2 WSDL Definition

```
<element name="InvalidTransactionId">
  <complexType>
    <sequence>
      <element name="id" type="tl5:UUID" minOccurs="0"
   maxOccurs="1" />
    </sequence>
  </complexType>
</element>
```

8.3.5 LoggingFailed

Returned if the server could not complete the logging procedure.

8.3.5.1 WSDL Definition

```
<element name="LoggingFailed">
  <complexType />
</element>
```

8.3.6 LoggingProfileNotFound

Returned if the requested logging profile does not exist on the server.

8.3.6.1 WSDL Definition

```
<element name="LoggingProfileNotFound">
  <complexType />
</element>
```

8.3.7 LogParserError

Returned if the server could not parse submitted log XML or logging profile XSLT.

8.3.7.1 WSDL Definition

```
<element name="LogParserError">
  <complexType />
</element>
```

8.3.8 ReadOnly

Returned if the transaction is read-only and must not be modified.

8.3.8.1 WSDL Definition

```
<element name="ReadOnly">
  <complexType />
</element>
```

8.3.9 InvalidVersion

Returned if the client starts a transaction with a version which is not supported by the inspection system.

8.3.9.1 WSDL Definition

```
<element name="InvalidVersion">
  <complexType />
</element>
```

8.4 Operations

8.4.1 addLogData

Allows the client application to add custom log data to a transaction.

8.4.1.1 Request elements

Element name	Description
`transactionID`	`tl5:UUID` The transaction ID to which the data should be copied.
`logType`	`tl5:LogType` Specifies the type of the log data.

Element name	Description
`content`	`xsd:string` The xml-formatted log data.

Table 8.5 .

8.4.1.2 Response elements

None.

8.4.1.3 Faults

Fault type	Cause
`tl5:InvalidTransactionId`	Value of transactionID is invalid or has expired.
`tl5:LogParserError`	Value of content could not be parsed or validated.
`tl5:ReadOnly`	The transaction is write-protected and must not be modified.

Table 8.6 .

8.4.1.4 WSDL Definition

```
<!-- operation request element -->
<element name="addLogData">
 <complexType>
  <sequence>
   <element name="transactionID" type="tl5:UUID" />
   <element name="logType" type="tl5:LogType" />
   <element name="content" type="xsd:string" />
  </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="addLogDataResponse">
 <complexType />
</element>
```

8.4.2 addLoggingProfile

Transfers a new logging profile definition to the server. The profile must be a valid XSL transformation that operates on a transaction log according to Part 1 of this Technical Guideline. If an identically named logging profile exists on the server, it SHALL be replaced with the newly submitted definitions.

8.4.2.1 Request elements

Element name	Description
`name`	`xsd:string` Reference to the profile for use with applicable SOAP calls.
`profile`	`xsd:base64Binary` The base64-encoded XSLT document.

Table 8.7 .

8.4.2.2 Response elements

None.

8.4.2.3 Faults

Fault type	Cause
t15:LogParserError	An error occurred while validating the submitted logging profile.

Table 8.8 .

8.4.2.4 WSDL Definition

```
<!-- operation request element -->
<element name="addLoggingProfile">
 <complexType>
  <sequence>
   <element name="name" type="xsd:string" />
   <element name="profile" type="xsd:base64Binary" />
  </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="addLoggingProfileResponse">
 <complexType />
</element>
```

8.4.3 getSupportedVersions

Returns a list of supported TR-03135 versions. One of the Versions SHALL be used by the client to begin a transaction.

8.4.3.1 Request elements

None.

8.4.3.2 Response elements

Element name	Description
version+	t15:TR03135Version A single version information.

Table 8.9 .

8.4.3.3 Faults

None.

8.4.3.4 WSDL Definition

```
<!-- operation request element -->
<element name="getSupportedVersions">
 <complexType/>
<element>

<!-- operation response element -->
<element name="getSupportedVersionsResponse">
 <complexType>
  <sequence>
   <element name="version" type="t15:TR03135Version" minOccurs="1" maxOccurs="unbounded"/>
  <sequence>
 <complexType>
<element>
```

8.4.4 beginTransaction

Opens a [BSI TR-03135] transaction and returns a new transaction ID.

8.4.4.1 Request elements

Element name	Description
version	t15:TR03135Version A TR-03135 version element. The client SHALL select a version to specify the logging schema which will be used.
location	xsd:string The location of the inspection system. Copied into the [BSI TR-03135] XML data.
operationalEnvironment	xsd:string The operational environment which is copied into the TR-03135 XML data. Examples: entry, exit, domestic, training, mobile, undefined.
externalKey*	t15:ExternalKey External keys to be copied into the [BSI TR-03135] XML data.

Table 8.10 .

8.4.4.2 Response elements

Element name	Description
transactionID	t15:UUID Automatically generated unique ID for the new transaction. This ID will be used as a reference to this transaction.

Table 8.11 .

8.4.4.3 Faults

Fault type	Cause
t15:InvalidVersion	Value of version is invalid or not supported by the inspection system.

Table 8.12 .

8.4.4.4 WSDL Definition

```
<!-- operation request element -->
<element name="beginTransaction">
 <complexType>
  <sequence>
   <xsd:element name="version" type="t15:TR03135Version"/>
   <element name="location" type="xsd:string" />
   <element name="operationalEnvironment" type="xsd:string" />
   <element name="externalKey" type="t15:ExternalKey"
        minOccurs="0" maxOccurs="unbounded"/>
  </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="beginTransactionResponse">
 <complexType>
  <sequence>
   <element name="transactionID" type="t15:UUID" />
  </sequence>
 </complexType>
</element>
```

8.4.5 endTransaction

Closes a transaction. Invalidates the corresponding transaction ID.

8.4.5.1 Request elements

Element name	Description
`transactionID`	`t15:UUID` ID of the transaction to end.

Table 8.13 .

8.4.5.2 Response elements

None.

8.4.5.3 Faults

Fault type	Cause
`t15:InvalidTransactionId`	Value of transactionID is invalid or has expired.

Table 8.14 .

8.4.5.4 WSDL Definition

```
<!-- operation request element -->
<element name="endTransaction">
 <complexType>
  <sequence>
   <element name="transactionID" type="tl:UUID" />
  </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="endTransactionResponse">
 <complexType />
</element>
```

8.4.6 getAllLoggingProviders

Returns a list of all available logging providers. This function is OPTIONAL.

8.4.6.1 Request elements

None.

8.4.6.2 Response elements

Element name	Description
`loggingProvider*`	`t15:LoggingProvider` Available logging providers.

Table 8.15 .

8.4.6.3 Faults

None.

8.4.6.4 WSDL Definition

```
<!-- operation request element -->
<element name="getAllLoggingProviders">
 <complexType />
</element>

<!-- operation response element -->
<element name="getAllLoggingProvidersResponse">
```

```
  <complexType>
   <sequence>
    <element name="loggingProvider" type="tl5:LoggingProvider"
         minOccurs="0" maxOccurs="unbounded"/>
   </sequence>
  </complexType>
 </element>
```

8.4.7 getTransactionXML

Returns the [BSI TR-03135]-compliant XML data of the corresponding transaction. If called, SHALL be called between `beginTransaction` and `endTransaction`.

If the transaction is currently being modified, e.g., by the document check process, the server MAY fail the call with `InProgress` fault.

On success, the transaction SHALL be flagged as read-only to prevent further modification, if the `keepWritable` is not present or has the value `false`, otherwise – if the `keepWritable` attribute has the value `true`, the transaction MUST be kept open for further writes.

8.4.7.1 Request elements

Element name	Description
`transactionID`	`tl5:UUID` A valid transaction ID.
`errorCode`	`xsd:int` Application-defined error code which is copied to the [BSI TR-03135]-XML. If there was no error, the `errorCode` SHALL be set to 0.
`loggingProfile`	`xsd:string` Name of the logging profile.
`keepWritable?`	`xsd:boolean` Whether the transaction XML is kept open for further writes.

Table 8.16 .

8.4.7.2 Response elements

Element name	Description
`xml`	`xsd:string` The [BSI TR-03135]-compliant XML data of the specified transaction.

Table 8.17 .

8.4.7.3 Faults

Fault type	Cause
`tl5:InProgress`	The transaction is currently being written to and cannot be write-protected.
`tl5:InvalidTransactionId`	Va.lue of `transactionID` is invalid or has expired.
`tl5:LoggingProfileNotFound`	The requested logging profile `loggingProfile` does not exist on the server.

Table 8.18 .

8.4.7.4 WSDL Definition

```
<!-- operation request element -->
<element name="getTransactionXML">
 <complexType>
```

```
  <sequence>
   <element name="transactionID" type="tl5:UUID" />
   <element name="errorCode" type="xsd:int" />
   <element name="loggingProfile" type="xsd:string" />
   <element name="keepWritable" type="xsd:boolean" minOccurs="0" />
  </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="getTransactionXMLResponse">
 <complexType>
  <sequence>
   <element name="xml" type="xsd:string" />
   </sequence>
 </complexType>
</element>
```

8.4.8 mergeTransaction

Merges contents of the source transaction into the target transaction and ends the source transaction afterwards.

8.4.8.1 Request elements

Element name	Description
`targetID`	`tl5:UUID` Transaction ID of the target transaction.
`sourceID`	`tl5:UUID` Transaction ID of the source transaction.

Table 8.19 .

8.4.8.2 Response elements

None.

8.4.8.3 Faults

Fault type	Cause
`tl5:InProgress`	The transaction `sourceID` is currently being written to and cannot be write-protected.
`tl5:InvalidTransactionId`	Either the value of `targetID` or `sourceID` is invalid or has expired
`tl5:ReadOnly`	The transaction `targetID` is write-protected and must not be modified.

Table 8.20 .

8.4.8.4 WSDL Definition

```
<!-- operation request element -->
<element name="mergeTransaction">
 <complexType>
  <sequence>
   <element name="targetID" type="tl5:UUID" />
   <element name="sourceID" type="tl5:UUID" />
  </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="mergeTransactionResponse">
 <complexType />
</element>
```

8.4.9 removeLoggingProfile

Removes a logging profile which was previously loaded with `addLoggingProfile`.

8.4.9.1 Request elements

Element name	Description
`profileName`	`xsd:string` The name of the logging profile to remove.

Table 8.21 .

8.4.9.2 Response elements

None.

8.4.9.3 Faults

Fault type	Cause
`tl5:LoggingProfileNotFound`	The logging profile `profileName` does not exist on the server

Table 8.22 .

8.4.9.4 WSDL Definition

```
<!-- operation request element -->
<element name="removeLoggingProfile">
 <complexType>
  <sequence>
   <element name="profileName" type="xsd:string" />
 </sequence>
 </complexType>
</element>

<!-- operation response element -->
<element name="removeLoggingProfileResponse">
 <complexType />
</element>
```

8.4.10 saveTransaction

Saves the current transaction state as [BSI TR-03135]-compliant XML data the via a logging provider. If called, SHALL be called between `beginTransaction` and `endTransaction`.

In order to save log data from an HLDC5 workflow, it is RECOMMENDED to call `saveTransaction` only after workflow execution has stopped to ensure that the log data is complete with respect to the workflow.

Workflow execution has stopped if either of the following is true:

- `getWorkflowFeedback` or `getWorkflowFeedbackById` return `workflowStatus` finished or cancelled
- `cancelWorkflow` was called
- `endWorkflow` was called.

Otherwise, the server MAY fail the call with `InProgress` fault.

On success, the transaction SHALL be flagged as read-only to prevent further modification.

The server MAY log the data asynchronously and return the call of `saveTransaction` before the actual logging has finished.

8.4.10.1 Request elements

Element name	Description
transactionID	t15:UUID A valid transaction ID.
provider?	t15:string The name of the logging provider which is used for the save operation. If empty, the server MAY use a default logging provider. The names of the logging providers are server-defined. They can be queried if the server provides the optional function getAllLoggingProviders.
providerParam?	xsd:string Provider-specific parameter. MAY be empty. The server MAY return an error if the value of providerParam is invalid for provider. The server SHALL accept empty providerParam for the default logging provider. The server MAY ignore providerParam if provider is empty.
errorCode	xsd:int Application-defined error code which is copied to the [BSI TR-03135]-XML. If there was no error, the errorCode SHALL be set to 0.
loggingProfile	xsd:string Name of the logging profile to use.

Table 8.23 .

8.4.10.2 Response elements

None.

8.4.10.3 Faults

Fault type	Cause
t15:InProgress	The transaction is currently being written to and cannot be write-protected.
t15:InvalidLoggingParameter	Value of providerParam is invalid for the selected provider.
t15:InvalidLoggingProvider	Value of provider is invalid.
t15:InvalidTransactionId	Value of transactionID is invalid or has expired.
t15:LoggingFailed	The server was unable to store the data. This fault MAY be omitted if the server processes the logging call asynchronously.
t15:LoggingProfileNotFound	The requested logging profile loggingProfile does not exist on the server.

Table 8.24 .

8.4.10.4 WSDL Definition

```
<!-- operation request element -->
<element name="saveTransaction">
 <complexType>
  <sequence>
   <element name="transactionID" type="t15:UUID" />
   <element name="provider" type="xsd:string"
    minOccurs="0" maxOccurs="1" />
   <element name="providerParam" type="xsd:string"
    minOccurs="0" maxOccurs="1" />
   <element name="errorCode" type="xsd:int" />
   <element name="loggingProfile" type="xsd:string" />
  </sequence>
 </complexType>
</element>
```

```
<!-- operation response element -->
<element name="saveTransactionResponse">
 <complexType />
</element>
```

8.4.11 saveTransactionXML

Saves a complete transaction XML document via a logging provider.

The server MAY log the data asynchronously and return the call of `saveTransactionXML` before the actual logging has finished.

8.4.11.1 Request elements

Element name	Description
`transactionXML`	`xsd:string` A valid XML-formatted log file according to [BSI TR-03135-1].
`containsExtendedData?`	`xsd:boolean` Signifies if the XML document contains extended data according to version 1.x of [BSI TR-03135], if the XML complies to version 1.x of [BSI TR-03135].
`provider?`	`xsd:string` The name of the logging provider which is used for the save operation. If empty, the server MAY use a default logging provider. The names of the logging providers are server-defined. They can be queried if the server provides the optional function `getAllLoggingProviders`.
`providerParam?`	`xsd:string` Provider-specific parameter. MAY be empty. The server MAY return an error if the value of `providerParam` is invalid for `provider`. The server SHALL accept empty `providerParam` for the default logging provider. The server MAY ignore `providerParam` if `provider` is empty.

Table 8.25 .

8.4.11.2 Response elements

None.

8.4.11.3 Faults

Fault type	Cause
`tl5:InvalidLoggingParameter`	Value of `providerParam` is invalid for the selected `provider`.
`tl5:InvalidLoggingProvider`	Value of provider is invalid.
`tl5:LoggingFailed`	The server was unable to store the data. This fault MAY be omitted if the server processes the logging call asynchronously.

Table 8.26 .

8.4.11.4 WSDL Definition

```
<!-- operation request element -->
<element name="saveTransactionXML">
 <complexType>
  <sequence>
   <element name="transactionXML" type="xsd:string" />
   <element name="containsExtendedData" type="xsd:boolean"
    minOccurs="0" maxOccurs="1" />
   <element name="provider" type="xsd:string"
     minOccurs="0" maxOccurs="1" />
   <element name="providerParam" type="xsd:string"
    minOccurs="0" maxOccurs="1" />
```

```
   </sequence>
  </complexType>
</element>

<!-- operation response element -->
<element name="saveTransactionXMLResponse">
  <complexType />
</element>
```

8.4.12 setSystemInformation

Set system information according to the given values.

8.4.12.1 Request elements

Element name	Description
transactionID	t15:UUID A valid transaction ID.
vendor	xsd:string The vendor name to be set.
name	xsd:string The system name to be set.
version	xsd:string The version to be set.

Table 8.27 .

8.4.12.2 Response elements

None.

8.4.12.3 Faults

Fault type	Cause
t15:InvalidTransactionId	Value of transactionID is invalid or has expired.
t15:ReadOnly	The transaction is write-protected and must not be modified.

Table 8.28 .

8.4.12.4 WSDL Definition

```
<!-- operation request element -->
<element name="setSystemInformation">
  <complexType>
   <sequence>
    <element name="transactionID" type="t15:UUID" />
    <element name="vendor" type="xsd:string" />
    <element name="name" type="xsd:string" />
    <element name="version" type="xsd:string" />
   </sequence>
  </complexType>
</element>

<!-- operation response element -->
<element name="setSystemInformationResponse">
  <complexType />
</element>
```

9 Shared data

The schemas are using shared datatypes.

The definitions of the shared data types are provided in `hldc_shared_v5.xsd` and `log_hldc_shared-v5_0_0.xsd`.

9.1 Overall shared types

The elements that are defined in this section are shared within other schemas, this includes the webservice and the logging schemas.

The definitions of the shared data are provided in `log_hldc_shared-v5_0_0.xsd`.

9.1.1 type.shared.enum.document.semantic.type.specific

List of types of documents. Derived from xs:string.

9.1.1.1 Values

The type.shared.enum.document.semantic.type.specific values are given in ▸Table 9.1.

Value	Description
`passport`	A passport issued by a country or supra-national organization for its citizens / members.
`identity_card`	An identity card issued by a country for its citizens.
`passport_substitute`	A passport substitute issued by a country for person of other or no citizenship.
`residence_permit`	A residence permit issued by a country.
`residence_card`	A residence card for family members of European Economic Area (EEA) nationals.
`visa`	A visa issued by a country.
`drivers_licence`	A licence that entitles a person to move a vehicle.
`insurance_card`	A card issued by an insurance.
`birth_certificate`	A birth certificate issued by a country.

Table 9.1 type.shared.enum.document.semantic.type.specific - values

9.1.1.2 XSD Definition

```
<xs:simpleType name="type.shared.enum.document.semantic.type.specific">
 <xs:restriction base="xs:string">
  <xs:enumeration value="passport"/>
  <xs:enumeration value="identity_card"/>
  <xs:enumeration value="passport_substitute"/>
  <xs:enumeration value="residence_permit"/>
  <xs:enumeration value="residence_card"/>
  <xs:enumeration value="visa"/>
  <xs:enumeration value="drivers_licence"/>
  <xs:enumeration value="insurance_card"/>
  <xs:enumeration value="birth_certificate"/>
 </xs:restriction>
</xs:simpleType>
```

9.1.2 type.shared.enum.document.semantic.type.unidentified

List of additional results for document semantic type identification. Derived from xs:string.

9.1.2.1 Values

The type.shared.enum.document.semantic.type.unidentified values are given in ▸Table 9.2.

Value	Description
other	Any other document that does not fit the specific document type enumeration.
unknown	The document check was aborted before the semantic document type could be determined.

Table 9.2 type.shared.enum.document.semantic.type.unidentified - values

9.1.2.2 XSD Definition

```
<xs:simpleType name="type.shared.enum.document.semantic.type.unidentified">
 <xs:restriction base="xs:string">
  <xs:enumeration value="other"/>
  <xs:enumeration value="unknown"/>
 </xs:restriction>
</xs:simpleType>
```

9.1.3 type.shared.enum.document.semantic.type

List of results for document semantic type identification.

9.1.3.1 XSD Definition

```
<xs:simpleType name="type.shared.enum.document.semantic.type">
 <xs:union memberTypes="type.shared.enum.document.semantic.type.specific
   type.shared.enum.document.semantic.type.unidentified"/>
</xs:simpleType>
```

9.1.4 type.shared.errorcode

If an error occurred during the step, this element SHALL be present.

9.1.4.1 XSD Definition

```
<xs:simpleType name="type.shared.errorcode">
 <xs:union memberTypes="type.shared.errorcode.predefined type.shared.errorcode.proprietary"/>
</xs:simpleType>
```

9.1.5 type.shared.errorcode.predefined

List of predefined error codes. Derived from xs:int.

9.1.5.1 Values

The type.shared.errorcode.predefined values are given in Table 9.3.

Value	Description
0	This error code SHALL be used if an error code is mandatory, but no error occurred.
-1	This error code MAY be used if an unexpected error occurred. A more detailed predefined error code SHALL be used if possible. It is RECOMMENDED to define and use a more detailed proprietary error codes if needed.
-2	This error code SHALL be used if no document reader is available.
-3	This error code SHALL be used when the document was removed prematurely.
-10000	This error code SHALL be used if no document reader with optical capabilities is available, but optical processing is required according to the workflow.
-10001	This error code SHALL be used if the document reader does not support live streaming, but streaming is activated according to the workflow.

Value	Description
-10002	This error code SHALL be used if the optical reading may be influenced by too much ambient light.
-20000	This error code SHALL be used if no document reader with electronic capabilities is available, but electronic processing is required according to the workflow.
-20001	This error code SHALL be used if accessing the chip is not possible, because of a chip sharing violation.
-30000	This error code SHALL be used if no PKI is available, but a PKI is required for document processing according to the workflow.

Table 9.3 type.shared.errorcode.predefined - values

9.1.5.2 XSD Definition

```
<xs:simpleType name="type.shared.errorcode.predefined">
 <xs:restriction base="xs:int">
  <xs:enumeration value="0"/>
  <xs:enumeration value="-1"/>
  <xs:enumeration value="-2"/>
  <xs:enumeration value="-3"/>
  <xs:enumeration value="-10000"/>
  <xs:enumeration value="-10001"/>
  <xs:enumeration value="-10002"/>
  <xs:enumeration value="-20000"/>
  <xs:enumeration value="-20001"/>
  <xs:enumeration value="-30000"/>
 </xs:restriction>
</xs:simpleType>
```

9.1.6 type.shared.errorcode.proprietary

Allows the usage of proprietary error codes. All proprietary error codes SHALL be positive integers.

9.1.6.1 XSD Definition

```
<xs:simpleType name="type.shared.errorcode.proprietary">
 <xs:restriction base="xs:int">
  <xs:minInclusive value="1"/>
 </xs:restriction>
</xs:simpleType>
```

9.1.7 type.shared.enum.page.class

List of page classes. Derived from xs:string.

9.1.7.1 Values

The type.shared.enum.page.class values are given in ▸Table 9.4.

Value	Description
outside_front_cover	The outside of the front cover in case of booklets.
inside_front_cover	The inside of the front cover in case of booklets.
recto_identity	The recto side of the identity data page. The recto side of an identity data page is considered to be the page which contains the primary facial image (e.g. passport or identity card).
verso_identity	The verso side of the identity data page (i.e. the backside of the recto page).
inner_page	An inner page in case of booklets.
outside_back_cover	The outside of the back cover in case of booklets.

Value	Description
inside_back_cover	The inside of the back cover in case of booklets.

Table 9.4 type.shared.enum.page.class - values

9.1.7.2 XSD Definition

```
<xs:simpleType name="type.shared.enum.page.class">
 <xs:restriction base="xs:string">
  <xs:enumeration value="outside_front_cover"/>
  <xs:enumeration value="inside_front_cover"/>
  <xs:enumeration value="recto_identitiy"/>
  <xs:enumeration value="verso_identity"/>
  <xs:enumeration value="inner_page"/>
  <xs:enumeration value="outside_back_cover"/>
  <xs:enumeration value="inside_back_cover"/>
 </xs:restriction>
</xs:simpleType>
```

9.1.8 type.enum.shared.ssv.spectrum

List of light spectra. Derived from xs:string.

9.1.8.1 Values

The type.enum.shared.ssv.spectrums values are given in ▸Table 9.5.

Value	Description
IR	Infrared light.
UV	Ultraviolet light.
VI	Visible light.

Table 9.5 type.enum.shared.ssv.spectrum - values

9.1.8.2 XSD Definition

```
<xs:simpleType name="type.enum.shared.ssv.spectrum">
 <xs:restriction base="xs:string">
  <xs:enumeration value="IR"/>
  <xs:enumeration value="UV"/>
  <xs:enumeration value="VI"/>
 </xs:restriction>
</xs:simpleType>
```

9.1.9 attributeGroup.shared.lightsource

Attribute group for a light source.

.

9.1.9.1 Attributes

Attribute name	Description
spectrum	type.enum.shared.ssv.spectrum The spectrum of the light source.
polarAngleDegrees	xs:int The polar angle in degrees. Restricted from 0 to 180 degrees.
azimuthAngleDegrees	xs:int The azimuth angle in degrees. Restricted from 0 to 359 degrees.

Table 9.6 .

9.1.9.2 XSD Definition

```
<xs:attributeGroup name="attributeGroup.shared.lightsource">
 <xs:attribute name="spectrum" type="type.enum.shared.ssv.spectrum"/>
 <xs:attribute name="polarAngleDegrees">
  <xs:simpleType>
   <xs:restriction base="xs:int">
    <xs:minInclusive value="0"/>
    <xs:maxInclusive value="180"/>
   </xs:restriction>
  </xs:simpleType>
 </xs:attribute>
 <xs:attribute name="azimuthAngleDegrees">
  <xs:simpleType>
   <xs:restriction base="xs:int">
    <xs:minInclusive value="0"/>
    <xs:maxInclusive value="359"/>
   </xs:restriction>
  </xs:simpleType>
 </xs:attribute>
</xs:attributeGroup>
```

9.2 Interface shared types

The elements that are defined in this section are only used within the webservice, the workflow and feedback schema, the logging schemas are not sharing these elements.

The definitions of the shared data are provided in `hldc_shared_v5.xsd`.

9.2.1 type.shared.enum.page.class.any

Additional enumeration value to be able to define an arbitrary page. Derived from xs:string.

9.2.1.1 Values

The type.shared.enum.page.class.any values are given in Table 9.7.

Value	Description
any	Reference to a non-specific page. The client MAY use this value for request of not further specified pages.

Table 9.7 type.shared.enum.page.class.any - values

9.2.1.2 XSD Definition

```
<xs:simpleType name="type.shared.enum.page.class.any">
 <xs:restriction base="xs:string">
  <xs:enumeration value="any"/>
 </xs:restriction>
</xs:simpleType>
```

9.2.2 type.shared.enum.page.class.extended

Union which extends specific page classes by non-specific pages.

9.2.2.1 XSD Definition

```
<xs:simpleType name="type.shared.enum.page.class.extended">
 <xs:union memberTypes="type.shared.enum.page.class type.shared.enum.page.class.any"/>
</xs:simpleType>
```

9.2.3 attributeGroup.shared.page.classification.extended

Attribute group for extended page classification.

.

9.2.3.1 Attributes

Attribute name	Description
pageClass	type.shared.enum.page.class.extended Extended page class for specific and non-specific document pages.
innerPageNumber	xs:positiveInteger If the class is inner_page the attribute innerPageNumber SHALL contain the page number visible on the page.

Table 9.8 .

9.2.3.2 XSD Definition

```
<xs:attributeGroup name="attributeGroup.shared.page.classification.extended">
 <xs:attribute name="pageClass" type="type.shared.enum.page.class.extended" use="required">
 </xs:attribute>
 <xs:attribute name="innerPageNumber" type="xs:positiveInteger">
 </xs:attribute>
</xs:attributeGroup>
```

9.2.4 type.shared.enum.document.semantic.type.extended

Extended document type list to be able to request any not predefined document types.

9.2.4.1 XSD Definition

```
<xs:simpleType name="type.shared.enum.document.semantic.type.extended">
 <xs:union memberTypes="type.shared.enum.document.semantic.type.specific xs:string"/>
</xs:simpleType>
```

List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
BAC	Basic Access Control
CA	Chip Authentication
CAN	Card Access Number
DTC	Digital Travel Credential
EAC	Extended Access Control
EEA	European Economic Area
eID	Electronic Identity Document
HLDC	High Level Document Check
ICAO	International Civil Aviation Organization
ID	Identity
MRTD	Machine Readable Travel Document
MRZ	Machine Readable Zone
OCR	Optical Character Recognition
OID	Object Identifier
PACE	Password Authenticated Connection Establishment
PNG	Portable Network Graphics
RFID	Radio-Frequency Identification
SOAP	Simple Object Access Protocol
TA	Terminal Authentication
VIZ	Visual Zone
WSDL	Web Services Description Language
XML	Extensible Markup Language
XSD	XML Schema Definition
XSLT	Extensible Stylesheet Language Transformation

Bibliography

[BSI TR-03110] *Bundesamt für Sicherheit in der Informationstechnik (BSI): Technische Richtlinie TR-03110 – Advanced Security Mechanisms for Machine Readable Travel Documents, Part 1 & 3, 2012, Version 2.10.*

[BSI TR-03121] *Bundesamt für Sicherheit in der Informationstechnik (BSI): Technische Richtlinie TR-03121 - Biometrics in public sector applications.*

[BSI TR-03129-2] *Protocols for the Management of Certificates and CRLs in Public-Key-Infrastructures (PKIs) - Part 2: Supplemental specifications for public and official authorities, Version 1.4.1.*

[BSI TR-03135] *BSI TR-03135, Technical Guideline Machine Authentication of MRTDs for Public Sector Applications, Version 5.0.0.*

[BSI TR-03135-1] *BSI TR-03135-1, Technical Guideline Machine Authentication of MRTDs for Public Sector Applications. Part 1: Overview and Functional Requirements, Version 5.0.0.*

[ICAO9303] *ICAO Doc 9303 – Machine Readable Travel Documents, 8th Edition, 2021.*

[KeepAChangelog] *Keep a Changelog – https://keepachangelog.com/en/1.0.0/.*

[RFC2119] *Request For Comments Editor (RFC), Bradner, Scott: Key words for use in RFCs to indicate requirement levels, RFC 2119, 1997.*